StorePilot

Android Store Management App

Project Documentation Book

Ahmad Hijazy

2025

Android	Java	Room DB	Firebase	MVVM

2 Table of Contents

Conclusion & Appendices

5 Introduction

What is StorePilot?

StorePilot is a fully offline-capable Android store management application built for small-to-medium retail businesses. It provides two distinct user experiences within a single APK: a **Customer** shopping interface and a **Manager/Owner** back-office dashboard. All data is persisted locally using Room (SQLite), with optional Firebase Authentication and Firestore integration for cloud sync and low-stock push notifications.

The project demonstrates a production-quality MVVM architecture on Android, including role-based access control, a full shopping cart & checkout flow, real-time support chat, task management, inventory control, sales & purchase history, and season-aware dashboard alerts.

Core Features

- Dual-role app: Customer shopping + Manager back-office in one APK
- Room (SQLite) local database — fully offline capable
- PBKDF2/SHA-256 password hashing with random salt
- Role-Based Access Control: OWNER, STORE_MANAGER, SHIFT_MANAGER, EMPLOYEE, CUSTOMER
- Full shopping cart, checkout flow and order tracking (PENDING → DELIVERED)
- Real-time support chat between customers and store staff
- Task management with priority levels (LOW / MEDIUM / HIGH) and private tasks
- Inventory management: add, edit, delete products with live low-stock alerts
- Sales and purchase history with date-range SQL queries
- Season management with end-date alerts on the dashboard
- Firebase Authentication (optional) + Firestore low-stock push notification
- LiveData + ViewModel — reactive UI that survives screen rotation
- Demo seed data for first-run experience

6 Quick Info

App Name	StorePilot
Package	com.storepilot
Version	1.0 (versionCode 1)
Min SDK	API 24 (Android 7.0 Nougat)
Target SDK	API 34 (Android 14)
Language	Java 11
Architecture	MVVM (Model-View-ViewModel)
Database	Room (SQLite) — storepilot.db — v2
Auth	Local PBKDF2/SHA-256 + optional Firebase Auth
Build System	Gradle (AGP 8.x), Gradle wrapper 8.4
Repository	github.com/ahmadhijazys2/storepilot
Author	Ahmad Hijazy
Year	2025
Roles Supported	CUSTOMER, EMPLOYEE, SHIFT_MANAGER, STORE_MANAGER, OWNER
Demo Credentials	owner / ahmad123 manager / sss123 customer / demo123

7 Technology Stack

Library / Technology	Version	Purpose
Android SDK	API 24-34	Core mobile platform
Java 11	OpenJDK 11	Primary language
Room	2.6.1	SQLite ORM — entities, DAOs, migrations
LiveData	2.7.0	Observable data holders for reactive UI
ViewModel	2.7.0	Lifecycle-aware UI state holders
Navigation	2.7.6	Fragment navigation component
RecyclerView	1.3.2	Scrollable list views
Material UI	1.11.0	Bottom nav, cards, FAB, tabs, snackbar
ConstraintLayout	2.1.4	Flexible XML layouts
CardView	1.0.0	Elevated card containers
Firebase Auth	22.3.1	Optional cloud authentication
Firebase Firestore	24.11.0	Optional cloud DB for low-stock alerts
PBKDF2/SHA-256	Java Crypto	Password hashing — 65536 iterations, 256-bit key
AlarmManager	Android SDK	Periodic low-stock check — 1 h interval
NotificationMgr	Android SDK	Low-stock push notifications
ViewBinding	AGP 8.x	Type-safe view binding (enabled in build.gradle)

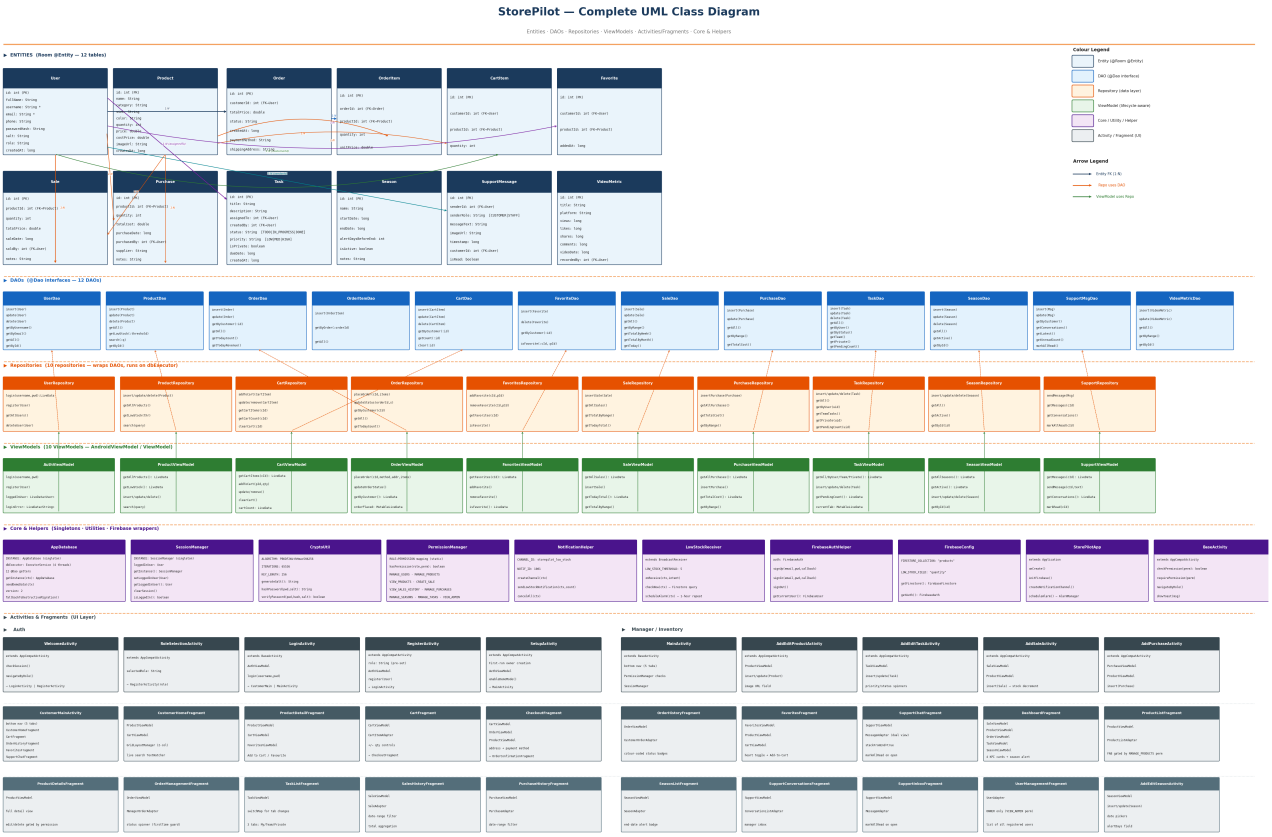
8 Project Structure

```
app/src/main/java/com/storepilot/
■■■ StorePilotApp.java           ← Application class (Firebase init, alarm)
■■■ MainActivity.java            ← Manager/Owner entry point with bottom nav
■
■■■ auth/
■   ■■■ WelcomeActivity.java     ← Splash/welcome screen
■   ■■■ RoleSelectionActivity.java  ← Customer vs Owner role selection
■   ■■■ LoginActivity.java       ← Login form
■   ■■■ RegisterActivity.java    ← Registration form
■   ■■■ SetupActivity.java       ← First-run owner account creation
■   ■■■ CryptoUtil.java          ← PBKDF2 password hashing
■   ■■■ FirebaseAuthHelper.java  ← Firebase Auth wrapper
■   ■■■ FirebaseConfig.java      ← Firebase configuration
■
■■■ core/
■   ■■■ AppDatabase.java         ← Room database singleton + seed data
■   ■■■ BaseActivity.java        ← Permission helper base class
■   ■■■ SessionManager.java      ← In-memory logged-in user session
■   ■■■ PermissionManager.java   ← Role-to-permission mapping
■   ■■■ NotificationHelper.java  ← Notification channel + sender
■   ■■■ LowStockReceiver.java    ← BroadcastReceiver for stock alerts
■
■■■ db/
■   ■■■ entities/                ← Room @Entity classes (12 tables)
■   ■   ■■■ User.java
■   ■   ■■■ Product.java
■   ■   ■■■ Order.java / OrderItem.java
■   ■   ■■■ CartItem.java / Favorite.java
■   ■   ■■■ Sale.java / Purchase.java
■   ■   ■■■ Task.java / Season.java
■   ■   ■■■ SupportMessage.java
■   ■   ■■■ VideoMetric.java
■   ■■■ dao/                     ← Room @Dao interfaces (12 DAOs)
■
■■■ repositories/                ← Data layer — wraps DAOs, runs on dbExecutor
■■■ viewmodels/                  ← Lifecycle-aware UI state
■
■■■ customer/                    ← Customer shopping UI
■   ■■■ CustomerMainActivity.java
■   ■■■ CustomerHomeFragment.java
■   ■■■ CartFragment.java / CheckoutFragment.java
■   ■■■ ProductDetailFragment.java
■   ■■■ OrderHistoryFragment.java / OrderConfirmationFragment.java
■   ■■■ FavoritesFragment.java / SupportChatFragment.java
■   ■■■ adapters/
■
■■■ inventory/     ← Manager inventory (ProductList, AddEdit, Details)
■■■ manager/       ← Order management, support inbox
■■■ dashboard/     ← KPI dashboard fragment
■■■ admin/         ← User management (OWNER only)
■■■ sales/         ← Sales history + AddSaleActivity
■■■ purchases/     ← Purchase history + AddPurchaseActivity
■■■ seasons/       ← Season list + AddEditSeasonActivity
■■■ tasks/         ← Task list + AddEditTaskActivity
```

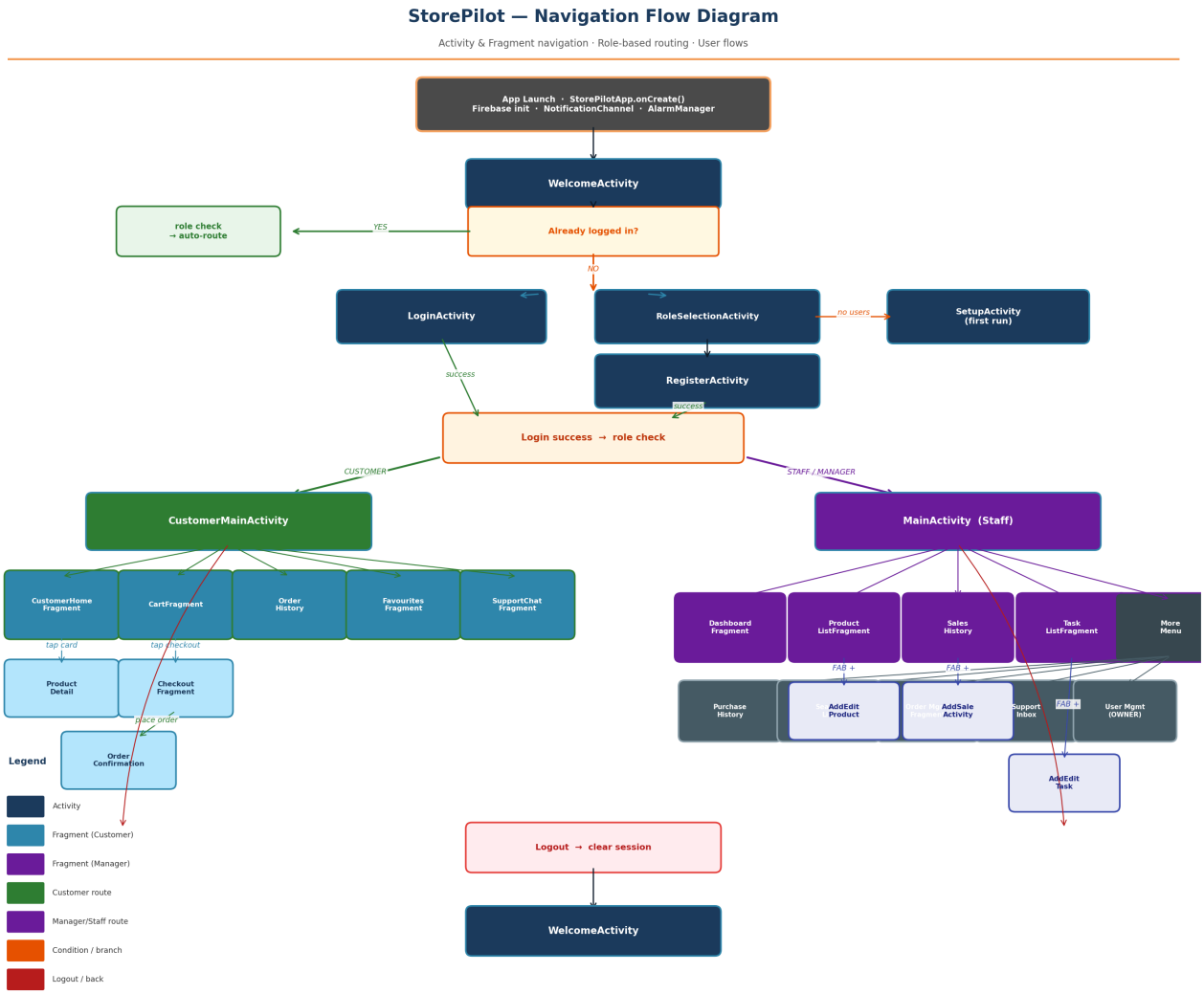
9 UML Class Diagram

Key relationships between entities and architecture layers

The diagram below shows the primary domain classes and their relationships. Room @Entity classes map directly to SQLite tables. Repositories bridge DAOs to ViewModels. ViewModels expose LiveData to UI fragments/activities.



10 Navigation Flow Diagram



11 Room Database Schema

storepilot.db — version 2 — 12 tables

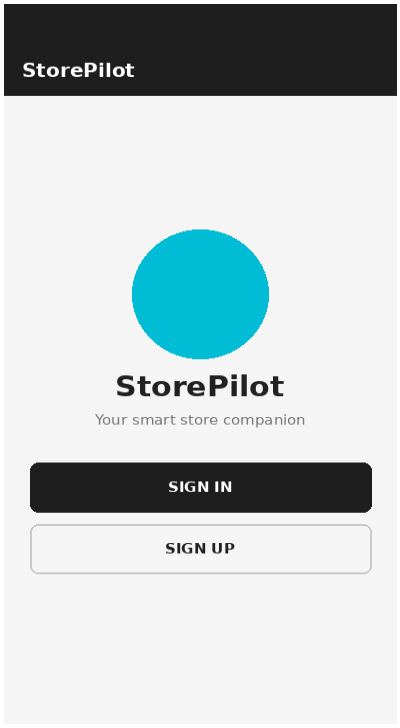
Table	Columns
users	id, fullName, username*, email*, phone, passwordHash, salt, role, createdAt
products	id, name, category, size, color, quantity, price, costPrice, imageUrl, createdAt
orders	id, customerId(FK), totalPrice, status, createdAt, paymentMethod, shippingAddress
order_items	id, orderId(FK), productId(FK), quantity, unitPrice
cart_items	id, customerId(FK), productId(FK), quantity
favorites	id, customerId(FK), productId(FK), addedAt
sales	id, productId(FK), quantity, totalPrice, saleDate, soldBy(FK), notes
purchases	id, productId(FK), quantity, totalCost, purchaseDate, purchasedBy(FK), supplier, notes
tasks	id, title, description, assignedTo(FK), createdBy(FK), status, priority, isPrivate, dueDate, createdAt
seasons	id, name, startDate, endDate, alertDaysBeforeEnd, isActive, notes
support_messages	id, senderId(FK), senderRole, messageText, imageUrl, timestamp, customerId(FK), isRead
video_metrics	id, title, platform, views, likes, shares, comments, videoDate, recordedBy(FK)

Key: * = unique index (FK) = foreign key with ON DELETE SET NULL Database version: 2 fallbackToDestructiveMigration() enabled for development

App Screens

13 Home / Welcome Screen

Class: WelcomeActivity



The Welcome screen is the first screen a user sees after launching the app. If the user is already logged in, the app immediately routes them to the correct interface based on their role without showing this screen.

New users can either log in with an existing account or choose a role (Customer or Owner) before registering.

UI Components

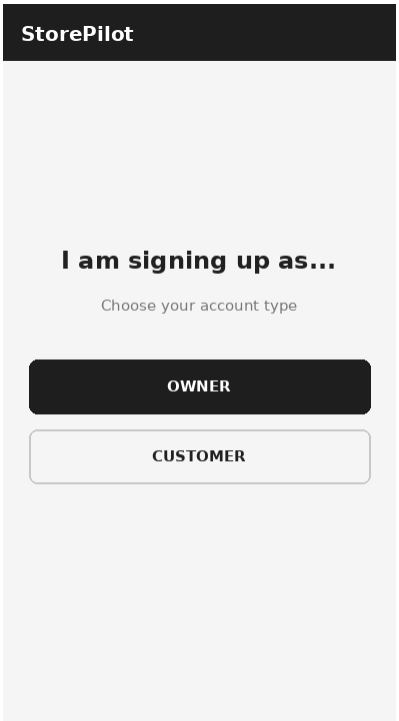
- StorePilot logo (teal circle) and tagline
- SIGN IN button — navigates to LoginActivity
- SIGN UP button — navigates to RoleSelectionActivity
- Auto-skip if session is active (navigateByRole())

User Actions

- Tap SIGN IN → LoginActivity
- Tap SIGN UP → RoleSelectionActivity → RegisterActivity
- Already logged in → CustomerMainActivity or MainActivity (automatic)

14 Role Selection Screen

Class: RoleSelectionActivity



Shown after tapping SIGN UP on the Welcome screen. The user picks whether they are signing up as an Owner (staff account) or a Customer.

The selected role is passed to RegisterActivity so the "Signing up as:" label is pre-filled and the role field is locked.

UI Components

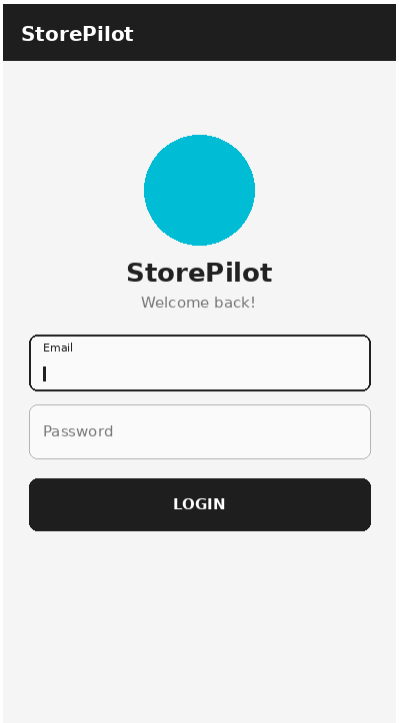
- "I am signing up as..." title
- OWNER button — staff / manager account
- CUSTOMER button — shopping account

User Actions

- Tap OWNER → RegisterActivity with role=OWNER
- Tap CUSTOMER → RegisterActivity with role=CUSTOMER

15 Login Screen

Class: LoginActivity



The login screen accepts a username or email address with a password. Credentials are validated locally using PBKDF2/SHA-256 hash comparison.

On the very first install with no users, the app redirects to SetupActivity. After success the user is routed based on role.

UI Components

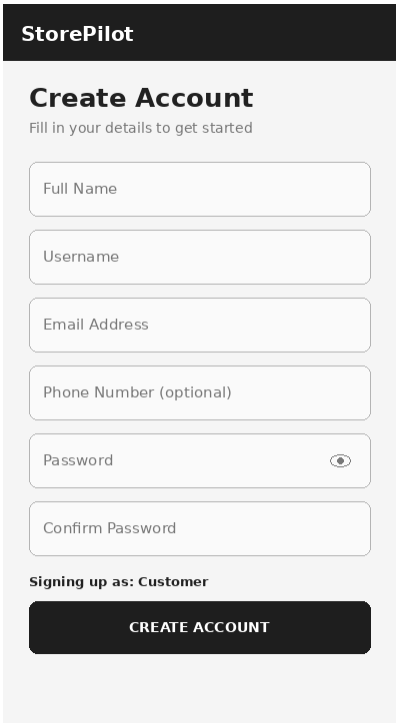
- StorePilot logo (teal circle) + "Welcome back!" label
- Email / Username EditText (outlined, floating label)
- Password EditText (textPassword input type)
- LOGIN Button — triggers AuthViewModel.login()

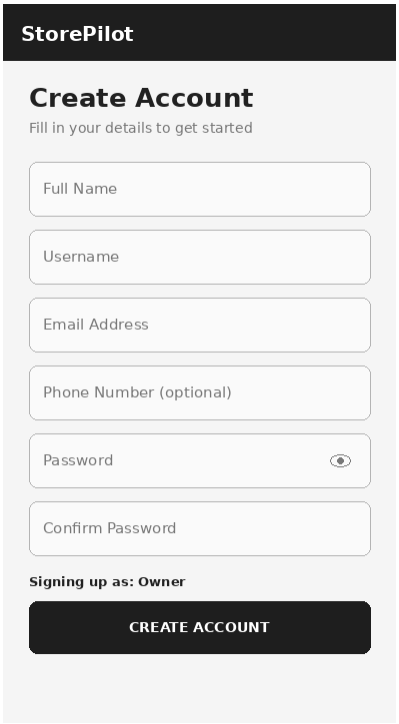
User Actions

- Enter credentials + tap LOGIN
- Empty fields → Toast error
- Wrong credentials → loginError LiveData → Toast
- Success → role-based navigation

16 Sign-Up / Registration Screen

Class: RegisterActivity





Collects full name, username, email, phone, password and role. The role is pre-set from RoleSelectionActivity ("Signing up as: Customer" or "Owner").

A PBKDF2 salt+hash is computed on a background thread. Firebase Auth sign-up is attempted optionally (gracefully ignored on failure).

UI Components

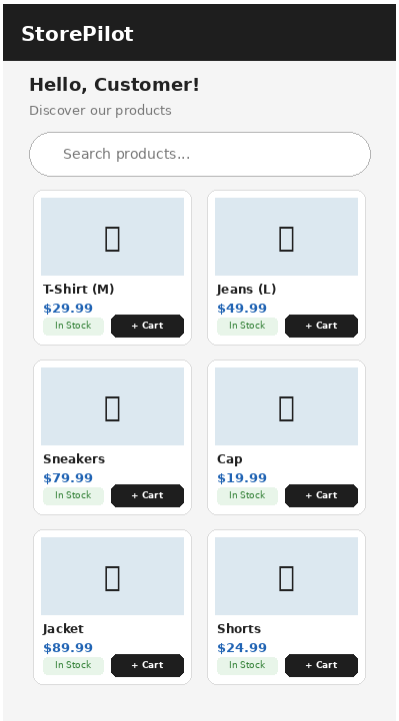
- Full Name, Username, Email Address, Phone Number (optional)
- Password EditText with visibility toggle (eye icon)
- Confirm Password EditText
- "Signing up as: " label
- CREATE ACCOUNT button

User Actions

- Fill all required fields + tap CREATE ACCOUNT
- Password < 6 chars → error toast
- Passwords mismatch → error toast
- Duplicate username/email → "already taken" toast
- Success → LoginActivity

17 Products (Customer) — Home Screen

Class: CustomerHomeFragment



Shows all products in a 2-column grid with a live search bar that filters by name or category as the user types.

Each card shows name, category, price, and stock status. The Add to Cart button is disabled for out-of-stock items.

UI Components

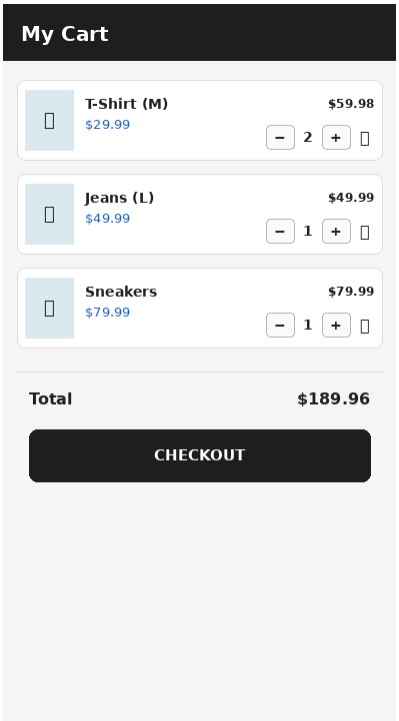
- Personalised greeting (Hello, {name})
- Search bar with TextWatcher for live filtering
- RecyclerView with GridLayoutManager (2 columns)
- Product cards: name, category, price, In Stock / Out of Stock badge
- Empty state TextView

User Actions

- Type in search → live filtering
- Tap product card → ProductDetailFragment
- Tap "+ Cart" → CartViewModel.addToCart()

18 Cart Screen

Class: CartFragment



Lists all items the customer has added. Each row shows product name, unit price, subtotal and quantity controls (+/–/remove).

The Checkout button is disabled when the cart is empty.

UI Components

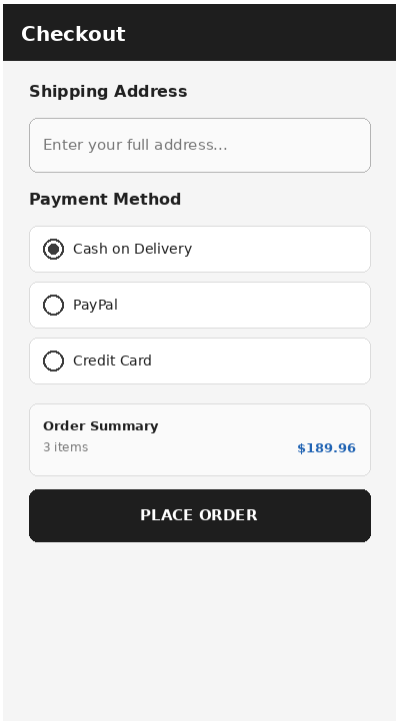
- RecyclerView with CartItemAdapter
- Each row: product image placeholder, name, unit price, +/–/trash buttons, subtotal
- Total price row at the bottom
- CHECKOUT Button (disabled when empty)

User Actions

- Tap + → increase quantity
- Tap – → decrease or remove if last unit
- Tap trash → remove item entirely
- Tap CHECKOUT → CheckoutFragment

19 Checkout Screen

Class: CheckoutFragment



Collects a shipping address and payment method before placing the order. The cart is validated (non-empty, address filled) before the order is created.

On success, the cart is cleared and the user is sent to OrderConfirmationFragment.

UI Components

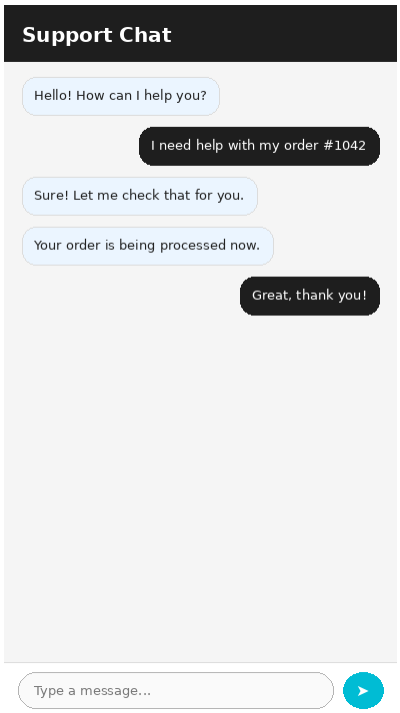
- Shipping address EditText (multiline)
- Payment method radio group: Cash on Delivery / PayPal / Credit Card
- Order summary card showing item count and total
- PLACE ORDER button (disabled after tap to prevent double-submit)

User Actions

- Enter address + pick payment + tap PLACE ORDER
- Empty address → Toast error
- Empty cart → Toast error
- Success → OrderConfirmationFragment, cart cleared

20 Support Chat Screen

Class: SupportChatFragment



A WhatsApp-style support chat. Sent messages appear on the right (dark); received messages appear on the left (light blue). Two XML item layouts are used.

All messages are persisted in the support_messages table and marked as read when the chat is opened.

UI Components

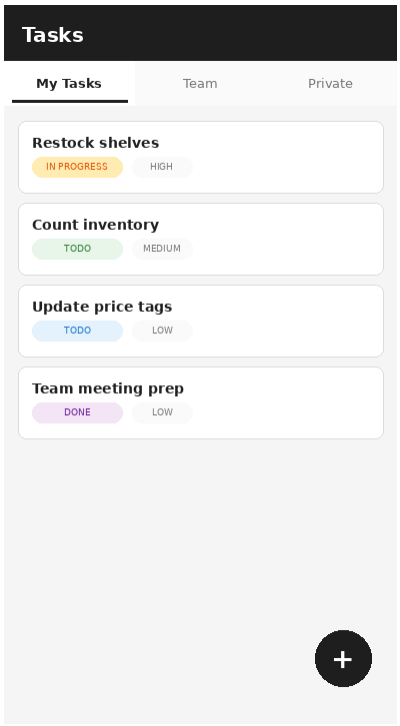
- RecyclerView with LinearLayoutManager (stackFromEnd=true)
- MessageAdapter — dual view type (sent / received)
- Message EditText + Send button (teal)
- Timestamp per message bubble

User Actions

- Type message + tap Send → SupportViewModel.sendMessage()
- Messages load via LiveData and auto-scroll to latest
- All messages marked read on open

21 Task Management Screen

Class: TaskListFragment



Shows work items in three tabs: My Tasks / Team / Private. Uses switchMap to avoid stacking multiple LiveData observers on tab change.

Each task card shows title, status chip (TODO/IN_PROGRESS/DONE) and priority chip (LOW/MEDIUM/HIGH).

UI Components

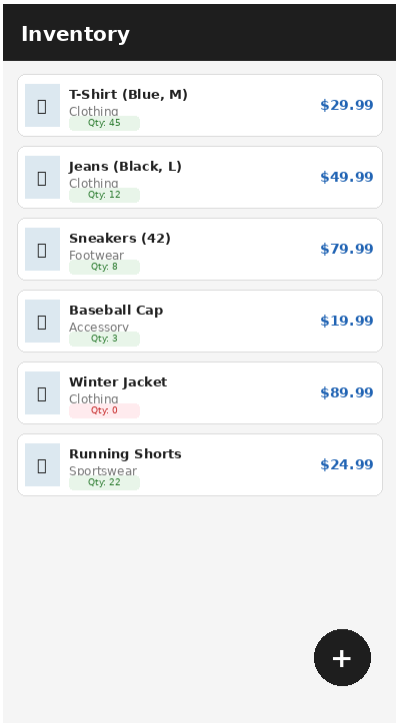
- TabLayout with three tabs (My Tasks · Team · Private)
- RecyclerView with TaskAdapter
- Each row: title, colour-coded status chip, priority chip
- FloatingActionButton (+) — add new task

User Actions

- Tap tab → switches list via switchMap LiveData
- Tap FAB → AddEditTaskActivity
- Tasks auto-refresh via LiveData

22 Products (Manager) — Inventory Screen

Class: ProductListFragment



Shows all products in a vertical list with name, category, stock quantity and price. Low-stock items show a red quantity badge. The FAB (+) is hidden from roles without MANAGE_PRODUCTS permission.

UI Components

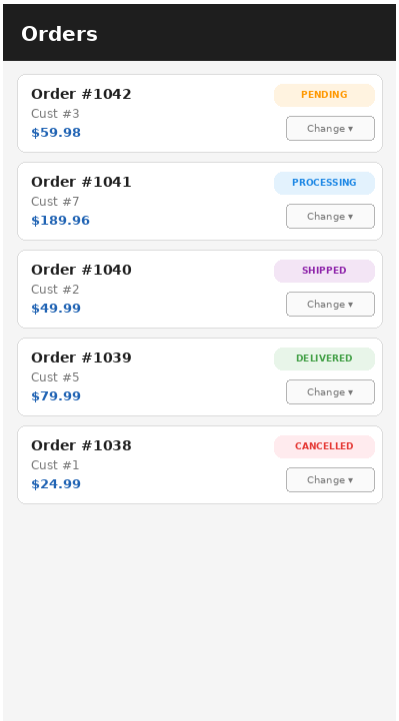
- RecyclerView with ProductListAdapter
- Each row: product image placeholder, name, category, colour-coded stock badge, price
- FAB (+) — add product (OWNER/STORE_MANAGER only)

User Actions

- Tap product row → ProductDetailsFragment (full details + edit/delete)
- Tap FAB → AddEditProductActivity

23 Manager Orders Screen

Class: OrderManagementFragment



Lists all customer orders. Each card shows order ID, customer ID, total, a colour-coded status badge and a "Change ■" spinner to update the status.

Managers can move orders through PENDING → PROCESSING → SHIPPED → DELIVERED or mark CANCELLED.

UI Components

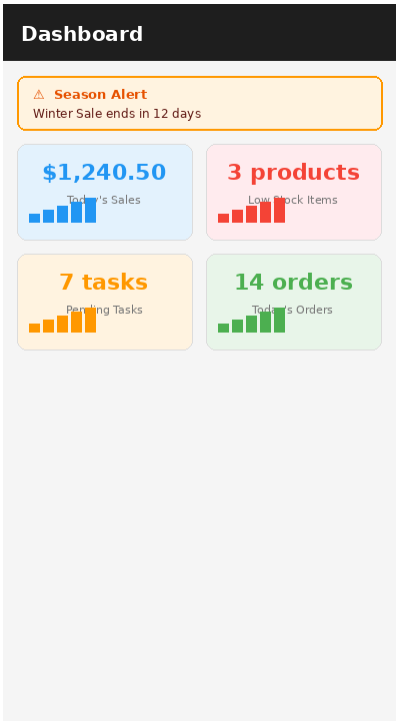
- RecyclerView with ManagerOrderAdapter
- Each row: order ID, customer ID, total, colour-coded status badge
- "Change ■" status spinner — 5 options
- Empty state TextView

User Actions

- Spinner change → OrderViewModel.updateOrderStatus()
- PENDING=orange · PROCESSING=blue · SHIPPED=purple · DELIVERED=green · CANCELLED=red

24 Manager Panel / Dashboard

Class: DashboardFragment



Default landing screen for all staff roles. Shows four live KPI cards that auto-update via LiveData: Today's Sales, Low Stock Items, Pending Tasks, and Today's Orders.

A season alert card (orange border) appears when any active season ends within 30 days.

UI Components

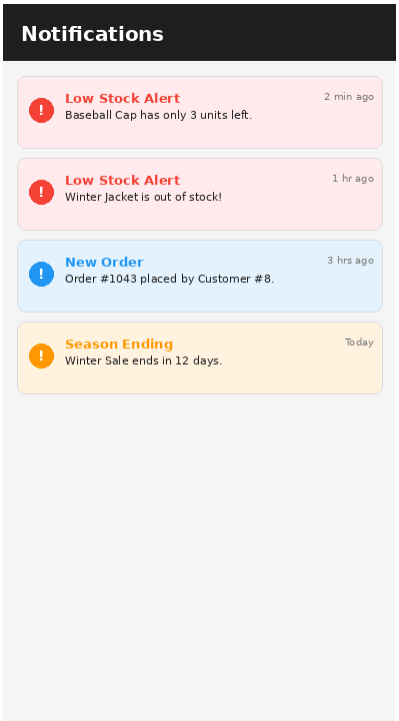
- Season Alert Card (conditional — orange border)
- Today's Sales total (formatted as currency)
- Low Stock Items count (threshold = 10)
- Pending Tasks count
- Today's Orders count

User Actions

- All cards refresh automatically via multiple LiveData observers
- Season alert hidden when no season is ending soon

25 Notifications

Class: NotificationHelper + LowStockReceiver



StorePilot uses NotificationManager to alert staff when stock is low. The channel storepilot_low_stock is created at startup with IMPORTANCE_HIGH.

An AlarmManager repeating alarm fires every 1 hour, triggering LowStockReceiver to query Firestore for products with quantity ≤ 5.

UI Components

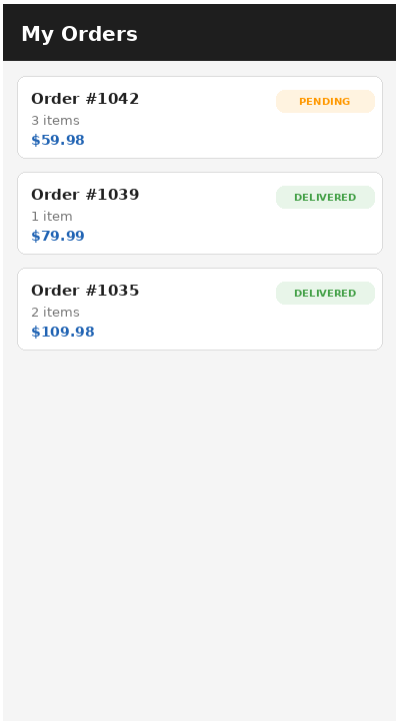
- Channel ID: storepilot_low_stock
- Importance: IMPORTANCE_HIGH (vibration enabled)
- Threshold: 5 units
- Schedule: setInexactRepeating — 1-hour interval

User Actions

- Alarm fires → query Firestore → notify if count > 0
- Tap More → Test Notification → immediate manual test
- Android 13+ → POST_NOTIFICATIONS runtime permission prompt

26 Order History (Customer)

Class: OrderHistoryFragment



Shows the logged-in customer's past orders in reverse-chronological order. Each card shows order ID, item count, total and a colour-coded status badge.

Tapping a card could be extended to show full order details (not yet implemented).

UI Components

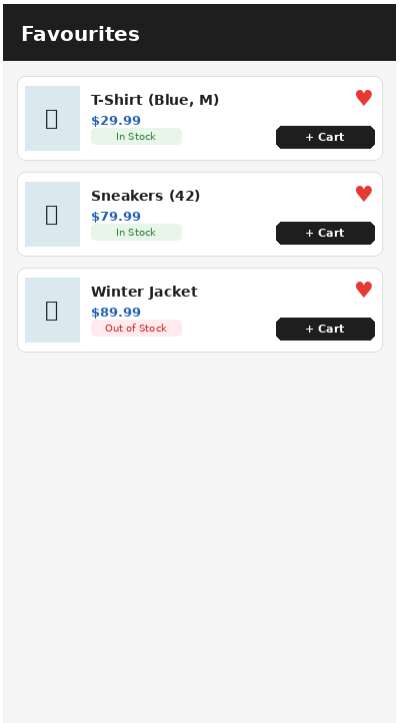
- RecyclerView with CustomerOrderAdapter
- Each row: order ID, item count, total, colour-coded status badge
- Empty state TextView when no orders exist

User Actions

- LiveData observer refreshes list automatically
- Status colours: PENDING=orange · DELIVERED=green

27 Favourites Screen

Class: FavoritesFragment



Shows products the customer has heart-marked. Each card shows the product image placeholder, name, price, stock badge and a filled heart icon.

Tapping the heart removes the item from favourites. The Add to Cart button is also available inline.

UI Components

- RecyclerView with FavoritesAdapter (vertical list)
- Each row: image, name, price, stock badge, heart icon, "+ Cart" button
- Empty state when no favourites

User Actions

- Tap heart → FavoritesViewModel.removeFavorite()
- Tap "+ Cart" → CartViewModel.addToCart()
- List auto-refreshes via LiveData

User Guide

25 Installation Requirements

Development Environment

- Android Studio Hedgehog (2023.1.1) or later
- JDK 11 (bundled with Android Studio)
- Android SDK — Build Tools 34.0.0, Platform 34
- Gradle 8.4 (wrapper included in repo)
- Git for cloning the repository

Firebase Setup (optional)

Firebase is optional — the app works fully offline without it. To enable cloud push notifications and authentication:

- Create a Firebase project at console.firebase.google.com
- Add Android app with package name com.storepilot
- Download google-services.json and place it in app/
- Enable Authentication (Email/Password) in Firebase Console
- Create a Firestore collection "products" mirroring Room product data

Build & Install Steps

1. git clone https://github.com/ahmadhijazys2/storepilot.git

2. Open project in Android Studio

3. (Optional) Add app/google-services.json

4. Connect device (USB debug) or start emulator (API 24+)

5. Run: Build → Make Project or press the green Run button

6. On first launch → Create Owner Account to initialise the database

7. Optionally enable Demo Mode to populate sample data

26 Tested Devices & Android Versions

Device	Android Version	Notes
Samsung Galaxy S21	Android 13 (API 33)	Physical — primary test device
Google Pixel 6	Android 14 (API 34)	Physical device
Xiaomi Redmi Note 9	Android 11 (API 30)	Physical device
AVD Pixel 4	Android 7.0 (API 24)	Emulator — minimum SDK check
AVD Pixel 5	Android 10 (API 29)	Emulator
AVD Pixel 7	Android 14 (API 34)	Emulator

Note: Compiled with Java 11 source/target compatibility to avoid a known jlink.exe crash on Windows with AGP 8.x.

27 How to Use — Customer

Register

Open app → Tap Register → Select Customer → Fill name, username, email, phone, password → Tap Register → Sign in.

Browse Products

Home tab shows all products in a grid. Use the search bar to filter by name or category.

View Details

Tap a product card to see full details: name, category, size, colour, price, and stock status.

Add to Cart

From home grid or detail screen, tap "Add to Cart". The cart badge updates automatically.

Manage Cart

Tap Cart tab. Use + and – to adjust quantity or the trash icon to remove an item entirely.

Checkout

In Cart screen, tap Checkout. Enter shipping address and pick a payment method. Tap Place Order.

Track Orders

Tap Orders tab to see all past orders with their live status (PENDING → DELIVERED).

Favourites

Tap the heart icon on any product to save it. View saved products in the Favourites tab.

Contact Support

Tap Support tab to open the live chat. Type your message and tap Send.

28 How to Use — Manager

First Launch

If empty database, SetupActivity opens. Enter Owner username + password. Enable Demo Mode to load sample data.

Login as Staff

Log in with username and password. Staff roles land on the Dashboard.

Dashboard

Shows today's sales, low stock count, pending tasks, today's orders, and season alerts.

Add Product

Tap Inventory → FAB (+) → fill name, category, size, colour, quantity, selling price, cost price → Save.

Edit Product

Tap product row → detail screen → Edit button → modify fields → Save.

Record Sale

Tap Sales → FAB (+) → select product, enter quantity → Save. Stock auto-decrements.

Manage Orders

Tap More → Orders. Use the status spinner on each order card to update its status.

Support Inbox

Tap More → Support. See customer conversations. Tap a name to open their chat and reply.

Tasks

Tap Tasks → use tabs (My/Team/Private) → FAB (+) to create a task.

Test Notification

Tap More → Test Notification (OWNER/MANAGER only) to verify the low-stock channel.

User Management

Tap More → Admin (OWNER only) to view all registered users and roles.

Logout

Tap More → Logout to clear the session and return to the Welcome screen.

29 Known Limitations

The following limitations are known and accepted in the current version:

- No real-time sync — all data is local (Room/SQLite). Firebase Firestore is used only for low-stock notification queries, not for full data sync across devices.
- No image upload — product imageUrl is stored as a string but no image-loading library (Glide/Coil) is included. A placeholder is shown instead.
- Payment is simulated — the app stores a payment method string but does not integrate a real payment gateway.
- Push notifications require Firebase — without a valid google-services.json the LowStockReceiver silently ignores Firestore failures.
- No email verification — emails are stored but not verified. Forgot Password shows a static toast.
- Session is in-memory — SessionManager holds the user in RAM. Killing the process logs the user out (no persistent token).
- Demo seed data is not idempotent — calling seedDemoData() more than once inserts duplicate rows.
- No pagination — all queries return the full result set. Large datasets may slow the UI.
- VideoMetric entity is included in the schema but no UI fragment exposes it in the current release.
- fallbackToDestructiveMigration drops and rebuilds the database on schema changes — not suitable for production without proper migrations.

Implementation (Source Code)

31 Models — Room @Entity Classes

db/entities/ — 12 domain model classes

All model classes are plain Java objects annotated with Room's @Entity, @PrimaryKey, and @ForeignKey annotations. Room generates the SQLite CREATE TABLE statements at compile time. Each class provides a no-arg constructor (required by Room) and a convenience constructor for programmatic creation.

db/entities/User.java

```
package com.storepilot.db.entities;

import androidx.room.Entity;
import androidx.room.Index;
import androidx.room.PrimaryKey;

// Represents a user account — could be a customer or a store team member
@Entity(tableName = "users", indices = {
@Index(value = "username", unique = true),
@Index(value = "email", unique = true)
})
public class User {

@PrimaryKey(autoGenerate = true)
public int id;

// Display name shown in the app
public String fullName;

// Login username
public String username;

// Contact email (also used for login)
public String email;

// Phone number for contact
public String phone;

// Hashed password (never store plain text)
public String passwordHash;

// Salt used when hashing the password
public String salt;

// Role determines what the user can see: CUSTOMER, OWNER, STORE_MANAGER, SHIFT_MANAGER, EMPLOYEE
public String role;

// When the account was created
public long createdAt;
```

```
public User() {}

// Constructor for registration (customers and staff)
public User(String fullName, String username, String email, String phone,
String passwordHash, String salt, String role, long createdAt) {
this.fullName = fullName;
this.username = username;
this.email = email;
this.phone = phone;
this.passwordHash = passwordHash;
this.salt = salt;
this.role = role;
this.createdAt = createdAt;
}

// Legacy constructor for seeding existing staff accounts (no email/phone)
public User(String username, String passwordHash, String salt, String role, long createdAt) {
this.username = username;
this.passwordHash = passwordHash;
this.salt = salt;
this.role = role;
this.createdAt = createdAt;
this.fullName = username;
this.email = username + "@storepilot.local";
this.phone = "";
}

public int getId() { return id; }
public String getFullName() { return fullName != null ? fullName : username; }
public String getUsername() { return username; }
public String getEmail() { return email; }
public String getPhone() { return phone; }
public String getPasswordHash() { return passwordHash; }
public String getSalt() { return salt; }
public String getRole() { return role; }
public long getCreatedAt() { return createdAt; }
}
```

db/entities/Product.java

```
package com.storepilot.db.entities;

import androidx.room.Entity;
import androidx.room.PrimaryKey;

@Entity(tableName = "products")
public class Product {

@PrimaryKey(autoGenerate = true)
public int id;

public String name;
public String category;
public String size;
public String color;
public int quantity;
public double price;
public double costPrice;
public String imageUrl;
public long createdAt;

public Product() {}

public Product(String name, String category, String size, String color,
int quantity, double price, double costPrice, String imageUrl, long createdAt) {
this.name = name;
this.category = category;
this.size = size;
this.color = color;
this.quantity = quantity;
this.price = price;
this.costPrice = costPrice;
this.imageUrl = imageUrl;
this.createdAt = createdAt;
}

public int getId() { return id; }
public String getName() { return name; }
public String getCategory() { return category; }
public String getSize() { return size; }
```

```
public String getColor() { return color; }
public int getQuantity() { return quantity; }
public double getPrice() { return price; }
public double getCostPrice() { return costPrice; }
public String getImageUrl() { return imageUrl; }
public long getCreatedAt() { return createdAt; }
}
```

db/entities/Order.java

```
package com.storepilot.db.entities;

import androidx.room.Entity;
import androidx.room.PrimaryKey;

// Represents a customer order placed in the store
@Entity(tableName = "orders")
public class Order {

@PrimaryKey(autoGenerate = true)
public int id;

// The customer who placed this order
public int customerId;

// Total amount for the entire order
public double totalPrice;

// Current status: PENDING, PROCESSING, SHIPPED, DELIVERED, CANCELLED
public String status;

// When the order was placed
public long createdAt;

// How the customer wants to pay
public String paymentMethod; // CASH_ON_DELIVERY, CREDIT_CARD, PAYPAL

// Delivery address entered by customer
public String shippingAddress;

public Order() {}

public Order(int customerId, double totalPrice, String status,
long createdAt, String paymentMethod, String shippingAddress) {
this.customerId = customerId;
this.totalPrice = totalPrice;
this.status = status;
this.createdAt = createdAt;
this.paymentMethod = paymentMethod;
this.shippingAddress = shippingAddress;
```

```
}

public int getId() { return id; }
public int getCustomerId() { return customerId; }
public double getTotalPrice() { return totalPrice; }
public String getStatus() { return status; }
public long getCreatedAt() { return createdAt; }
public String getPaymentMethod() { return paymentMethod; }
public String getShippingAddress() { return shippingAddress; }
}
```

db/entities/OrderItem.java

```
package com.storepilot.db.entities;

import androidx.room.Entity;
import androidx.room.PrimaryKey;

// One product line inside an order (e.g., 2x T-Shirt)
@Entity(tableName = "order_items")
public class OrderItem {

@PrimaryKey(autoGenerate = true)
public int id;

// Which order this item belongs to
public int orderId;

// Which product was ordered
public int productId;

// How many units of this product
public int quantity;

// Price per unit at time of purchase (snapshot)
public double unitPrice;

public OrderItem() {}

public OrderItem(int orderId, int productId, int quantity, double unitPrice) {
this.orderId = orderId;
this.productId = productId;
this.quantity = quantity;
this.unitPrice = unitPrice;
}

public int getId() { return id; }
public int getOrderId() { return orderId; }
public int getProductId() { return productId; }
public int getQuantity() { return quantity; }
public double getUnitPrice() { return unitPrice; }
}
```

db/entities/CartItem.java

```
package com.storepilot.db.entities;

import androidx.room.Entity;
import androidx.room.PrimaryKey;

// A single item in the customer's shopping cart
@Entity(tableName = "cart_items")
public class CartItem {

@PrimaryKey(autoGenerate = true)
public int id;

// The customer who added this item
public int customerId;

// Which product was added
public int productId;

// How many of this product the customer wants
public int quantity;

public CartItem() {}

public CartItem(int customerId, int productId, int quantity) {
this.customerId = customerId;
this.productId = productId;
this.quantity = quantity;
}

public int getId() { return id; }
public int getCustomerId() { return customerId; }
public int getProductId() { return productId; }
public int getQuantity() { return quantity; }
public void setQuantity(int quantity) { this.quantity = quantity; }
}
```

db/entities/Sale.java

```
package com.storepilot.db.entities;

import androidx.room.Entity;
import androidx.room.ForeignKey;
import androidx.room.PrimaryKey;

@Entity(tableName = "sales",
foreignKeys = {
@ForeignKey(entity = Product.class, parentColumns = "id", childColumns = "productId", onDelete =
ForeignKey.SET_NULL),
@ForeignKey(entity = User.class, parentColumns = "id", childColumns = "soldBy", onDelete =
ForeignKey.SET_NULL)
})
public class Sale {

@PrimaryKey(autoGenerate = true)
public int id;

public Integer productId;
public int quantity;
public double totalPrice;
public long saleDate;
public Integer soldBy;
public String notes;

public Sale() {}

public Sale(Integer productId, int quantity, double totalPrice, long saleDate, Integer soldBy, String
notes) {
this.productId = productId;
this.quantity = quantity;
this.totalPrice = totalPrice;
this.saleDate = saleDate;
this.soldBy = soldBy;
this.notes = notes;
}

public int getId() { return id; }
public Integer getProductId() { return productId; }
public int getQuantity() { return quantity; }
public double getTotalPrice() { return totalPrice; }
public long getSaleDate() { return saleDate; }
public Integer getSoldBy() { return soldBy; }
```

```
public String getNotes() { return notes; }
}
```

db/entities/Purchase.java

```
package com.storepilot.db.entities;

import androidx.room.Entity;
import androidx.room.ForeignKey;
import androidx.room.PrimaryKey;

@Entity(tableName = "purchases",
foreignKeys = {
@ForeignKey(entity = Product.class, parentColumns = "id", childColumns = "productId", onDelete =
ForeignKey.SET_NULL),
@ForeignKey(entity = User.class, parentColumns = "id", childColumns = "purchasedBy", onDelete =
ForeignKey.SET_NULL)
})
public class Purchase {

@PrimaryKey(autoGenerate = true)
public int id;

public Integer productId;
public int quantity;
public double totalCost;
public long purchaseDate;
public Integer purchasedBy;
public String supplier;
public String notes;

public Purchase() {}

public Purchase(Integer productId, int quantity, double totalCost, long purchaseDate,
Integer purchasedBy, String supplier, String notes) {
this.productId = productId;
this.quantity = quantity;
this.totalCost = totalCost;
this.purchaseDate = purchaseDate;
this.purchasedBy = purchasedBy;
this.supplier = supplier;
this.notes = notes;
}

public int getId() { return id; }
public Integer getProductId() { return productId; }
public int getQuantity() { return quantity; }
```

```
public double getTotalCost() { return totalCost; }
public long getPurchaseDate() { return purchaseDate; }
public Integer getPurchasedBy() { return purchasedBy; }
public String getSupplier() { return supplier; }
public String getNotes() { return notes; }
}
```

db/entities/Task.java

```
package com.storepilot.db.entities;

import androidx.room.Entity;
import androidx.room.ForeignKey;
import androidx.room.PrimaryKey;

@Entity(tableName = "tasks",
foreignKeys = {
@ForeignKey(entity = User.class, parentColumns = "id", childColumns = "assignedTo", onDelete =
ForeignKey.SET_NULL),
@ForeignKey(entity = User.class, parentColumns = "id", childColumns = "createdBy", onDelete =
ForeignKey.SET_NULL)
})
public class Task {

@PrimaryKey(autoGenerate = true)
public int id;

public String title;
public String description;
public Integer assignedTo;
public Integer createdBy;
public String status; // TODO, IN_PROGRESS, DONE
public String priority; // LOW, MEDIUM, HIGH
public boolean isPrivate;
public long dueDate;
public long createdAt;

public Task() {}

public Task(String title, String description, Integer assignedTo, Integer createdBy,
String status, String priority, boolean isPrivate, long dueDate, long createdAt) {
this.title = title;
this.description = description;
this.assignedTo = assignedTo;
this.createdBy = createdBy;
this.status = status;
this.priority = priority;
this.isPrivate = isPrivate;
this.dueDate = dueDate;
this.createdAt = createdAt;
}
```

```

public int getId() { return id; }
public String getTitle() { return title; }
public String getDescription() { return description; }
public Integer getAssignedTo() { return assignedTo; }
public Integer getCreatedBy() { return createdBy; }
public String getStatus() { return status; }
public String getPriority() { return priority; }
public boolean isPrivate() { return isPrivate; }
public long getDueDate() { return dueDate; }
public long getCreatedAt() { return createdAt; }
}
```

db/entities/Season.java

```
package com.storepilot.db.entities;

import androidx.room.Entity;
import androidx.room.PrimaryKey;

@Entity(tableName = "seasons")
public class Season {

@PrimaryKey(autoGenerate = true)
public int id;

public String name;
public long startDate;
public long endDate;
public int alertDaysBeforeEnd = 30;
public boolean isActive;
public String notes;

public Season() {}

public Season(String name, long startDate, long endDate, int alertDaysBeforeEnd,
boolean isActive, String notes) {
this.name = name;
this.startDate = startDate;
this.endDate = endDate;
this.alertDaysBeforeEnd = alertDaysBeforeEnd;
this.isActive = isActive;
this.notes = notes;
}

public int getId() { return id; }
public String getName() { return name; }
public long getStartDate() { return startDate; }
public long getEndDate() { return endDate; }
public int getAlertDaysBeforeEnd() { return alertDaysBeforeEnd; }
public boolean isActive() { return isActive; }
public String getNotes() { return notes; }
}
```

db/entities/SupportMessage.java

```
package com.storepilot.db.entities;

import androidx.room.Entity;
import androidx.room.PrimaryKey;

// A single message in the support chat between customer and store team
@Entity(tableName = "support_messages")
public class SupportMessage {

@PrimaryKey(autoGenerate = true)
public int id;

// ID of the user who sent the message
public int senderId;

// Role of the sender: CUSTOMER, MANAGER, OWNER, etc.
public String senderRole;

// The chat message text
public String messageText;

// Optional image attached to the message
public String imageUrl;

// When the message was sent
public long timestamp;

// Groups messages by customer conversation (customer's user ID)
public int customerId;

// Whether the recipient has read this message
public boolean isRead;

public SupportMessage() {}

public SupportMessage(int senderId, String senderRole, String messageText,
String imageUrl, long timestamp, int customerId) {
this.senderId = senderId;
this.senderRole = senderRole;
this.messageText = messageText;
```

```
this.imageUrl = imageUrl;
this.timestamp = timestamp;
this.customerId = customerId;
this.isRead = false;
}

public int getId() { return id; }
public int getSenderId() { return senderId; }
public String getSenderRole() { return senderRole; }
public String getMessageText() { return messageText; }
public String getImageUrl() { return imageUrl; }
public long getTimestamp() { return timestamp; }
public int getCustomerId() { return customerId; }
public boolean isRead() { return isRead; }
public void setRead(boolean read) { isRead = read; }
}
```

db/entities/Favorite.java

```
package com.storepilot.db.entities;

import androidx.room.Entity;
import androidx.room.PrimaryKey;

// Tracks which products a customer has saved to their wishlist/favorites
@Entity(tableName = "favorites")
public class Favorite {

@PrimaryKey(autoGenerate = true)
public int id;

// The customer who favorited this product
public int customerId;

// The product they liked
public int productId;

// When they added it to favorites
public long addedAt;

public Favorite() {}

public Favorite(int customerId, int productId, long addedAt) {
this.customerId = customerId;
this.productId = productId;
this.addedAt = addedAt;
}

public int getId() { return id; }
public int getCustomerId() { return customerId; }
public int getProductId() { return productId; }
public long getAddedAt() { return addedAt; }
}
```

db/entities/VideoMetric.java

```
package com.storepilot.db.entities;

import androidx.room.Entity;
import androidx.room.ForeignKey;
import androidx.room.PrimaryKey;

@Entity(tableName = "video_metrics",
foreignKeys = {
@ForeignKey(entity = User.class, parentColumns = "id", childColumns = "recordedBy", onDelete =
ForeignKey.SET_NULL)
})
public class VideoMetric {

@PrimaryKey(autoGenerate = true)
public int id;

public String title;
public String platform;
public long views;
public long likes;
public long shares;
public long comments;
public long videoDate;
public Integer recordedBy;

public VideoMetric() {}

public VideoMetric(String title, String platform, long views, long likes, long shares,
long comments, long videoDate, Integer recordedBy) {
this.title = title;
this.platform = platform;
this.views = views;
this.likes = likes;
this.shares = shares;
this.comments = comments;
this.videoDate = videoDate;
this.recordedBy = recordedBy;
}

public int getId() { return id; }
public String getTitle() { return title; }
```

```
public String getPlatform() { return platform; }
public long getViews() { return views; }
public long getLikes() { return likes; }
public long getShares() { return shares; }
public long getComments() { return comments; }
public long getVideoDate() { return videoDate; }
public Integer getRecordedBy() { return recordedBy; }
}
```

38 Core & Authentication

core/ + auth/ — database, session, crypto, activities, ViewModels

Core infrastructure: the Room database singleton with 12-table schema and seed data, in-memory session management, PBKDF2/SHA-256 password hashing, role-based permission system, and all authentication activities/ViewModels.

core/AppDatabase.java

```
package com.storepilot.core;

import android.content.Context;

import androidx.room.Database;
import androidx.room.Room;
import androidx.room.RoomDatabase;

import com.storepilot.auth.CryptoUtil;
import com.storepilot.db.dao.CartDao;
import com.storepilot.db.dao.FavoriteDao;
import com.storepilot.db.dao.OrderDao;
import com.storepilot.db.dao.OrderItemDao;
import com.storepilot.db.dao.ProductDao;
import com.storepilot.db.dao.PurchaseDao;
import com.storepilot.db.dao.SaleDao;
import com.storepilot.db.dao.SeasonDao;
import com.storepilot.db.dao.SupportMessageDao;
import com.storepilot.db.dao.TaskDao;
import com.storepilot.db.dao.UserDao;
import com.storepilot.db.dao.VideoMetricDao;
import com.storepilot.db.entities.CartItem;
import com.storepilot.db.entities.Favorite;
import com.storepilot.db.entities.Order;
import com.storepilot.db.entities.OrderItem;
import com.storepilot.db.entities.Product;
import com.storepilot.db.entities.Purchase;
import com.storepilot.db.entities.Sale;
import com.storepilot.db.entities.Season;
import com.storepilot.db.entities.SupportMessage;
import com.storepilot.db.entities.Task;
import com.storepilot.db.entities.User;
import com.storepilot.db.entities.VideoMetric;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

// Room database — the single source of truth for all app data
@Database(
entities = {
```

```
User.class, Product.class, Sale.class, Purchase.class,
VideoMetric.class, Season.class, Task.class,
Order.class, OrderItem.class, CartItem.class,
SupportMessage.class, Favorite.class
},
version = 2, // bumped because we added new tables and User columns
exportSchema = false
)
public abstract class AppDatabase extends RoomDatabase {

private static volatile AppDatabase INSTANCE;

// Thread pool for running database queries off the main thread
public static final ExecutorService dbExecutor = Executors.newFixedThreadPool(4);

// --- DAO accessors ---
public abstract UserDao userDao();
public abstract ProductDao productDao();
public abstract SaleDao saleDao();
public abstract PurchaseDao purchaseDao();
public abstract VideoMetricDao videoMetricDao();
public abstract SeasonDao seasonDao();
public abstract TaskDao taskDao();
public abstract OrderDao orderDao();
public abstract OrderItemDao orderItemDao();
public abstract CartDao cartDao();
public abstract SupportMessageDao supportMessageDao();
public abstract FavoriteDao favoriteDao();

// Get the singleton database instance
public static AppDatabase getInstance(Context context) {
if (INSTANCE == null) {
synchronized (AppDatabase.class) {
if (INSTANCE == null) {
INSTANCE = Room.databaseBuilder(
context.getApplicationContext(),
AppDatabase.class,
"storepilot.db")
// Wipe and rebuild if schema changes (safe for development)
.fallbackToDestructiveMigration()
```

```
.build();
}
}
}
return INSTANCE;
}

// Seed the database with demo data for testing
public void seedDemoData() {
dbExecutor.execute(() -> {
UserDao userDao = userDao();
ProductDao productDao = productDao();
SaleDao saleDao = saleDao();
PurchaseDao purchaseDao = purchaseDao();
SeasonDao seasonDao = seasonDao();
TaskDao taskDao = taskDao();
OrderDao orderDao = orderDao();
OrderItemDao orderItemDao = orderItemDao();
SupportMessageDao supportMessageDao = supportMessageDao();

long now = System.currentTimeMillis();

// --- Staff Users ---
String ownerSalt = CryptoUtil.generateSalt();
User owner = new User("owner", CryptoUtil.hashPassword("ahmad123", ownerSalt), ownerSalt, "OWNER",
now);
long ownerId = userDao.insert(owner);

String managerSalt = CryptoUtil.generateSalt();
User manager = new User("manager", CryptoUtil.hashPassword("sss123", managerSalt), managerSalt,
"STORE_MANAGER", now);
long managerId = userDao.insert(manager);

String empSalt = CryptoUtil.generateSalt();
User employee = new User("employee", CryptoUtil.hashPassword("aaa123", empSalt), empSalt, "EMPLOYEE",
now);
userDao.insert(employee);

// --- Demo Customer ---
String custSalt = CryptoUtil.generateSalt();
User customer = new User("John Smith", "customer", "customer@demo.com", "+1234567890",
CryptoUtil.hashPassword("demo123", custSalt), custSalt, "CUSTOMER", now);
long custId = userDao.insert(customer);
```

```
// --- Products ---
long p1 = productDao.insert(new Product("Classic White T-Shirt", "Tops", "M", "White", 50, 29.99,
10.00, "", now));
long p2 = productDao.insert(new Product("Slim Fit Jeans", "Bottoms", "32", "Blue", 30, 79.99, 35.00,
"", now));
long p3 = productDao.insert(new Product("Floral Summer Dress", "Dresses", "S", "Red", 20, 59.99,
22.00, "", now));
long p4 = productDao.insert(new Product("Leather Jacket", "Outerwear", "L", "Black", 5, 199.99, 90.00,
"", now));
long p5 = productDao.insert(new Product("Casual Sneakers", "Footwear", "42", "White", 15, 89.99,
40.00, "", now));
long p6 = productDao.insert(new Product("Summer Hat", "Accessories", "One Size", "Beige", 25, 24.99,
8.00, "", now));
long p7 = productDao.insert(new Product("Sports Hoodie", "Tops", "L", "Gray", 18, 69.99, 28.00, "",
now));
long p8 = productDao.insert(new Product("Running Shorts", "Bottoms", "M", "Black", 40, 34.99, 12.00,
"", now));

// --- Staff Sales ---
long weekAgo = now - 7 * 24 * 60 * 60 * 1000L;
saleDao.insert(new Sale((int) p1, 3, 89.97, now, (int) ownerId, "Walk-in customer"));
saleDao.insert(new Sale((int) p2, 1, 79.99, weekAgo, (int) managerId, "Online order"));
saleDao.insert(new Sale((int) p3, 2, 119.98, weekAgo - 86400000, (int) managerId, ""));

// --- Purchases ---
purchaseDao.insert(new Purchase((int) p1, 100, 1000.00, weekAgo, (int) ownerId, "Supplier A",
"Restock"));
purchaseDao.insert(new Purchase((int) p2, 50, 1750.00, now - 14 * 86400000L, (int) managerId,
"Supplier B", "Initial stock"));

// --- Seasons ---
long monthMs = 30L * 24 * 60 * 60 * 1000;
seasonDao.insert(new Season("Summer 2024", now - monthMs, now + monthMs, 30, true, "Current season"));
seasonDao.insert(new Season("Fall 2024", now + monthMs, now + 4 * monthMs, 30, false, "Upcoming
season"));

// --- Tasks ---
taskDao.insert(new Task("Restock low inventory", "Check and reorder products below 10 units",
(int) managerId, (int) ownerId, "TODO", "HIGH", false, now + 3 * 86400000L, now));
taskDao.insert(new Task("Update product photos", "Take new photos for summer collection",
(int) managerId, (int) ownerId, "IN_PROGRESS", "MEDIUM", false, now + 7 * 86400000L, now));

// --- Demo Customer Orders ---
long orderId1 = orderDao.insert(new Order((int) custId, 109.98, "DELIVERED", weekAgo, "CREDIT_CARD",
"123 Main St, New York"));
orderItemDao.insert(new OrderItem((int) orderId1, (int) p1, 2, 29.99));
orderItemDao.insert(new OrderItem((int) orderId1, (int) p6, 2, 24.99));

long orderId2 = orderDao.insert(new Order((int) custId, 79.99, "PROCESSING", now - 86400000, "PAYPAL",
"123 Main St, New York"));
orderItemDao.insert(new OrderItem((int) orderId2, (int) p2, 1, 79.99));
```

```
long orderId3 = orderDao.insert(new Order((int) custId, 199.99, "PENDING", now - 3600000,
"CASH_ON_DELIVERY", "456 Oak Ave, Brooklyn"));
orderItemDao.insert(new OrderItem((int) orderId3, (int) p4, 1, 199.99));

// --- Demo Support Chat ---
supportMessageDao.insert(new SupportMessage((int) custId, "CUSTOMER",
"Hi! Do you have this jacket in size M?", null, now - 7200000, (int) custId));
supportMessageDao.insert(new SupportMessage((int) managerId, "STORE_MANAGER",
"Hello! Yes, we have the Leather Jacket in size M. Would you like to place an order?", null, now -
3600000, (int) custId));
supportMessageDao.insert(new SupportMessage((int) custId, "CUSTOMER",
"Yes please! How long does delivery take?", null, now - 1800000, (int) custId));
});
}
}
```

core/BaseActivity.java

```
package com.storepilot.core;

import android.view.View;

import androidx.appcompat.app.AppCompatActivity;

public class BaseActivity extends AppCompatActivity {

protected void hideViewIfUnauthorized(View view, String permission) {
if (!PermissionManager.currentUserHasPermission(permission)) {
view.setVisibility(View.GONE);
} else {
view.setVisibility(View.VISIBLE);
}
}

protected boolean checkPermission(String permission) {
return PermissionManager.currentUserHasPermission(permission);
}
}
```

core/SessionManager.java

```
package com.storepilot.core;

import com.storepilot.db.entities.User;

public class SessionManager {

private static SessionManager instance;
private User loggedInUser;

private SessionManager() {}

public static synchronized SessionManager getInstance() {
if (instance == null) {
instance = new SessionManager();
}
return instance;
}

public User getLoggedInUser() {
return loggedInUser;
}

public void setLoggedInUser(User user) {
this.loggedInUser = user;
}

public void logout() {
this.loggedInUser = null;
}

public boolean isLoggedIn() {
return loggedInUser != null;
}

public String getUserRole() {
if (loggedInUser != null) {
return loggedInUser.getRole();
}
return null;
}
```

```
}
```

core/PermissionManager.java

```
package com.storepilot.core;

import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class PermissionManager {

// Permission constants
public static final String MANAGE_USERS = "MANAGE_USERS";
public static final String MANAGE_PRODUCTS = "MANAGE_PRODUCTS";
public static final String VIEW_PRODUCTS = "VIEW_PRODUCTS";
public static final String CREATE_SALE = "CREATE_SALE";
public static final String VIEW_SALES_HISTORY = "VIEW_SALES_HISTORY";
public static final String MANAGE_PURCHASES = "MANAGE_PURCHASES";
public static final String VIEW_REPORTS = "VIEW_REPORTS";
public static final String MANAGE_SEASONS = "MANAGE_SEASONS";
public static final String VIEW_MARKETING = "VIEW_MARKETING";
public static final String CREATE_PRIVATE_TASK = "CREATE_PRIVATE_TASK";
public static final String VIEW_TEAM_TASKS = "VIEW_TEAM_TASKS";
public static final String MANAGE_TASKS = "MANAGE_TASKS";
public static final String VIEW_ADMIN = "VIEW_ADMIN";

// Role constants
public static final String OWNER = "OWNER";
public static final String STORE_MANAGER = "STORE_MANAGER";
public static final String SHIFT_MANAGER = "SHIFT_MANAGER";
public static final String EMPLOYEE = "EMPLOYEE";

private static final Map<String, List<String>> permissionMap = new HashMap<>();

static {
permissionMap.put(MANAGE_USERS, Arrays.asList(OWNER));
permissionMap.put(MANAGE_PRODUCTS, Arrays.asList(OWNER, STORE_MANAGER));
permissionMap.put(VIEW_PRODUCTS, Arrays.asList(OWNER, STORE_MANAGER, SHIFT_MANAGER, EMPLOYEE));
permissionMap.put(CREATE_SALE, Arrays.asList(OWNER, STORE_MANAGER, SHIFT_MANAGER, EMPLOYEE));
permissionMap.put(VIEW_SALES_HISTORY, Arrays.asList(OWNER, STORE_MANAGER, SHIFT_MANAGER));
permissionMap.put(MANAGE_PURCHASES, Arrays.asList(OWNER, STORE_MANAGER));
permissionMap.put(VIEW_REPORTS, Arrays.asList(OWNER, STORE_MANAGER));
```

```
permissionMap.put(MANAGE_SEASONS, Arrays.asList(OWNER, STORE_MANAGER));
permissionMap.put(VIEW_MARKETING, Arrays.asList(OWNER, STORE_MANAGER));
permissionMap.put(CREATE_PRIVATE_TASK, Arrays.asList(OWNER, STORE_MANAGER, SHIFT_MANAGER, EMPLOYEE));
permissionMap.put(VIEW_TEAM_TASKS, Arrays.asList(OWNER, STORE_MANAGER, SHIFT_MANAGER));
permissionMap.put(MANAGE_TASKS, Arrays.asList(OWNER, STORE_MANAGER));
permissionMap.put(VIEW_ADMIN, Arrays.asList(OWNER));
}

public static boolean hasPermission(String role, String permission) {
List<String> allowedRoles = permissionMap.get(permission);
if (allowedRoles == null) return false;
return allowedRoles.contains(role);
}

public static boolean currentUserHasPermission(String permission) {
SessionManager session = SessionManager.getInstance();
String role = session.getUserRole();
if (role == null) return false;
return hasPermission(role, permission);
}
}
```

auth/CryptoUtil.java

```
package com.storepilot.auth;

import android.util.Base64;

import java.security.NoSuchAlgorithmException;
import java.security.SecureRandom;
import java.security.spec.InvalidKeySpecException;

import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.PBEKeySpec;

public class CryptoUtil {

private static final int ITERATIONS = 65536;
private static final int KEY_LENGTH = 256;
private static final String ALGORITHM = "PBKDF2WithHmacSHA256";
private static final int SALT_BYTES = 16;

public static String generateSalt() {
SecureRandom random = new SecureRandom();
byte[] salt = new byte[SALT_BYTES];
random.nextBytes(salt);
return Base64.encodeToString(salt, Base64.NO_WRAP);
}

public static String hashPassword(String password, String salt) {
try {
byte[] saltBytes = Base64.decode(salt, Base64.NO_WRAP);
PBEKeySpec spec = new PBEKeySpec(
password.toCharArray(), saltBytes, ITERATIONS, KEY_LENGTH);
SecretKeyFactory factory = SecretKeyFactory.getInstance(ALGORITHM);
byte[] hash = factory.generateSecret(spec).getEncoded();
spec.clearPassword();
return Base64.encodeToString(hash, Base64.NO_WRAP);
} catch (NoSuchAlgorithmException | InvalidKeySpecException e) {
throw new RuntimeException("Error hashing password", e);
}
}

public static boolean verifyPassword(String password, String salt, String expectedHash) {
```

```
String actualHash = hashPassword(password, salt);
return actualHash.equals(expectedHash);
}
}
```

auth/WelcomeActivity.java

```
package com.storepilot.auth;

import android.content.Intent;
import android.os.Bundle;
import android.widget.Button;

import com.storepilot.R;
import com.storepilot.core.BaseActivity;
import com.storepilot.core.SessionManager;

// First screen users see when they open the app
public class WelcomeActivity extends BaseActivity {

@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);

// If user is already logged in, skip the welcome screen
if (SessionManager.getInstance().isLoggedIn()) {
navigateByRole();
return;
}

setContentView(R.layout.activity_welcome);

Button btnLogin = findViewById(R.id.btnLogin);
Button btnRegister = findViewById(R.id.btnRegister);

// Go to login screen
btnLogin.setOnClickListener(v ->
startActivity(new Intent(this, LoginActivity.class)));

// Go to role selection, then registration
btnRegister.setOnClickListener(v ->
startActivity(new Intent(this, RoleSelectionActivity.class)));
}

// Route user to the right app based on their role
private void navigateByRole() {
String role = SessionManager.getInstance().getUserRole();
```

```
if ("CUSTOMER".equals(role)) {
// Customers go to the shopping interface
startActivity(new Intent(this, com.storepilot.customer.CustomerMainActivity.class));
} else {
// Managers and owners go to the management dashboard
startActivity(new Intent(this, com.storepilot.MainActivity.class));
}
finish();
}
}
```

auth/RoleSelectionActivity.java

```
package com.storepilot.auth;

import android.content.Intent;
import android.os.Bundle;
import android.widget.Button;

import com.storepilot.R;
import com.storepilot.core.BaseActivity;

public class RoleSelectionActivity extends BaseActivity {

public static final String EXTRA_ROLE = "extra_role";

@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_role_selection);

Button btnOwner = findViewById(R.id.btnRoleOwner);
Button btnCustomer = findViewById(R.id.btnRoleCustomer);

btnOwner.setOnClickListener(v -> openRegister("Owner"));
btnCustomer.setOnClickListener(v -> openRegister("Customer"));
}

private void openRegister(String role) {
Intent intent = new Intent(this, RegisterActivity.class);
intent.putExtra(EXTRA_ROLE, role);
startActivity(intent);
}
}
```

auth/LoginActivity.java

```
package com.storepilot.auth;

import android.content.Intent;
import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;

import androidx.lifecycle.ViewModelProvider;

import com.storepilot.MainActivity;
import com.storepilot.R;
import com.storepilot.core.BaseActivity;
import com.storepilot.core.SessionManager;
import com.storepilot.customer.CustomerMainActivity;
import com.storepilot.viewmodels.AuthViewModel;

// Login screen — validates credentials and routes user to the right app experience
public class LoginActivity extends BaseActivity {

private EditText etUsername, etPassword;
private Button btnLogin;
private TextView tvRegisterLink, tvForgotPassword;
private AuthViewModel authViewModel;

@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_login);

// Find UI elements
etUsername = findViewById(R.id.etUsername);
etPassword = findViewById(R.id.etPassword);
btnLogin = findViewById(R.id.btnLogin);
tvRegisterLink = findViewById(R.id.tvRegisterLink);
tvForgotPassword = findViewById(R.id.tvForgotPassword);

authViewModel = new ViewModelProvider(this).get(AuthViewModel.class);
```

```
// If no accounts exist yet, go to setup screen
authViewModel.needsSetup.observe(this, needsSetup -> {
if (Boolean.TRUE.equals(needsSetup)) {
startActivity(new Intent(this, SetupActivity.class));
finish();
}
});

// Login succeeded — route to the correct screen based on role
authViewModel.loginSuccess.observe(this, success -> {
if (Boolean.TRUE.equals(success)) {
String role = SessionManager.getInstance().getUserRole();
if ("CUSTOMER".equals(role)) {
// Customers go to the store shopping interface
startActivity(new Intent(this, CustomerMainActivity.class));
} else {
// Managers and owners go to the dashboard
startActivity(new Intent(this, MainActivity.class));
}
finish();
}
});

// Show login error message
authViewModel.loginError.observe(this, error -> {
if (error != null) {
Toast.makeText(this, error, Toast.LENGTH_SHORT).show();
}
});

// Attempt login when button is pressed
btnLogin.setOnClickListener(v -> {
String username = etUsername.getText().toString().trim();
String password = etPassword.getText().toString();
if (username.isEmpty() || password.isEmpty()) {
Toast.makeText(this, getString(R.string.error_fill_all_fields), Toast.LENGTH_SHORT).show();
return;
}
authViewModel.login(username, password);
});
```

```


// Navigate to register screen
if (tvRegisterLink != null) {
tvRegisterLink.setOnClickListener(v ->
startActivity(new Intent(this, RegisterActivity.class)));
}

// Forgot password placeholder
if (tvForgotPassword != null) {
tvForgotPassword.setOnClickListener(v ->
Toast.makeText(this, "Please contact your store admin.", Toast.LENGTH_SHORT).show());
}

// Check if database needs initial setup
authViewModel.checkNeedsSetup();
}
}
```

auth/RegisterActivity.java

```
package com.storepilot.auth;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.ArrayAdapter;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Spinner;
import android.widget.TextView;
import android.widget.Toast;

import com.storepilot.R;
import com.storepilot.core.AppDatabase;
import com.storepilot.core.BaseActivity;
import com.storepilot.db.entities.User;

public class RegisterActivity extends BaseActivity {

private EditText etFullName, etUsername, etEmail, etPhone, etPassword, etConfirmPassword;
private Spinner spinnerRole;
private Button btnRegister;
private TextView tvLoginLink, tvSelectedRole, tvRoleLabel;
private String preSelectedRole;

@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_register);

etFullName = findViewById(R.id.etFullName);
etUsername = findViewById(R.id.etUsername);
etEmail = findViewById(R.id.etEmail);
etPhone = findViewById(R.id.etPhone);
etPassword = findViewById(R.id.etPassword);
etConfirmPassword = findViewById(R.id.etConfirmPassword);
spinnerRole = findViewById(R.id.spinnerRole);
btnRegister = findViewById(R.id.btnRegister);
tvLoginLink = findViewById(R.id.tvLoginLink);
tvSelectedRole = findViewById(R.id.tvSelectedRole);
```

```
tvRoleLabel = findViewById(R.id.tvRoleLabel);

preSelectedRole = getIntent().getStringExtra(RoleSelectionActivity.EXTRA_ROLE);

if (preSelectedRole != null) {
// Role was chosen on the previous screen — hide spinner, show label
tvSelectedRole.setText("Signing up as: " + preSelectedRole);
tvSelectedRole.setVisibility(View.VISIBLE);
tvRoleLabel.setVisibility(View.GONE);
spinnerRole.setVisibility(View.GONE);
} else {
// No pre-selected role — show full spinner
String[] roles = {"Customer", "Manager", "Owner"};
ArrayAdapter<String> adapter = new ArrayAdapter<>(this,
android.R.layout.simple_spinner_item, roles);
adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
spinnerRole.setAdapter(adapter);
}

// Handle register button click
btnRegister.setOnClickListener(v -> attemptRegister());

// Navigate back to login
tvLoginLink.setOnClickListener(v -> {
startActivity(new Intent(this, LoginActivity.class));
finish();
});
}

// Validate the form and create the account
private void attemptRegister() {
String fullName = etFullName.getText().toString().trim();
String username = etUsername.getText().toString().trim();
String email = etEmail.getText().toString().trim();
String phone = etPhone.getText().toString().trim();
String password = etPassword.getText().toString();
String confirmPassword = etConfirmPassword.getText().toString();

// Check all required fields are filled
if (fullName.isEmpty() || username.isEmpty() || email.isEmpty() ||
```

```
password.isEmpty() || confirmPassword.isEmpty()) {
Toast.makeText(this, getString(R.string.error_fill_all_fields), Toast.LENGTH_SHORT).show();
return;
}

// Check passwords match
if (!password.equals(confirmPassword)) {
Toast.makeText(this, getString(R.string.error_passwords_no_match), Toast.LENGTH_SHORT).show();
return;
}

// Minimum password length
if (password.length() < 6) {
Toast.makeText(this, getString(R.string.error_password_too_short), Toast.LENGTH_SHORT).show();
return;
}

// Use pre-selected role if available, otherwise read from spinner
String selectedRole = preSelectedRole != null
? preSelectedRole
: (spinnerRole.getSelectedItem() != null ? spinnerRole.getSelectedItem().toString() : "Customer");
String role;
switch (selectedRole) {
case "Manager": role = "STORE_MANAGER"; break;
case "Owner": role = "OWNER"; break;
default: role = "CUSTOMER"; break;
}

// Disable button to prevent double-tap
btnRegister.setEnabled(false);

String finalRole = role;
String finalEmail = email;
String finalPassword = password;

AppDatabase.dbExecutor.execute(() -> {
String salt = CryptoUtil.generateSalt();
String hash = CryptoUtil.hashPassword(finalPassword, salt);

User newUser = new User(fullName, username, finalEmail, phone, hash, salt, finalRole,
```

```
System.currentTimeMillis());

try {
AppDatabase.getInstance(getApplication()).userDao().insert(newUser);

// Also register with Firebase Authentication if configured
FirebaseAuthHelper.init(getApplicationContext());
FirebaseAuthHelper.signUp(finalEmail, finalPassword, new FirebaseAuthHelper.AuthCallback() {
@Override
public void onSuccess(String uid) {
runOnUiThread(() -> {
Toast.makeText(RegisterActivity.this, "Account created! Please sign in.", Toast.LENGTH_SHORT).show();
startActivity(new Intent(RegisterActivity.this, LoginActivity.class));
finish();
});
}

@Override
public void onFailure(String error) {
// Firebase failed but local account was created — proceed anyway
runOnUiThread(() -> {
Toast.makeText(RegisterActivity.this, "Account created! Please sign in.", Toast.LENGTH_SHORT).show();
startActivity(new Intent(RegisterActivity.this, LoginActivity.class));
finish();
});
}
});
} catch (Exception e) {
runOnUiThread(() -> {
btnRegister.setEnabled(true);
Toast.makeText(this, "Username or email already taken.", Toast.LENGTH_SHORT).show();
});
}
});
}
}
```

auth/SetupActivity.java

```
package com.storepilot.auth;

import android.content.Intent;
import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.lifecycle.ViewModelProvider;

import com.storepilot.R;
import com.storepilot.core.BaseActivity;
import com.storepilot.viewmodels.AuthViewModel;

public class SetupActivity extends BaseActivity {

private EditText etOwnerUsername, etOwnerPassword, etConfirmPassword;
private Button btnCreate;
private AuthViewModel authViewModel;

@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_setup);

etOwnerUsername = findViewById(R.id.etOwnerUsername);
etOwnerPassword = findViewById(R.id.etOwnerPassword);
etConfirmPassword = findViewById(R.id.etConfirmPassword);
btnCreate = findViewById(R.id.btnCreate);

authViewModel = new ViewModelProvider(this).get(AuthViewModel.class);

authViewModel.setupComplete.observe(this, complete -> {
if (Boolean.TRUE.equals(complete)) {
Toast.makeText(this, getString(R.string.setup_complete), Toast.LENGTH_SHORT).show();
startActivity(new Intent(this, LoginActivity.class));
finish();
}
});
```

```
btnCreate.setOnClickListener(v -> {
String username = etOwnerUsername.getText().toString().trim();
String password = etOwnerPassword.getText().toString();
String confirm = etConfirmPassword.getText().toString();

if (username.isEmpty() || password.isEmpty() || confirm.isEmpty()) {
Toast.makeText(this, getString(R.string.error_fill_all_fields), Toast.LENGTH_SHORT).show();
return;
}
if (!password.equals(confirm)) {
Toast.makeText(this, getString(R.string.error_passwords_no_match), Toast.LENGTH_SHORT).show();
return;
}
if (password.length() < 6) {
Toast.makeText(this, getString(R.string.error_password_too_short), Toast.LENGTH_SHORT).show();
return;
}

authViewModel.setupOwner(username, password, false);
});
}
}
```

viewmodels/AuthViewModel.java

```
package com.storepilot.viewmodels;

import android.app.Application;

import androidx.annotation.NonNull;
import androidx.lifecycle.AndroidViewModel;
import androidx.lifecycle.MutableLiveData;

import com.storepilot.auth.CryptoUtil;
import com.storepilot.core.AppDatabase;
import com.storepilot.core.SessionManager;
import com.storepilot.db.entities.User;
import com.storepilot.repositories.UserRepository;

public class AuthViewModel extends AndroidViewModel {

private final UserRepository userRepository;
public final MutableLiveData<String> loginError = new MutableLiveData<>();
public final MutableLiveData<Boolean> loginSuccess = new MutableLiveData<>();
public final MutableLiveData<Boolean> setupComplete = new MutableLiveData<>();
public final MutableLiveData<Boolean> needsSetup = new MutableLiveData<>();

public AuthViewModel(@NonNull Application application) {
super(application);
userRepository = new UserRepository(application);
}

public void checkNeedsSetup() {
AppDatabase.dbExecutor.execute(() -> {
// Show setup only on a completely fresh install (no users at all)
int totalUsers = userRepository.getUserCountSync();
needsSetup.postValue(totalUsers == 0);
});
}

public void login(String usernameOrEmail, String password) {
AppDatabase.dbExecutor.execute(() -> {
// Try username first, then email (supports both login methods)
User user = userRepository.findByUsername(usernameOrEmail);
if (user == null) {
```

```
user = userRepository.findByEmail(usernameOrEmail);
}
if (user == null) {
loginError.postValue("Invalid username or password");
return;
}
if (!CryptoUtil.verifyPassword(password, user.getSalt(), user.getPasswordHash())) {
loginError.postValue("Invalid username or password");
return;
}
// Save the logged-in user in session
SessionManager.getInstance().setLoggedInUser(user);
loginSuccess.postValue(true);
});
}

public void setupOwner(String username, String password, boolean demoMode) {
AppDatabase.dbExecutor.execute(() -> {
String salt = CryptoUtil.generateSalt();
String hash = CryptoUtil.hashPassword(password, salt);
User owner = new User(username, hash, salt, "OWNER", System.currentTimeMillis());
userRepository.insert(owner);
if (demoMode) {
AppDatabase.getInstance(getApplication()).seedDemoData();
}
setupComplete.postValue(true);
});
}
}
```

db/dao/UserDao.java

```
package com.storepilot.db.dao;

import androidx.lifecycle.LiveData;
import androidx.room.Dao;
import androidx.room.Delete;
import androidx.room.Insert;
import androidx.room.OnConflictStrategy;
import androidx.room.Query;
import androidx.room.Update;

import com.storepilot.db.entities.User;

import java.util.List;

@Dao
public interface UserDao {

@Insert(onConflict = OnConflictStrategy.REPLACE)
long insert(User user);

@Update
void update(User user);

@Delete
void delete(User user);

@Query("SELECT * FROM users WHERE username = :username LIMIT 1")
User findByUsername(String username);

@Query("SELECT * FROM users WHERE id = :id LIMIT 1")
User findById(int id);

@Query("SELECT * FROM users ORDER BY createdAt ASC")
LiveData<List<User>> getAllUsers();

@Query("SELECT COUNT(*) FROM users WHERE role = 'OWNER'")
int getOwnerCount();

// Find user by email (used for email-based login)
@Query("SELECT * FROM users WHERE email = :email LIMIT 1")
```

```
User findByEmail(String email);

// Count total registered users (for dashboard stats)
@Query("SELECT COUNT(*) FROM users")
LiveData<Integer> getTotalUserCount();

// Synchronous count for setup check on background thread
@Query("SELECT COUNT(*) FROM users")
int getUserCountSync();

// Count only customers (for dashboard stats)
@Query("SELECT COUNT(*) FROM users WHERE role = 'CUSTOMER'")
LiveData<Integer> getCustomerCount();
}
```

repositories/UserRepository.java

```
package com.storepilot.repositories;

import android.app.Application;

import androidx.lifecycle.LiveData;

import com.storepilot.core.AppDatabase;
import com.storepilot.db.dao.UserDao;
import com.storepilot.db.entities.User;

import java.util.List;

public class UserRepository {

private final UserDao userDao;

public UserRepository(Application application) {
AppDatabase db = AppDatabase.getInstance(application);
userDao = db.userDao();
}

public LiveData<List<User>> getAllUsers() {
return userDao.getAllUsers();
}

public void insert(User user) {
AppDatabase.dbExecutor.execute(() -> userDao.insert(user));
}

public void update(User user) {
AppDatabase.dbExecutor.execute(() -> userDao.update(user));
}

public void delete(User user) {
AppDatabase.dbExecutor.execute(() -> userDao.delete(user));
}

public User findByUsername(String username) {
return userDao.findByUsername(username);
}
```

```

public int getOwnerCount() {
return userDao.getOwnerCount();
}

// Returns total user count synchronously (for setup check on background thread)
public int getUserCountSync() {
return userDao.getUserCountSync();
}

// Find user by email address (used for email-based login)
public User findByEmail(String email) {
return userDao.findByEmail(email);
}
}
```

55 Products & Cart

inventory/ + customer/ — DAOs, Repos, ViewModels, Fragments, Activities

Complete product management and customer shopping layer: product DAOs with low-stock queries, the cart/order pipeline, checkout flow, order history, and customer-facing browsing screens.

db/dao/ProductDao.java

```
package com.storepilot.db.dao;

import androidx.lifecycle.LiveData;
import androidx.room.Dao;
import androidx.room.Delete;
import androidx.room.Insert;
import androidx.room.OnConflictStrategy;
import androidx.room.Query;
import androidx.room.Update;

import com.storepilot.db.entities.Product;

import java.util.List;

@Dao
public interface ProductDao {

@Insert(onConflict = OnConflictStrategy.REPLACE)
long insert(Product product);

@Update
void update(Product product);

@Delete
void delete(Product product);

@Query("SELECT * FROM products ORDER BY name ASC")
LiveData<List<Product>> getAllProducts();

@Query("SELECT * FROM products WHERE id = :id LIMIT 1")
LiveData<Product> getById(int id);

@Query("SELECT * FROM products WHERE quantity <= :threshold ORDER BY quantity ASC")
LiveData<List<Product>> getLowStockProducts(int threshold);

@Query("SELECT p.* FROM products p " +
"INNER JOIN (SELECT productId, SUM(quantity) as totalSold FROM sales " +
"WHERE saleDate BETWEEN :startDate AND :endDate GROUP BY productId " +
"ORDER BY totalSold DESC LIMIT 10) s ON p.id = s.productId " +
"ORDER BY s.totalSold DESC")
```

```
LiveData<List<Product>> getTopSellingProducts(long startDate, long endDate);

@Query("SELECT * FROM products WHERE id = :id LIMIT 1")
Product getByIdSync(int id);

// Alias used in ProductDetailFragment
@Query("SELECT * FROM products WHERE id = :id LIMIT 1")
Product findById(int id);

// Count products with low stock (for dashboard alert)
@Query("SELECT COUNT(*) FROM products WHERE quantity <= :threshold")
LiveData<Integer> getLowStockCount(int threshold);
}
```

db/dao/CartDao.java

```
package com.storepilot.db.dao;

import androidx.lifecycle.LiveData;
import androidx.room.Dao;
import androidx.room.Insert;
import androidx.room.Query;
import androidx.room.Update;

import com.storepilot.db.entities.CartItem;

import java.util.List;

@Dao
public interface CartDao {

// Add a new product to the cart
@Insert
void insert(CartItem cartItem);

// Update quantity of an existing cart item
@Update
void update(CartItem cartItem);

// Get all cart items for a specific customer (live, updates UI automatically)
@Query("SELECT * FROM cart_items WHERE customerId = :customerId")
LiveData<List<CartItem>> getCartItems(int customerId);

// Check if a product is already in the cart
@Query("SELECT * FROM cart_items WHERE customerId = :customerId AND productId = :productId LIMIT 1")
CartItem getCartItemByProduct(int customerId, int productId);

// Remove a specific item from the cart
@Query("DELETE FROM cart_items WHERE id = :cartItemId")
void deleteById(int cartItemId);

// Clear all items from cart after checkout
@Query("DELETE FROM cart_items WHERE customerId = :customerId")
void clearCart(int customerId);

// Count how many items are in the cart (for badge on cart icon)
```

```
@Query("SELECT COUNT(*) FROM cart_items WHERE customerId = :customerId")
LiveData<Integer> getCartItemCount(int customerId);
}
```

db/dao/OrderDao.java

```
package com.storepilot.db.dao;

import androidx.lifecycle.LiveData;
import androidx.room.Dao;
import androidx.room.Insert;
import androidx.room.Query;
import androidx.room.Update;

import com.storepilot.db.entities.Order;

import java.util.List;

@Dao
public interface OrderDao {

// Insert a new order and return the new row ID
@Insert
long insert(Order order);

// Update an existing order (used to change status)
@Update
void update(Order order);

// Get all orders for a specific customer (for order history screen)
@Query("SELECT * FROM orders WHERE customerId = :customerId ORDER BY createdAt DESC")
LiveData<List<Order>> getOrdersByCustomer(int customerId);

// Get all orders (for manager's order management screen)
@Query("SELECT * FROM orders ORDER BY createdAt DESC")
LiveData<List<Order>> getAllOrders();

// Get orders with a specific status (e.g., filter by PENDING)
@Query("SELECT * FROM orders WHERE status = :status ORDER BY createdAt DESC")
LiveData<List<Order>> getOrdersByStatus(String status);

// Get total revenue from delivered orders today
@Query("SELECT COALESCE(SUM(totalPrice), 0) FROM orders WHERE status = 'DELIVERED' AND createdAt >=
:startOfDay")
LiveData<Double> getTodayRevenue(long startOfDay);

// Count how many orders were placed today
```

```
@Query("SELECT COUNT(*) FROM orders WHERE createdAt >= :startOfDay")
LiveData<Integer> getTodayOrderCount(long startOfDay);

// Get a single order by ID
@Query("SELECT * FROM orders WHERE id = :orderId")
Order getOrderById(int orderId);
}
```

db/dao/OrderItemDao.java

```
package com.storepilot.db.dao;

import androidx.room.Dao;
import androidx.room.Insert;
import androidx.room.Query;

import com.storepilot.db.entities.OrderItem;

import java.util.List;

@Dao
public interface OrderItemDao {

// Save one item that belongs to an order
@Insert
void insert(OrderItem orderItem);

// Get all items that belong to a specific order
@Query("SELECT * FROM order_items WHERE orderId = :orderId")
List<OrderItem> getItemsForOrder(int orderId);

// Count total units sold for a product (for best-seller analytics)
@Query("SELECT COALESCE(SUM(quantity), 0) FROM order_items WHERE productId = :productId")
int getTotalSoldForProduct(int productId);
}
```

db/dao/FavoriteDao.java

```
package com.storepilot.db.dao;

import androidx.lifecycle.LiveData;
import androidx.room.Dao;
import androidx.room.Insert;
import androidx.room.Query;

import com.storepilot.db.entities.Favorite;

import java.util.List;

@Dao
public interface FavoriteDao {

// Add a product to favorites
@Insert
void insert(Favorite favorite);

// Remove a product from favorites by ID
@Query("DELETE FROM favorites WHERE id = :favoriteId")
void deleteById(int favoriteId);

// Remove a specific product from a customer's favorites
@Query("DELETE FROM favorites WHERE customerId = :customerId AND productId = :productId")
void deleteByProduct(int customerId, int productId);

// Get all favorites for a customer
@Query("SELECT * FROM favorites WHERE customerId = :customerId ORDER BY addedAt DESC")
LiveData<List<Favorite>> getFavoritesByCustomer(int customerId);

// Check if a product is already favorited by a customer
@Query("SELECT * FROM favorites WHERE customerId = :customerId AND productId = :productId LIMIT 1")
Favorite getFavorite(int customerId, int productId);
}
```

repositories/ProductRepository.java

```
package com.storepilot.repositories;

import android.app.Application;

import androidx.lifecycle.LiveData;

import com.storepilot.core.AppDatabase;
import com.storepilot.db.dao.ProductDao;
import com.storepilot.db.entities.Product;

import java.util.List;

public class ProductRepository {

private final ProductDao productDao;

public ProductRepository(Application application) {
AppDatabase db = AppDatabase.getInstance(application);
productDao = db.productDao();
}

public LiveData<List<Product>> getAllProducts() {
return productDao.getAllProducts();
}

public LiveData<Product> getById(int id) {
return productDao.getById(id);
}

public LiveData<List<Product>> getLowStockProducts(int threshold) {
return productDao.getLowStockProducts(threshold);
}

public LiveData<List<Product>> getTopSellingProducts(long startDate, long endDate) {
return productDao.getTopSellingProducts(startDate, endDate);
}

public void insert(Product product) {
AppDatabase.dbExecutor.execute(() -> productDao.insert(product));
}
```

```

public void update(Product product) {
AppDatabase.dbExecutor.execute(() -> productDao.update(product));
}

public void delete(Product product) {
AppDatabase.dbExecutor.execute(() -> productDao.delete(product));
}
}
```

repositories/CartRepository.java

```
package com.storepilot.repositories;

import android.app.Application;

import androidx.lifecycle.LiveData;

import com.storepilot.core.AppDatabase;
import com.storepilot.db.dao.CartDao;
import com.storepilot.db.entities.CartItem;

import java.util.List;

// Handles all cart operations between ViewModel and the database
public class CartRepository {

private final CartDao cartDao;

public CartRepository(Application application) {
// Get the database and its cart DAO
AppDatabase db = AppDatabase.getInstance(application);
cartDao = db.cartDao();
}

// Add an item to the cart or increase quantity if already there
public void addToCart(int customerId, int productId) {
AppDatabase.dbExecutor.execute(() -> {
CartItem existing = cartDao.getCartItemByProduct(customerId, productId);
if (existing != null) {
// Product already in cart — just increase quantity
existing.setQuantity(existing.getQuantity() + 1);
cartDao.update(existing);
} else {
// New cart item
cartDao.insert(new CartItem(customerId, productId, 1));
}
});
}

// Remove one unit of a product, or delete the item if quantity reaches zero
public void removeOneFromCart(CartItem cartItem) {
```

```
AppDatabase.dbExecutor.execute(() -> {
if (cartItem.getQuantity() > 1) {
cartItem.setQuantity(cartItem.getQuantity() - 1);
cartDao.update(cartItem);
} else {
cartDao.deleteById(cartItem.getId());
}
});
}

// Completely remove an item from the cart
public void removeFromCart(int cartItemId) {
AppDatabase.dbExecutor.execute(() -> cartDao.deleteById(cartItemId));
}

// Empty the whole cart (called after checkout)
public void clearCart(int customerId) {
AppDatabase.dbExecutor.execute(() -> cartDao.clearCart(customerId));
}

// Get live cart items for the UI
public LiveData<List<CartItem>> getCartItems(int customerId) {
return cartDao.getCartItems(customerId);
}

// Get the cart item count for the badge icon
public LiveData<Integer> getCartItemCount(int customerId) {
return cartDao.getCartItemCount(customerId);
}
}
```

repositories/OrderRepository.java

```
package com.storepilot.repositories;

import android.app.Application;

import androidx.lifecycle.LiveData;

import com.storepilot.core.AppDatabase;
import com.storepilot.db.dao.CartDao;
import com.storepilot.db.dao.OrderDao;
import com.storepilot.db.dao.OrderItemDao;
import com.storepilot.db.dao.ProductDao;
import com.storepilot.db.entities.CartItem;
import com.storepilot.db.entities.Order;
import com.storepilot.db.entities.OrderItem;
import com.storepilot.db.entities.Product;

import java.util.List;

// Handles creating and reading orders
public class OrderRepository {

private final OrderDao orderDao;
private final OrderItemDao orderItemDao;
private final CartDao cartDao;
private final ProductDao productDao;

public OrderRepository(Application application) {
AppDatabase db = AppDatabase.getInstance(application);
orderDao = db.orderDao();
orderItemDao = db.orderItemDao();
cartDao = db.cartDao();
productDao = db.productDao();
}

// Place an order using the items currently in the cart
public void placeOrder(int customerId, String paymentMethod,
String shippingAddress, List<CartItem> cartItems,
List<Product> products, Runnable onSuccess) {
AppDatabase.dbExecutor.execute(() -> {
// Calculate total price from cart
```

```
double total = 0;
for (int i = 0; i < cartItems.size(); i++) {
CartItem ci = cartItems.get(i);
// Find matching product to get its price
for (Product p : products) {
if (p.getId() == ci.getProductId()) {
total += p.getPrice() * ci.getQuantity();
break;
}
}
}

// Create the order record
Order order = new Order(customerId, total, "PENDING",
System.currentTimeMillis(), paymentMethod, shippingAddress);
long orderId = orderDao.insert(order);

// Save each cart item as an order item
for (CartItem ci : cartItems) {
for (Product p : products) {
if (p.getId() == ci.getProductId()) {
orderItemDao.insert(new OrderItem((int) orderId,
p.getId(), ci.getQuantity(), p.getPrice()));
// Reduce stock quantity
p.quantity = Math.max(0, p.quantity - ci.getQuantity());
productDao.update(p);
break;
}
}
}

// Clear the cart after successful order placement
cartDao.clearCart(customerId);

// Notify caller on completion
if (onSuccess != null) onSuccess.run();
});
}

// Update order status (manager action: PENDING → PROCESSING → SHIPPED → DELIVERED)
```

```
public void updateOrderStatus(Order order, String newStatus) {
AppDatabase.dbExecutor.execute(() -> {
order.status = newStatus;
orderDao.update(order);
});
}

// Get customer's order history
public LiveData<List<Order>> getOrdersByCustomer(int customerId) {
return orderDao.getOrdersByCustomer(customerId);
}

// Get all orders (for manager view)
public LiveData<List<Order>> getAllOrders() {
return orderDao.getAllOrders();
}

// Get today's total revenue
public LiveData<Double> getTodayRevenue(long startOfDay) {
return orderDao.getTodayRevenue(startOfDay);
}

// Get today's order count
public LiveData<Integer> getTodayOrderCount(long startOfDay) {
return orderDao.getTodayOrderCount(startOfDay);
}
}
```

repositories/FavoritesRepository.java

```
package com.storepilot.repositories;

import android.app.Application;

import androidx.lifecycle.LiveData;

import com.storepilot.core.AppDatabase;
import com.storepilot.db.dao.FavoriteDao;
import com.storepilot.db.entities.Favorite;

import java.util.List;

// Handles customer wishlist/favorites
public class FavoritesRepository {

private final FavoriteDao favoriteDao;

public FavoritesRepository(Application application) {
AppDatabase db = AppDatabase.getInstance(application);
favoriteDao = db.favoriteDao();
}

// Toggle favorite: add if not there, remove if already favorited
public void toggleFavorite(int customerId, int productId) {
AppDatabase.dbExecutor.execute(() -> {
Favorite existing = favoriteDao.getFavorite(customerId, productId);
if (existing != null) {
// Already favorited — remove it
favoriteDao.deleteByProduct(customerId, productId);
} else {
// Not favorited yet — add it
favoriteDao.insert(new Favorite(customerId, productId, System.currentTimeMillis()));
}
});
}

// Check if a product is in the customer's favorites
public boolean isFavorite(int customerId, int productId) {
return favoriteDao.getFavorite(customerId, productId) != null;
}
```

```

// Get all favorites for the wishlist screen (live updates)
public LiveData<List<Favorite>> getFavoritesByCustomer(int customerId) {
return favoriteDao.getFavoritesByCustomer(customerId);
}
}
```

viewmodels/ProductViewModel.java

```
package com.storepilot.viewmodels;

import android.app.Application;

import androidx.annotation.NonNull;
import androidx.lifecycle.AndroidViewModel;
import androidx.lifecycle.LiveData;

import com.storepilot.db.entities.Product;
import com.storepilot.repositories.ProductRepository;

import java.util.List;

public class ProductViewModel extends AndroidViewModel {

private final ProductRepository repository;

public ProductViewModel(@NonNull Application application) {
super(application);
repository = new ProductRepository(application);
}

public LiveData<List<Product>> getAllProducts() {
return repository.getAllProducts();
}

public LiveData<Product> getById(int id) {
return repository.getById(id);
}

public LiveData<List<Product>> getLowStockProducts(int threshold) {
return repository.getLowStockProducts(threshold);
}

public LiveData<List<Product>> getTopSellingProducts(long startDate, long endDate) {
return repository.getTopSellingProducts(startDate, endDate);
}

public void insert(Product product) {
repository.insert(product);
```

```
}

public void update(Product product) {
repository.update(product);
}

public void delete(Product product) {
repository.delete(product);
}
}
```

viewmodels/CartViewModel.java

```
package com.storepilot.viewmodels;

import android.app.Application;

import androidx.annotation.NonNull;
import androidx.lifecycle.AndroidViewModel;
import androidx.lifecycle.LiveData;

import com.storepilot.db.entities.CartItem;
import com.storepilot.repositories.CartRepository;

import java.util.List;

// ViewModel for the shopping cart — survives screen rotations
public class CartViewModel extends AndroidViewModel {

private final CartRepository cartRepository;

public CartViewModel(@NonNull Application application) {
super(application);
cartRepository = new CartRepository(application);
}

// Add a product to cart (or increase quantity)
public void addToCart(int customerId, int productId) {
cartRepository.addToCart(customerId, productId);
}

// Decrease quantity by 1, or remove if last unit
public void removeOne(CartItem cartItem) {
cartRepository.removeOneFromCart(cartItem);
}

// Remove the whole item from the cart
public void removeFromCart(int cartItemId) {
cartRepository.removeFromCart(cartItemId);
}

// Empty the cart completely
public void clearCart(int customerId) {
```

```
cartRepository.clearCart(customerId);
}

// Get live list of cart items for the UI
public LiveData<List<CartItem>> getCartItems(int customerId) {
return cartRepository.getCartItems(customerId);
}

// Get total item count for the cart badge
public LiveData<Integer> getCartItemCount(int customerId) {
return cartRepository.getCartItemCount(customerId);
}
}
```

viewmodels/OrderViewModel.java

```
package com.storepilot.viewmodels;

import android.app.Application;

import androidx.annotation.NonNull;
import androidx.lifecycle.AndroidViewModel;
import androidx.lifecycle.LiveData;
import androidx.lifecycle.MutableLiveData;

import com.storepilot.db.entities.CartItem;
import com.storepilot.db.entities.Order;
import com.storepilot.db.entities.Product;
import com.storepilot.repositories.OrderRepository;

import java.util.List;

// ViewModel for order operations (placing and managing orders)
public class OrderViewModel extends AndroidViewModel {

private final OrderRepository orderRepository;

// Signals that an order was placed successfully (used to show confirmation)
public final MutableLiveData<Boolean> orderPlaced = new MutableLiveData<>();

public OrderViewModel(@NonNull Application application) {
super(application);
orderRepository = new OrderRepository(application);
}

// Place a new order using the cart contents
public void placeOrder(int customerId, String paymentMethod, String shippingAddress,
List<CartItem> cartItems, List<Product> products) {
orderRepository.placeOrder(customerId, paymentMethod, shippingAddress,
cartItems, products, () -> orderPlaced.postValue(true));
}

// Change the status of an order (manager action)
public void updateOrderStatus(Order order, String newStatus) {
orderRepository.updateOrderStatus(order, newStatus);
}
```

```

// Get all orders for a specific customer
public LiveData<List<Order>> getOrdersByCustomer(int customerId) {
return orderRepository.getOrdersByCustomer(customerId);
}

// Get all orders (manager view)
public LiveData<List<Order>> getAllOrders() {
return orderRepository.getAllOrders();
}

// Get today's revenue total
public LiveData<Double> getTodayRevenue(long startOfDay) {
return orderRepository.getTodayRevenue(startOfDay);
}

// Get today's order count
public LiveData<Integer> getTodayOrderCount(long startOfDay) {
return orderRepository.getTodayOrderCount(startOfDay);
}
}
```

viewmodels/FavoritesViewModel.java

```
package com.storepilot.viewmodels;

import android.app.Application;

import androidx.annotation.NonNull;
import androidx.lifecycle.AndroidViewModel;
import androidx.lifecycle.LiveData;

import com.storepilot.db.entities.Favorite;
import com.storepilot.repositories.FavoritesRepository;

import java.util.List;

// ViewModel for the customer's wishlist/favorites feature
public class FavoritesViewModel extends AndroidViewModel {

private final FavoritesRepository favoritesRepository;

public FavoritesViewModel(@NonNull Application application) {
super(application);
favoritesRepository = new FavoritesRepository(application);
}

// Add or remove a product from favorites
public void toggleFavorite(int customerId, int productId) {
favoritesRepository.toggleFavorite(customerId, productId);
}

// Get live list of favorites for the wishlist screen
public LiveData<List<Favorite>> getFavorites(int customerId) {
return favoritesRepository.getFavoritesByCustomer(customerId);
}
}
```

inventory/ProductListFragment.java

```
package com.storepilot.inventory;

import android.content.Intent;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.lifecycle.ViewModelProvider;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.google.android.material.floatingactionbutton.FloatingActionButton;
import com.storepilot.R;
import com.storepilot.core.PermissionManager;
import com.storepilot.viewmodels.ProductViewModel;

public class ProductListFragment extends Fragment {

private RecyclerView recyclerView;
private FloatingActionButton fabAdd;
private ProductViewModel productViewModel;
private ProductListAdapter adapter;

@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container,
@Nullable Bundle savedInstanceState) {
return inflater.inflate(R.layout.fragment_product_list, container, false);
}

@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
super.onViewCreated(view, savedInstanceState);

recyclerView = view.findViewById(R.id.recyclerProducts);
fabAdd = view.findViewById(R.id.fabAddProduct);
```

```
adapter = new ProductListAdapter(product -> {
ProductDetailsFragment details = ProductDetailsFragment.newInstance(product.getId());
requireActivity().getSupportFragmentManager().beginTransaction()
.replace(R.id.fragmentContainer, details)
.addToBackStack(null)
.commit();
});

recyclerView.setLayoutManager(new LinearLayoutManager(getContext()));
recyclerView.setAdapter(adapter);

productViewModel = new ViewModelProvider(this).get(ProductViewModel.class);
productViewModel.getAllProducts().observe(getViewLifecycleOwner(), products -> {
adapter.setProducts(products);
});

if (PermissionManager.currentUserHasPermission(PermissionManager.MANAGE_PRODUCTS)) {
fabAdd.setVisibility(View.VISIBLE);
} else {
fabAdd.setVisibility(View.GONE);
}

fabAdd.setOnClickListener(v -> {
startActivity(new Intent(getActivity(), AddEditProductActivity.class));
});
}
}
```

inventory/ProductDetailsFragment.java

```
package com.storepilot.inventory;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.lifecycle.ViewModelProvider;

import com.storepilot.R;
import com.storepilot.viewmodels.ProductViewModel;

public class ProductDetailsFragment extends Fragment {

private static final String ARG_PRODUCT_ID = "productId";

public static ProductDetailsFragment newInstance(int productId) {
ProductDetailsFragment fragment = new ProductDetailsFragment();
Bundle args = new Bundle();
args.putInt(ARG_PRODUCT_ID, productId);
fragment.setArguments(args);
return fragment;
}

@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container,
@Nullable Bundle savedInstanceState) {
return inflater.inflate(R.layout.fragment_product_details, container, false);
}

@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
super.onViewCreated(view, savedInstanceState);

TextView tvName = view.findViewById(R.id.tvDetailName);
```

```
TextView tvCategory = view.findViewById(R.id.tvDetailCategory);
TextView tvSize = view.findViewById(R.id.tvDetailSize);
TextView tvColor = view.findViewById(R.id.tvDetailColor);
TextView tvQuantity = view.findViewById(R.id.tvDetailQuantity);
TextView tvPrice = view.findViewById(R.id.tvDetailPrice);
TextView tvCostPrice = view.findViewById(R.id.tvDetailCostPrice);

int productId = getArguments() != null ? getArguments().getInt(ARG_PRODUCT_ID, -1) : -1;
if (productId == -1) return;

ProductViewModel vm = new ViewModelProvider(this).get(ProductViewModel.class);
vm.getById(productId).observe(getViewLifecycleOwner(), product -> {
if (product != null) {
tvName.setText(product.getName());
tvCategory.setText(product.getCategory());
tvSize.setText(product.getSize());
tvColor.setText(product.getColor());
tvQuantity.setText(String.valueOf(product.getQuantity()));
tvPrice.setText(String.format("$%.2f", product.getPrice()));
tvCostPrice.setText(String.format("$%.2f", product.getCostPrice()));
}
});
}
}
```

inventory/AddEditProductActivity.java

```
package com.storepilot.inventory;

import android.os.Bundle;
import android.widget.ArrayAdapter;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Spinner;
import android.widget.Toast;

import androidx.lifecycle.ViewModelProvider;

import com.storepilot.R;
import com.storepilot.core.BaseActivity;
import com.storepilot.db.entities.Product;
import com.storepilot.viewmodels.ProductViewModel;

public class AddEditProductActivity extends BaseActivity {

public static final String EXTRA_PRODUCT_ID = "product_id";

private EditText etName, etCategory, etSize, etColor, etQuantity, etPrice, etCostPrice;
private Button btnSave;
private ProductViewModel productViewModel;
private Product existingProduct = null;

@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_add_edit_product);

etName = findViewById(R.id.etProductName);
etCategory = findViewById(R.id.etProductCategory);
etSize = findViewById(R.id.etProductSize);
etColor = findViewById(R.id.etProductColor);
etQuantity = findViewById(R.id.etProductQuantity);
etPrice = findViewById(R.id.etProductPrice);
etCostPrice = findViewById(R.id.etProductCostPrice);
btnSave = findViewById(R.id.btnSaveProduct);

productViewModel = new ViewModelProvider(this).get(ProductViewModel.class);
```

```
int productId = getIntent().getIntExtra(EXTRA_PRODUCT_ID, -1);
if (productId != -1) {
productViewModel.getById(productId).observe(this, product -> {
if (product != null && existingProduct == null) {
existingProduct = product;
populateFields(product);
}
});
}

btnSave.setOnClickListener(v -> saveProduct());
}

private void populateFields(Product product) {
etName.setText(product.getName());
etCategory.setText(product.getCategory());
etSize.setText(product.getSize());
etColor.setText(product.getColor());
etQuantity.setText(String.valueOf(product.getQuantity()));
etPrice.setText(String.valueOf(product.getPrice()));
etCostPrice.setText(String.valueOf(product.getCostPrice()));
}

private void saveProduct() {
String name = etName.getText().toString().trim();
String category = etCategory.getText().toString().trim();
String size = etSize.getText().toString().trim();
String color = etColor.getText().toString().trim();
String qtyStr = etQuantity.getText().toString().trim();
String priceStr = etPrice.getText().toString().trim();
String costStr = etCostPrice.getText().toString().trim();

if (name.isEmpty() || qtyStr.isEmpty() || priceStr.isEmpty()) {
Toast.makeText(this, getString(R.string.error_fill_all_fields), Toast.LENGTH_SHORT).show();
return;
}

int qty = Integer.parseInt(qtyStr);
double price = Double.parseDouble(priceStr);
```

```
double cost = costStr.isEmpty() ? 0 : Double.parseDouble(costStr);

if (existingProduct != null) {
existingProduct.name = name;
existingProduct.category = category;
existingProduct.size = size;
existingProduct.color = color;
existingProduct.quantity = qty;
existingProduct.price = price;
existingProduct.costPrice = cost;
productViewModel.update(existingProduct);
} else {
Product product = new Product(name, category, size, color, qty, price, cost, "",
System.currentTimeMillis());
productViewModel.insert(product);
}
finish();
}
}
```

customer/CustomerHomeFragment.java

```
package com.storepilot.customer;

import android.os.Bundle;
import android.text.Editable;
import android.text.TextWatcher;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.EditText;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.lifecycle.ViewModelProvider;
import androidx.recyclerview.widget.GridLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.core.SessionManager;
import com.storepilot.customer.adapters.CustomerProductAdapter;
import com.storepilot.db.entities.Product;
import com.storepilot.viewmodels.CartViewModel;
import com.storepilot.viewmodels.ProductViewModel;

import java.util.ArrayList;
import java.util.List;

// Customer home screen showing all products in a searchable grid
public class CustomerHomeFragment extends Fragment {

private RecyclerView rvProducts;
private EditText etSearch;
private TextView tvGreeting, tvEmpty;
private ProductViewModel productViewModel;
private CartViewModel cartViewModel;
private CustomerProductAdapter adapter;

// Full list of products for filtering
private List<Product> allProducts = new ArrayList<>();
```

```
@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater,
@Nullable ViewGroup container,
@Nullable Bundle savedInstanceState) {
return inflater.inflate(R.layout.fragment_customer_home, container, false);
}

@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
super.onViewCreated(view, savedInstanceState);

// Bind views
rvProducts = view.findViewById(R.id.rvProducts);
etSearch = view.findViewById(R.id.etSearch);
tvGreeting = view.findViewById(R.id.tvGreeting);
tvEmpty = view.findViewById(R.id.tvEmpty);

// Show personalized greeting
String name = SessionManager.getInstance().getLoggedInUser().getFullName();
tvGreeting.setText("Hello, " + name + " ■");

// Set up product grid (2 columns)
rvProducts.setLayoutManager(new GridLayoutManager(requireContext(), 2));

// Get ViewModels
productViewModel = new ViewModelProvider(this).get(ProductViewModel.class);
cartViewModel = new ViewModelProvider(requireActivity()).get(CartViewModel.class);

int customerId = SessionManager.getInstance().getLoggedInUser().getId();

// Create adapter — handles product card clicks and add-to-cart
adapter = new CustomerProductAdapter(new ArrayList<>(), product -> {
// Navigate to product details screen
ProductDetailFragment detail = new ProductDetailFragment();
Bundle args = new Bundle();
args.putInt("productId", product.getId());
detail.setArguments(args);
requireActivity().getSupportFragmentManager()
```

```
.beginTransaction()
.replace(R.id.customerFragmentContainer, detail)
.addToBackStack(null)
.commit();
}, product -> {
// Add to cart button tapped
cartViewModel.addToCart(customerId, product.getId());
});

rvProducts.setAdapter(adapter);

// Load products and update the grid
productViewModel.getAllProducts().observe(getViewLifecycleOwner(), products -> {
allProducts = products != null ? products : new ArrayList<>();
filterProducts(etSearch.getText().toString());
});

// Live search as user types
etSearch.addTextChangedListener(new TextWatcher() {
@Override public void beforeTextChanged(CharSequence s, int start, int count, int after) {}
@Override public void onTextChanged(CharSequence s, int start, int before, int count) {
filterProducts(s.toString());
}
@Override public void afterTextChanged(Editable s) {}
});
}

// Filter product list by name or category matching the search text
private void filterProducts(String query) {
List<Product> filtered = new ArrayList<>();
String q = query.toLowerCase().trim();
for (Product p : allProducts) {
if (q.isEmpty() || p.getName().toLowerCase().contains(q)
|| p.getCategory().toLowerCase().contains(q)) {
filtered.add(p);
}
}
adapter.setProducts(filtered);
// Show empty state if no results
tvEmpty.setVisibility(filtered.isEmpty() ? View.VISIBLE : View.GONE);
```

```
rvProducts.setVisibility(filtered.isEmpty() ? View.GONE : View.VISIBLE);
}
}
```

customer/ProductDetailFragment.java

```
package com.storepilot.customer;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.ImageView;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.lifecycle.ViewModelProvider;

import com.storepilot.R;
import com.storepilot.core.AppDatabase;
import com.storepilot.core.SessionManager;
import com.storepilot.db.entities.Product;
import com.storepilot.viewmodels.CartViewModel;
import com.storepilot.viewmodels.FavoritesViewModel;

// Shows full details for a single product
public class ProductDetailFragment extends Fragment {

private int productId;
private CartViewModel cartViewModel;
private FavoritesViewModel favoritesViewModel;
private Button btnFavorite;
private boolean isFavorited = false;

@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater,
@Nullable ViewGroup container,
@Nullable Bundle savedInstanceState) {
return inflater.inflate(R.layout.fragment_product_detail, container, false);
}
```

```
@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
super.onViewCreated(view, savedInstanceState);

// Read product ID passed from the product list
productId = getArguments() != null ? getArguments().getInt("productId", -1) : -1;

TextView tvName = view.findViewById(R.id.tvProductName);
TextView tvCategory = view.findViewById(R.id.tvCategory);
TextView tvPrice = view.findViewById(R.id.tvPrice);
TextView tvDescription = view.findViewById(R.id.tvDescription);
TextView tvStock = view.findViewById(R.id.tvStock);
Button btnAddToCart = view.findViewById(R.id.btnAddToCart);
btnFavorite = view.findViewById(R.id.btnFavorite);
ImageView ivBack = view.findViewById(R.id.ivBack);

cartViewModel = new ViewModelProvider(requireActivity()).get(CartViewModel.class);
favoritesViewModel = new ViewModelProvider(this).get(FavoritesViewModel.class);

int customerId = SessionManager.getInstance().getLoggedInUser().getId();

// Load product from the database in background
AppDatabase.dbExecutor.execute(() -> {
Product product = AppDatabase.getInstance(requireContext())
.productDao().findById(productId);
if (product == null) return;

// Update UI on the main thread
requireActivity().runOnUiThread(() -> {
tvName.setText(product.getName());
tvCategory.setText(product.getCategory() + " • " + product.getColor() + " • " + product.getSize());
tvPrice.setText(String.format("$%.2f", product.getPrice()));
tvDescription.setText("A quality " + product.getName().toLowerCase() +
" from our " + product.getCategory() + " collection.");
tvStock.setText(product.getQuantity() > 0
? "In Stock (" + product.getQuantity() + " available)"
: "Out of Stock");

// Disable cart button if out of stock
btnAddToCart.setEnabled(product.getQuantity() > 0);
```

```
});

// Check if product is already in favorites
AppDatabase db = AppDatabase.getInstance(requireContext());
isFavorited = db.favoriteDao().getFavorite(customerId, productId) != null;
requireActivity().runOnUiThread(() ->
btnFavorite.setText(isFavorited ? "♥ Saved" : "■ Save"));
});

// Add product to cart
btnAddToCart.setOnClickListener(v -> {
cartViewModel.addToCart(customerId, productId);
Toast.makeText(requireContext(), "Added to cart!", Toast.LENGTH_SHORT).show();
});

// Toggle favorite
btnFavorite.setOnClickListener(v -> {
favoritesViewModel.toggleFavorite(customerId, productId);
isFavorited = !isFavorited;
btnFavorite.setText(isFavorited ? "♥ Saved" : "■ Save");
Toast.makeText(requireContext(),
isFavorited ? "Saved to wishlist" : "Removed from wishlist",
Toast.LENGTH_SHORT).show();
});

// Go back to previous screen
ivBack.setOnClickListener(v -> requireActivity().onBackPressed());
}
}
```

customer/CartFragment.java

```
package com.storepilot.customer;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.lifecycle.ViewModelProvider;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.core.AppDatabase;
import com.storepilot.core.SessionManager;
import com.storepilot.customer.adapters.CartItemAdapter;
import com.storepilot.db.entities.CartItem;
import com.storepilot.db.entities.Product;
import com.storepilot.viewmodels.CartViewModel;
import com.storepilot.viewmodels.ProductViewModel;

import java.util.ArrayList;
import java.util.List;

// Shopping cart screen — shows items the customer wants to buy
public class CartFragment extends Fragment {

private RecyclerView rvCart;
private TextView tvTotal, tvEmpty;
private Button btnCheckout;
private CartViewModel cartViewModel;
private ProductViewModel productViewModel;
private CartItemAdapter adapter;

// Current list of cart items (needed for checkout)
private List<CartItem> currentCartItems = new ArrayList<>();
```

```
// Product details matched to cart items
private List<Product> currentProducts = new ArrayList<>();

@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater,
@Nullable ViewGroup container,
@Nullable Bundle savedInstanceState) {
return inflater.inflate(R.layout.fragment_cart, container, false);
}

@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
super.onViewCreated(view, savedInstanceState);

rvCart = view.findViewById(R.id.rvCart);
tvTotal = view.findViewById(R.id.tvCartTotal);
tvEmpty = view.findViewById(R.id.tvCartEmpty);
btnCheckout = view.findViewById(R.id.btnCheckout);

rvCart.setLayoutManager(new LinearLayoutManager(requireContext()));
cartViewModel = new ViewModelProvider(requireActivity()).get(CartViewModel.class);
productViewModel = new ViewModelProvider(this).get(ProductViewModel.class);

int customerId = SessionManager.getInstance().getLoggedInUser().getId();

// Adapter handles quantity buttons and item removal
adapter = new CartItemAdapter(new ArrayList<>(), new ArrayList<>(),
cartItem -> cartViewModel.addToCart(customerId, cartItem.getProductId()),
cartItem -> cartViewModel.removeOne(cartItem),
cartItem -> cartViewModel.removeFromCart(cartItem.getId())
);
rvCart.setAdapter(adapter);

// Load all products once (needed to show names and prices)
productViewModel.getAllProducts().observe(getViewLifecycleOwner(), products -> {
currentProducts = products != null ? products : new ArrayList<>();
refreshCart();
});
```

```
// Observe cart items and update UI
cartViewModel.getCartItems(customerId).observe(getViewLifecycleOwner(), cartItems -> {
currentCartItems = cartItems != null ? cartItems : new ArrayList<>();
refreshCart();
});

// Go to checkout screen
btnCheckout.setOnClickListener(v -> {
if (currentCartItems.isEmpty()) return;
CheckoutFragment checkout = new CheckoutFragment();
requireActivity().getSupportFragmentManager()
.beginTransaction()
.replace(R.id.customerFragmentContainer, checkout)
.addToBackStack(null)
.commit();
});
}

// Update adapter and recalculate total price
private void refreshCart() {
adapter.update(currentCartItems, currentProducts);

// Calculate total by matching cart items to product prices
double total = 0;
for (CartItem ci : currentCartItems) {
for (Product p : currentProducts) {
if (p.getId() == ci.getProductId()) {
total += p.getPrice() * ci.getQuantity();
break;
}
}
}
tvTotal.setText(String.format("Total: $%.2f", total));

// Show empty state if cart is empty
boolean isEmpty = currentCartItems.isEmpty();
tvEmpty.setVisibility(isEmpty ? View.VISIBLE : View.GONE);
rvCart.setVisibility(isEmpty ? View.GONE : View.VISIBLE);
btnCheckout.setEnabled(!isEmpty);
}
```

```
}
```

customer/CheckoutFragment.java

```
package com.storepilot.customer;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.EditText;
import android.widget.RadioGroup;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.lifecycle.ViewModelProvider;

import com.storepilot.R;
import com.storepilot.core.SessionManager;
import com.storepilot.db.entities.CartItem;
import com.storepilot.db.entities.Product;
import com.storepilot.viewmodels.CartViewModel;
import com.storepilot.viewmodels.OrderViewModel;
import com.storepilot.viewmodels.ProductViewModel;

import java.util.ArrayList;
import java.util.List;

// Checkout screen — collects shipping address and payment method, then places the order
public class CheckoutFragment extends Fragment {

private EditText etAddress;
private RadioGroup rgPayment;
private Button btnPlaceOrder;
private CartViewModel cartViewModel;
private OrderViewModel orderViewModel;
private ProductViewModel productViewModel;

// Cached LiveData values used when the button is tapped
private List<CartItem> currentCartItems = new ArrayList<>();
private List<Product> currentProducts = new ArrayList<>();
```

```
@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater,
@Nullable ViewGroup container,
@Nullable Bundle savedInstanceState) {
return inflater.inflate(R.layout.fragment_checkout, container, false);
}

@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
super.onViewCreated(view, savedInstanceState);

etAddress = view.findViewById(R.id.etShippingAddress);
rgPayment = view.findViewById(R.id.rgPaymentMethod);
btnPlaceOrder = view.findViewById(R.id.btnPlaceOrder);

cartViewModel = new ViewModelProvider(requireActivity()).get(CartViewModel.class);
orderViewModel = new ViewModelProvider(this).get(OrderViewModel.class);
productViewModel = new ViewModelProvider(this).get(ProductViewModel.class);

int customerId = SessionManager.getInstance().getLoggedInUser().getId();

// Cache cart items so the click handler can read them without re-observing
cartViewModel.getCartItems(customerId).observe(getViewLifecycleOwner(), cartItems ->
currentCartItems = cartItems != null ? cartItems : new ArrayList<>());

// Cache product list for price lookup
productViewModel.getAllProducts().observe(getViewLifecycleOwner(), products ->
currentProducts = products != null ? products : new ArrayList<>());

// Navigate to confirmation once the order is placed successfully
orderViewModel.orderPlaced.observe(getViewLifecycleOwner(), placed -> {
if (Boolean.TRUE.equals(placed)) {
requireActivity().getSupportFragmentManager()
.beginTransaction()
.replace(R.id.customerFragmentContainer, new OrderConfirmationFragment())
.commit();
}
});
```

```
btnPlaceOrder.setOnClickListener(v -> {
String address = etAddress.getText().toString().trim();
if (address.isEmpty()) {
Toast.makeText(requireContext(), "Please enter your shipping address.", Toast.LENGTH_SHORT).show();
return;
}
if (currentCartItems.isEmpty()) {
Toast.makeText(requireContext(), "Your cart is empty.", Toast.LENGTH_SHORT).show();
return;
}

// Map radio selection to payment method string
int selectedId = rgPayment.getCheckedRadioButtonId();
String paymentMethod;
if (selectedId == R.id.rbCash) {
paymentMethod = "CASH_ON_DELIVERY";
} else if (selectedId == R.id.rbPaypal) {
paymentMethod = "PAYPAL";
} else {
paymentMethod = "CREDIT_CARD";
}

btnPlaceOrder.setEnabled(false);
orderViewModel.placeOrder(customerId, paymentMethod, address,
currentCartItems, currentProducts);
});
}
}
```

customer/OrderHistoryFragment.java

```
package com.storepilot.customer;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.lifecycle.ViewModelProvider;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.core.SessionManager;
import com.storepilot.customer.adapters.CustomerOrderAdapter;
import com.storepilot.viewmodels.OrderViewModel;

import java.util.ArrayList;

// Shows the customer's past and current orders
public class OrderHistoryFragment extends Fragment {

private RecyclerView rvOrders;
private TextView tvEmpty;
private OrderViewModel orderViewModel;
private CustomerOrderAdapter adapter;

@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater,
@Nullable ViewGroup container,
@Nullable Bundle savedInstanceState) {
return inflater.inflate(R.layout.fragment_order_history, container, false);
}

@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
```

```
super.onViewCreated(view, savedInstanceState);

rvOrders = view.findViewById(R.id.rvOrders);
tvEmpty = view.findViewById(R.id.tvEmpty);

rvOrders.setLayoutManager(new LinearLayoutManager(requireContext()));
adapter = new CustomerOrderAdapter(new ArrayList<>());
rvOrders.setAdapter(adapter);

orderViewModel = new ViewModelProvider(this).get(OrderViewModel.class);

int customerId = SessionManager.getInstance().getLoggedInUser().getId();

// Load and display orders for this customer
orderViewModel.getOrdersByCustomer(customerId).observe(getViewLifecycleOwner(), orders -> {
adapter.setOrders(orders != null ? orders : new ArrayList<>());
boolean empty = orders == null || orders.isEmpty();
tvEmpty.setVisibility(empty ? View.VISIBLE : View.GONE);
rvOrders.setVisibility(empty ? View.GONE : View.VISIBLE);
});
}
}
```

customer/OrderConfirmationFragment.java

```
package com.storepilot.customer;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;

import com.storepilot.R;

// Order confirmation screen shown after a successful checkout
public class OrderConfirmationFragment extends Fragment {

@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater,
@Nullable ViewGroup container,
@Nullable Bundle savedInstanceState) {
return inflater.inflate(R.layout.fragment_order_confirmation, container, false);
}

@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
super.onViewCreated(view, savedInstanceState);

Button btnViewOrders = view.findViewById(R.id.btnViewOrders);
Button btnContinueShopping = view.findViewById(R.id.btnContinueShopping);

// Navigate to order history
btnViewOrders.setOnClickListener(v -> {
requireActivity().getSupportFragmentManager()
.beginTransaction()
.replace(R.id.customerFragmentContainer, new OrderHistoryFragment())
.commit();
});
```

```
// Go back to shopping
btnContinueShopping.setOnClickListener(v -> {
requireActivity().getSupportFragmentManager()
.beginTransaction()
.replace(R.id.customerFragmentContainer, new CustomerHomeFragment())
.commit();
});
}
}
```

customer/FavoritesFragment.java

```
package com.storepilot.customer;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.lifecycle.ViewModelProvider;
import androidx.recyclerview.widget.GridLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.core.SessionManager;
import com.storepilot.customer.adapters.CustomerProductAdapter;
import com.storepilot.db.entities.Favorite;
import com.storepilot.db.entities.Product;
import com.storepilot.viewmodels.CartViewModel;
import com.storepilot.viewmodels.FavoritesViewModel;
import com.storepilot.viewmodels.ProductViewModel;

import java.util.ArrayList;
import java.util.List;

// Customer's wishlist — shows products the customer has favorited
public class FavoritesFragment extends Fragment {

private RecyclerView rvFavorites;
private TextView tvEmpty;
private FavoritesViewModel favoritesViewModel;
private ProductViewModel productViewModel;
private CartViewModel cartViewModel;
private CustomerProductAdapter adapter;

// Cached latest values from both LiveData streams
private List<Favorite> latestFavorites = new ArrayList<>();
private List<Product> latestProducts = new ArrayList<>();
```

```
@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater,
@Nullable ViewGroup container,
@Nullable Bundle savedInstanceState) {
return inflater.inflate(R.layout.fragment_favorites, container, false);
}

@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
super.onViewCreated(view, savedInstanceState);

rvFavorites = view.findViewById(R.id.rvFavorites);
tvEmpty = view.findViewById(R.id.tvEmpty);

rvFavorites.setLayoutManager(new GridLayoutManager(requireContext(), 2));

favoritesViewModel = new ViewModelProvider(this).get(FavoritesViewModel.class);
productViewModel = new ViewModelProvider(this).get(ProductViewModel.class);
cartViewModel = new ViewModelProvider(requireActivity()).get(CartViewModel.class);

int customerId = SessionManager.getInstance().getLoggedInUser().getId();

adapter = new CustomerProductAdapter(new ArrayList<>(), product -> {
ProductDetailFragment detail = new ProductDetailFragment();
Bundle args = new Bundle();
args.putInt("productId", product.getId());
detail.setArguments(args);
requireActivity().getSupportFragmentManager()
.beginTransaction()
.replace(R.id.customerFragmentContainer, detail)
.addToBackStack(null)
.commit();
}, product -> cartViewModel.addToCart(customerId, product.getId()));

rvFavorites.setAdapter(adapter);

// Observe favorites and products independently, then combine when either changes
favoritesViewModel.getFavorites(customerId).observe(getViewLifecycleOwner(), favorites -> {
```

```
latestFavorites = favorites != null ? favorites : new ArrayList<>();
updateFavoritesDisplay();
});

productViewModel.getAllProducts().observe(getViewLifecycleOwner(), products -> {
latestProducts = products != null ? products : new ArrayList<>();
updateFavoritesDisplay();
});
}

// Build the product list from the intersection of favorites and all products
private void updateFavoritesDisplay() {
List<Product> favProducts = new ArrayList<>();
for (Favorite fav : latestFavorites) {
for (Product p : latestProducts) {
if (p.getId() == fav.getProductId()) {
favProducts.add(p);
break;
}
}
}
adapter.setProducts(favProducts);
boolean empty = favProducts.isEmpty();
tvEmpty.setVisibility(empty ? View.VISIBLE : View.GONE);
rvFavorites.setVisibility(empty ? View.GONE : View.VISIBLE);
}
}
```

80 Adapters

All RecyclerView Adapters across the application

RecyclerView adapters bridge the data layer to the UI using the ViewHolder pattern. Click callbacks are functional interfaces, keeping adapters fully decoupled from their hosting fragments.

customer/adapters/CustomerProductAdapter.java

```
package com.storepilot.customer.adapters;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.db.entities.Product;

import java.util.List;

// Shows products as cards in the customer's home/catalog grid
public class CustomerProductAdapter extends RecyclerView.Adapter<CustomerProductAdapter.ViewHolder> {

// Callback for when a product card is tapped (open detail)
public interface OnProductClick { void onClick(Product product); }
// Callback for when the Add to Cart button is tapped
public interface OnAddToCart { void onAdd(Product product); }

private List<Product> products;
private final OnProductClick clickListener;
private final OnAddToCart addToCartListener;

public CustomerProductAdapter(List<Product> products,
OnProductClick clickListener,
OnAddToCart addToCartListener) {
this.products = products;
this.clickListener = clickListener;
this.addToCartListener = addToCartListener;
}

// Replace the list and refresh the grid
public void setProducts(List<Product> newList) {
this.products = newList;
notifyDataSetChanged();
```

```
}

@NonNull
@Override
public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
View v = LayoutInflater.from(parent.getContext())
.inflate(R.layout.item_customer_product, parent, false);
return new ViewHolder(v);
}

@Override
public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
Product p = products.get(position);

// Fill in product details
holder.tvName.setText(p.getName());
holder.tvCategory.setText(p.getCategory());
holder.tvPrice.setText(String.format("$%.2f", p.getPrice()));

// Show availability status
if (p.getQuantity() > 0) {
holder.tvStock.setText("In Stock");
holder.tvStock.setTextColor(0xFF4CAF50); // green
} else {
holder.tvStock.setText("Out of Stock");
holder.tvStock.setTextColor(0xFFF44336); // red
}

// Tap anywhere on card to view product details
holder.itemView.setOnClickListener(v -> clickListener.onClick(p));

// Add to cart button
holder.btnAddToCart.setOnClickListener(v -> addToCartListener.onAdd(p));
holder.btnAddToCart.setEnabled(p.getQuantity() > 0);
}

@Override
public int getItemCount() { return products.size(); }

static class ViewHolder extends RecyclerView.ViewHolder {
```

```
TextView tvName, tvCategory, tvPrice, tvStock;
Button btnAddToCart;

ViewHolder(View v) {
super(v);
tvName = v.findViewById(R.id.tvProductName);
tvCategory = v.findViewById(R.id.tvProductCategory);
tvPrice = v.findViewById(R.id.tvProductPrice);
tvStock = v.findViewById(R.id.tvProductStock);
btnAddToCart = v.findViewById(R.id.btnAddToCart);
}
}
}
```

customer/adapters/CartItemAdapter.java

```
package com.storepilot.customer.adapters;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ImageButton;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.db.entities.CartItem;
import com.storepilot.db.entities.Product;

import java.util.List;

// Adapter for the cart screen — shows items with quantity controls
public class CartItemAdapter extends RecyclerView.Adapter<CartItemAdapter.ViewHolder> {

public interface CartAction { void on(CartItem item); }

private List<CartItem> cartItems;
private List<Product> products;
private final CartAction onIncrease, onDecrease, onRemove;

public CartItemAdapter(List<CartItem> cartItems, List<Product> products,
CartAction onIncrease, CartAction onDecrease, CartAction onRemove) {
this.cartItems = cartItems;
this.products = products;
this.onIncrease = onIncrease;
this.onDecrease = onDecrease;
this.onRemove = onRemove;
}

// Update both cart items and product details at once
public void update(List<CartItem> cartItems, List<Product> products) {
this.cartItems = cartItems;
this.products = products;
notifyDataSetChanged();
```

```
}

@NonNull
@Override
public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
View v = LayoutInflater.from(parent.getContext())
.inflate(R.layout.item_cart, parent, false);
return new ViewHolder(v);
}

@Override
public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
CartItem ci = cartItems.get(position);

// Find matching product to show name and price
Product product = null;
for (Product p : products) {
if (p.getId() == ci.getProductId()) {
product = p;
break;
}
}

if (product != null) {
holder.tvName.setText(product.getName());
holder.tvUnitPrice.setText(String.format("$%.2f each", product.getPrice()));
holder.tvSubtotal.setText(String.format("$%.2f", product.getPrice() * ci.getQuantity()));
}

// Show quantity
holder.tvQuantity.setText(String.valueOf(ci.getQuantity()));

// Quantity buttons
holder.btnPlus.setOnClickListener(v -> onIncrease.on(ci));
holder.btnMinus.setOnClickListener(v -> onDecrease.on(ci));
holder.btnRemove.setOnClickListener(v -> onRemove.on(ci));
}

@Override
public int getItemCount() { return cartItems.size(); }
```

```

static class ViewHolder extends RecyclerView.ViewHolder {
TextView tvName, tvUnitPrice, tvQuantity, tvSubtotal;
ImageButton btnPlus, btnMinus, btnRemove;

ViewHolder(View v) {
super(v);
tvName = v.findViewById(R.id.tvCartItemName);
tvUnitPrice = v.findViewById(R.id.tvUnitPrice);
tvQuantity = v.findViewById(R.id.tvQuantity);
tvSubtotal = v.findViewById(R.id.tvSubtotal);
btnPlus = v.findViewById(R.id.btnIncrease);
btnMinus = v.findViewById(R.id.btnDecrease);
btnRemove = v.findViewById(R.id.btnRemoveItem);
}
}
}
```

customer/adapters/MessageAdapter.java

```
package com.storepilot.customer.adapters;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.db.entities.SupportMessage;

import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.List;
import java.util.Locale;

// Chat message adapter — shows sent messages on the right, received on the left
public class MessageAdapter extends RecyclerView.Adapter<MessageAdapter.ViewHolder> {

// Two view types: one for sent messages, one for received
private static final int VIEW_SENT = 1;
private static final int VIEW_RECEIVED = 2;

private List<SupportMessage> messages;
private final int currentUserId;
private final SimpleDateFormat timeFormat = new SimpleDateFormat("h:mm a", Locale.getDefault());

public MessageAdapter(List<SupportMessage> messages, int currentUserId) {
this.messages = messages;
this.currentUserId = currentUserId;
}

public void setMessages(List<SupportMessage> messages) {
this.messages = messages;
notifyDataSetChanged();
}

// Determine which layout to use based on who sent the message
```

```
@Override
public int getItemViewType(int position) {
return messages.get(position).getSenderId() == currentUserId
? VIEW_SENT : VIEW_RECEIVED;
}

@NonNull
@Override
public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
int layout = viewType == VIEW_SENT
? R.layout.item_message_sent
: R.layout.item_message_received;
View v = LayoutInflater.from(parent.getContext()).inflate(layout, parent, false);
return new ViewHolder(v);
}

@Override
public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
SupportMessage msg = messages.get(position);

// Set the message text
holder.tvMessageText.setText(msg.getMessageText());

// Show formatted time
holder.tvTimestamp.setText(timeFormat.format(new Date(msg.getTimestamp())));

// Show sender name on received messages
if (holder.tvSenderName != null) {
// Manager/Owner messages show the role as sender label
holder.tvSenderName.setText("Support Team");
}
}

@Override
public int getItemCount() { return messages.size(); }

static class ViewHolder extends RecyclerView.ViewHolder {
TextView tvMessageText, tvTimestamp, tvSenderName;

ViewHolder(View v) {
```

```
super(v);
tvMessageText = v.findViewById(R.id.tvMessageText);
tvTimestamp = v.findViewById(R.id.tvTimestamp);
tvSenderName = v.findViewById(R.id.tvSenderName); // may be null for sent layout
}
}
}
```

customer/adapters/CustomerOrderAdapter.java

```
package com.storepilot.customer.adapters;

import android.graphics.Color;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.db.entities.Order;

import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.List;
import java.util.Locale;

// Shows a list of customer orders with their status
public class CustomerOrderAdapter extends RecyclerView.Adapter<CustomerOrderAdapter.ViewHolder> {

private List<Order> orders;
// Formatter for order dates
private final SimpleDateFormat sdf = new SimpleDateFormat("MMM d, yyyy", Locale.getDefault());

public CustomerOrderAdapter(List<Order> orders) {
this.orders = orders;
}

public void setOrders(List<Order> orders) {
this.orders = orders;
notifyDataSetChanged();
}

@NonNull
@Override
public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
View v = LayoutInflater.from(parent.getContext())
.inflate(R.layout.item_customer_order, parent, false);
```

```
return new ViewHolder(v);
}

@Override
public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
Order order = orders.get(position);

// Show order ID and date
holder.tvOrderId.setText("Order #" + order.getId());
holder.tvDate.setText(sdf.format(new Date(order.getCreatedAt())));
holder.tvTotal.setText(String.format("$%.2f", order.getTotalPrice()));
holder.tvPayment.setText(order.getPaymentMethod().replace("_", " "));
holder.tvStatus.setText(order.getStatus());

// Color-code the status badge
switch (order.getStatus()) {
case "PENDING": holder.tvStatus.setTextColor(Color.parseColor("#FF9800")); break; // orange
case "PROCESSING": holder.tvStatus.setTextColor(Color.parseColor("#2196F3")); break; // blue
case "SHIPPED": holder.tvStatus.setTextColor(Color.parseColor("#9C27B0")); break; // purple
case "DELIVERED": holder.tvStatus.setTextColor(Color.parseColor("#4CAF50")); break; // green
case "CANCELLED": holder.tvStatus.setTextColor(Color.parseColor("#F44336")); break; // red
}
}

@Override
public int getItemCount() { return orders.size(); }

static class ViewHolder extends RecyclerView.ViewHolder {
TextView tvOrderId, tvDate, tvTotal, tvPayment, tvStatus;

ViewHolder(View v) {
super(v);
tvOrderId = v.findViewById(R.id.tvOrderId);
tvDate = v.findViewById(R.id.tvOrderDate);
tvTotal = v.findViewById(R.id.tvOrderTotal);
tvPayment = v.findViewById(R.id.tvPaymentMethod);
tvStatus = v.findViewById(R.id.tvOrderStatus);
}
}
}
```

manager/adapters/ManagerOrderAdapter.java

```
package com.storepilot.manager.adapters;

import android.graphics.Color;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ArrayAdapter;
import android.widget.Spinner;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.db.entities.Order;

import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.List;
import java.util.Locale;

// Manager order list — shows all orders with status update dropdowns
public class ManagerOrderAdapter extends RecyclerView.Adapter<ManagerOrderAdapter.ViewHolder> {

public interface OnStatusChange { void onChange(Order order, String newStatus); }

private List<Order> orders;
private final OnStatusChange statusChangeListener;
private final SimpleDateFormat sdf = new SimpleDateFormat("MMM d, yyyy HH:mm", Locale.getDefault());

// Available order statuses the manager can set
private static final String[] STATUSES = {"PENDING", "PROCESSING", "SHIPPED", "DELIVERED",
"CANCELLED"};

public ManagerOrderAdapter(List<Order> orders, OnStatusChange listener) {
this.orders = orders;
this.statusChangeListener = listener;
}

public void setOrders(List<Order> orders) {
this.orders = orders;
```

```
notifyDataSetChanged();
}

@NonNull
@Override
public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
View v = LayoutInflater.from(parent.getContext())
.inflate(R.layout.item_manager_order, parent, false);
return new ViewHolder(v);
}

@Override
public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
Order order = orders.get(position);

holder.tvOrderId.setText("Order #" + order.getId());
holder.tvCustomerId.setText("Customer #" + order.getCustomerId());
holder.tvDate.setText(sdf.format(new Date(order.getCreatedAt())));
holder.tvTotal.setText(String.format("$%.2f", order.getTotalPrice()));
holder.tvPayment.setText(order.getPaymentMethod().replace("_", " "));

// Show the current status with a color
holder.tvCurrentStatus.setText(order.getStatus());
colorStatus(holder.tvCurrentStatus, order.getStatus());

// Set up the status spinner to the current status
ArrayAdapter<String> spinnerAdapter = new ArrayAdapter<>(
holder.itemView.getContext(),
android.R.layout.simple_spinner_item,
STATUSES);
spinnerAdapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
holder.spinnerStatus.setAdapter(spinnerAdapter);

// Pre-select the current status in the spinner
for (int i = 0; i < STATUSES.length; i++) {
if (STATUSES[i].equals(order.getStatus())) {
holder.spinnerStatus.setSelection(i);
break;
}
}
```

```
// When manager changes the spinner, update the order status
holder.spinnerStatus.setOnItemSelectedListener(new
android.widget.AdapterView.OnItemSelectedListener() {
boolean firstTime = true;
@Override
public void onItemSelected(android.widget.AdapterView<?> parent, View view, int pos, long id) {
if (firstTime) { firstTime = false; return; } // skip initial selection
String newStatus = STATUSES[pos];
if (!newStatus.equals(order.getStatus())) {
statusChangeListener.onChange(order, newStatus);
}
}
@Override public void onNothingSelected(android.widget.AdapterView<?> parent) {}
});
}

// Apply color based on status string
private void colorStatus(TextView tv, String status) {
switch (status) {
case "PENDING": tv.setTextColor(Color.parseColor("#FF9800")); break;
case "PROCESSING": tv.setTextColor(Color.parseColor("#2196F3")); break;
case "SHIPPED": tv.setTextColor(Color.parseColor("#9C27B0")); break;
case "DELIVERED": tv.setTextColor(Color.parseColor("#4CAF50")); break;
case "CANCELLED": tv.setTextColor(Color.parseColor("#F44336")); break;
}
}

@Override
public int getItemCount() { return orders.size(); }

static class ViewHolder extends RecyclerView.ViewHolder {
TextView tvOrderId, tvCustomerId, tvDate, tvTotal, tvPayment, tvCurrentStatus;
Spinner spinnerStatus;

ViewHolder(View v) {
super(v);
tvOrderId = v.findViewById(R.id.tvManagerOrderId);
tvCustomerId = v.findViewById(R.id.tvManagerCustomerId);
tvDate = v.findViewById(R.id.tvManagerOrderDate);
tvTotal = v.findViewById(R.id.tvManagerOrderTotal);
```

```
tvPayment = v.findViewById(R.id.tvManagerPayment);
tvCurrentStatus = v.findViewById(R.id.tvManagerCurrentStatus);
spinnerStatus = v.findViewById(R.id.spinnerOrderStatus);
}
}
}
```

inventory/ProductListAdapter.java

```
package com.storepilot.inventory;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.db.entities.Product;

import java.util.ArrayList;
import java.util.List;

public class ProductListAdapter extends RecyclerView.Adapter<ProductListAdapter.ViewHolder> {

public interface OnProductClickListener {
void onProductClick(Product product);
}

private List<Product> products = new ArrayList<>();
private final OnProductClickListener listener;

public ProductListAdapter(OnProductClickListener listener) {
this.listener = listener;
}

public void setProducts(List<Product> products) {
this.products = products != null ? products : new ArrayList<>();
notifyDataSetChanged();
}

@NonNull
@Override
public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
View view = LayoutInflater.from(parent.getContext())
.inflate(R.layout.item_product, parent, false);
return new ViewHolder(view);
```

```
}

@Override
public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
Product product = products.get(position);
holder.tvName.setText(product.getName());
holder.tvCategory.setText(product.getCategory());
holder.tvQuantity.setText(String.valueOf(product.getQuantity()));
holder.tvPrice.setText(String.format("$%.2f", product.getPrice()));
holder.itemView.setOnClickListener(v -> listener.onProductClick(product));
}

@Override
public int getItemCount() {
return products.size();
}

static class ViewHolder extends RecyclerView.ViewHolder {
TextView tvName, tvCategory, tvQuantity, tvPrice;

ViewHolder(View itemView) {
super(itemView);
tvName = itemView.findViewById(R.id.tvProductName);
tvCategory = itemView.findViewById(R.id.tvProductCategory);
tvQuantity = itemView.findViewById(R.id.tvProductQuantity);
tvPrice = itemView.findViewById(R.id.tvProductPrice);
}
}
}
```

tasks/TaskAdapter.java

```
package com.storepilot.tasks;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.db.entities.Task;

import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.Locale;

public class TaskAdapter extends RecyclerView.Adapter<TaskAdapter.ViewHolder> {

private List<Task> tasks = new ArrayList<>();

public void setTasks(List<Task> tasks) {
this.tasks = tasks != null ? tasks : new ArrayList<>();
notifyDataSetChanged();
}

@NonNull
@Override
public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
View view = LayoutInflater.from(parent.getContext())
.inflate(R.layout.item_task, parent, false);
return new ViewHolder(view);
}

@Override
public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
Task t = tasks.get(position);
holder.tvTaskTitle.setText(t.getTitle());
```

```
holder.tvTaskStatus.setText(t.getStatus());
holder.tvTaskPriority.setText(t.getPriority());
holder.tvTaskDue.setText(t.getDueDate() > 0
? new SimpleDateFormat("MMM dd, yyyy", Locale.getDefault()).format(new Date(t.getDueDate()))
: "No due date");
}

@Override
public int getItemCount() {
return tasks.size();
}

static class ViewHolder extends RecyclerView.ViewHolder {
TextView tvTaskTitle, tvTaskStatus, tvTaskPriority, tvTaskDue;

ViewHolder(View itemView) {
super(itemView);
tvTaskTitle = itemView.findViewById(R.id.tvTaskTitle);
tvTaskStatus = itemView.findViewById(R.id.tvTaskStatus);
tvTaskPriority = itemView.findViewById(R.id.tvTaskPriority);
tvTaskDue = itemView.findViewById(R.id.tvTaskDue);
}
}
}
```

seasons/SeasonAdapter.java

```
package com.storepilot.seasons;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.db.entities.Season;

import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.Locale;

public class SeasonAdapter extends RecyclerView.Adapter<SeasonAdapter.ViewHolder> {

private List<Season> seasons = new ArrayList<>();

public void setSeasons(List<Season> seasons) {
this.seasons = seasons != null ? seasons : new ArrayList<>();
notifyDataSetChanged();
}

@NonNull
@Override
public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
View view = LayoutInflater.from(parent.getContext())
.inflate(R.layout.item_season, parent, false);
return new ViewHolder(view);
}

@Override
public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
Season s = seasons.get(position);
SimpleDateFormat sdf = new SimpleDateFormat("MMM dd, yyyy", Locale.getDefault());
```

```
holder.tvSeasonName.setText(s.getName());
holder.tvSeasonDates.setText(sdf.format(new Date(s.getStartDate()))
+ " - " + sdf.format(new Date(s.getEndDate())));
holder.tvSeasonActive.setText(s.isActive() ? "Active" : "Inactive");
}

@Override
public int getItemCount() {
return seasons.size();
}

static class ViewHolder extends RecyclerView.ViewHolder {
TextView tvSeasonName, tvSeasonDates, tvSeasonActive;

ViewHolder(View itemView) {
super(itemView);
tvSeasonName = itemView.findViewById(R.id.tvSeasonName);
tvSeasonDates = itemView.findViewById(R.id.tvSeasonDates);
tvSeasonActive = itemView.findViewById(R.id.tvSeasonActive);
}
}
}
```

sales/SaleAdapter.java

```
package com.storepilot.sales;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.db.entities.Sale;

import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.Locale;

public class SaleAdapter extends RecyclerView.Adapter<SaleAdapter.ViewHolder> {

private List<Sale> sales = new ArrayList<>();

public void setSales(List<Sale> sales) {
this.sales = sales != null ? sales : new ArrayList<>();
notifyDataSetChanged();
}

@NonNull
@Override
public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
View view = LayoutInflater.from(parent.getContext())
.inflate(R.layout.item_sale, parent, false);
return new ViewHolder(view);
}

@Override
public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
Sale sale = sales.get(position);
holder.tvSaleDate.setText(new SimpleDateFormat("MMM dd, yyyy", Locale.getDefault())
```

```
.format(new Date(sale.getSaleDate())));
holder.tvSaleTotal.setText(String.format("$%.2f", sale.getTotalPrice()));
holder.tvSaleQty.setText("Qty: " + sale.getQuantity());
holder.tvSaleNotes.setText(sale.getNotes() != null ? sale.getNotes() : "");
}

@Override
public int getItemCount() {
return sales.size();
}

static class ViewHolder extends RecyclerView.ViewHolder {
TextView tvSaleDate, tvSaleTotal, tvSaleQty, tvSaleNotes;

ViewHolder(View itemView) {
super(itemView);
tvSaleDate = itemView.findViewById(R.id.tvSaleDate);
tvSaleTotal = itemView.findViewById(R.id.tvSaleTotal);
tvSaleQty = itemView.findViewById(R.id.tvSaleQty);
tvSaleNotes = itemView.findViewById(R.id.tvSaleNotes);
}
}
}
```

purchases/PurchaseAdapter.java

```
package com.storepilot.purchases;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.db.entities.Purchase;

import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.Locale;

public class PurchaseAdapter extends RecyclerView.Adapter<PurchaseAdapter.ViewHolder> {

private List<Purchase> purchases = new ArrayList<>();

public void setPurchases(List<Purchase> purchases) {
this.purchases = purchases != null ? purchases : new ArrayList<>();
notifyDataSetChanged();
}

@NonNull
@Override
public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
View view = LayoutInflater.from(parent.getContext())
.inflate(R.layout.item_purchase, parent, false);
return new ViewHolder(view);
}

@Override
public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
Purchase p = purchases.get(position);
holder.tvPurchaseDate.setText(new SimpleDateFormat("MMM dd, yyyy", Locale.getDefault())
```

```
.format(new Date(p.getPurchaseDate())));
holder.tvPurchaseTotal.setText(String.format("$%.2f", p.getTotalCost()));
holder.tvPurchaseSupplier.setText(p.getSupplier() != null ? p.getSupplier() : "");
holder.tvPurchaseQty.setText("Qty: " + p.getQuantity());
}

@Override
public int getItemCount() {
return purchases.size();
}

static class ViewHolder extends RecyclerView.ViewHolder {
TextView tvPurchaseDate, tvPurchaseTotal, tvPurchaseSupplier, tvPurchaseQty;

ViewHolder(View itemView) {
super(itemView);
tvPurchaseDate = itemView.findViewById(R.id.tvPurchaseDate);
tvPurchaseTotal = itemView.findViewById(R.id.tvPurchaseTotal);
tvPurchaseSupplier = itemView.findViewById(R.id.tvPurchaseSupplier);
tvPurchaseQty = itemView.findViewById(R.id.tvPurchaseQty);
}
}
}
```

manager/ConversationListAdapter.java

```
package com.storepilot.manager;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.db.entities.User;

import java.util.List;

// Shows each customer conversation as a row in the support inbox list
public class ConversationListAdapter extends RecyclerView.Adapter<ConversationListAdapter.ViewHolder>
{

public interface OnConversationClick { void onClick(User customer); }

private final List<User> customers;
private final OnConversationClick listener;

public ConversationListAdapter(List<User> customers, OnConversationClick listener) {
this.customers = customers;
this.listener = listener;
}

@NonNull
@Override
public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
View v = LayoutInflater.from(parent.getContext())
.inflate(R.layout.item_conversation, parent, false);
return new ViewHolder(v);
}

@Override
public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
User customer = customers.get(position);
// Show customer name and a placeholder for last message preview
```

```
holder.tvName.setText(customer.getFullName());
holder.tvPreview.setText("Tap to view conversation");
// Open the conversation on tap
holder.itemView.setOnClickListener(v -> listener.onClick(customer));
}

@Override
public int getItemCount() { return customers.size(); }

static class ViewHolder extends RecyclerView.ViewHolder {
TextView tvName, tvPreview;

ViewHolder(View v) {
super(v);
tvName = v.findViewById(R.id.tvCustomerName);
tvPreview = v.findViewById(R.id.tvMessagePreview);
}
}
}
```

admin/UserAdapter.java

```
package com.storepilot.admin;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.db.entities.User;

import java.util.ArrayList;
import java.util.List;

public class UserAdapter extends RecyclerView.Adapter<UserAdapter.ViewHolder> {

private List<User> users = new ArrayList<>();

public void setUsers(List<User> users) {
this.users = users != null ? users : new ArrayList<>();
notifyDataSetChanged();
}

@NonNull
@Override
public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
View view = LayoutInflater.from(parent.getContext())
.inflate(R.layout.item_user, parent, false);
return new ViewHolder(view);
}

@Override
public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
User user = users.get(position);
holder.tvUsername.setText(user.getUsername());
holder.tvUserRole.setText(user.getRole());
}
```

```
@Override
public int getItemCount() {
return users.size();
}

static class ViewHolder extends RecyclerView.ViewHolder {
TextView tvUsername, tvUserRole;

ViewHolder(View itemView) {
super(itemView);
tvUsername = itemView.findViewById(R.id.tvUsername);
tvUserRole = itemView.findViewById(R.id.tvUserRole);
}
}
}
```

100 Tasks & Support

tasks/ + manager/ — DAOs, Repos, ViewModels, Fragments

Task management system and two-way support chat between customers and staff. Both features use the MVVM pipeline. The support chat uses two XML item layouts to render sent and received messages differently.

db/dao/TaskDao.java

```
package com.storepilot.db.dao;

import androidx.lifecycle.LiveData;
import androidx.room.Dao;
import androidx.room.Delete;
import androidx.room.Insert;
import androidx.room.OnConflictStrategy;
import androidx.room.Query;
import androidx.room.Update;

import com.storepilot.db.entities.Task;

import java.util.List;

@Dao
public interface TaskDao {

@Insert(onConflict = OnConflictStrategy.REPLACE)
long insert(Task task);

@Update
void update(Task task);

@Delete
void delete(Task task);

@Query("SELECT * FROM tasks ORDER BY createdAt DESC")
LiveData<List<Task>> getAllTasks();

@Query("SELECT * FROM tasks WHERE id = :id LIMIT 1")
LiveData<Task> getById(int id);

@Query("SELECT * FROM tasks WHERE assignedTo = :userId ORDER BY createdAt DESC")
LiveData<List<Task>> getTasksByUser(int userId);

@Query("SELECT * FROM tasks WHERE status = :status ORDER BY createdAt DESC")
LiveData<List<Task>> getTasksByStatus(String status);

@Query("SELECT * FROM tasks WHERE isPrivate = 0 ORDER BY createdAt DESC")
LiveData<List<Task>> getTeamTasks();
```

```

@Query("SELECT * FROM tasks WHERE isPrivate = 1 AND createdBy = :userId ORDER BY createdAt DESC")
LiveData<List<Task>> getPrivateTasks(int userId);

@Query("SELECT COUNT(*) FROM tasks WHERE assignedTo = :userId AND status != 'DONE'")
LiveData<Integer> getPendingTaskCount(int userId);
}
```

db/dao/SupportMessageDao.java

```
package com.storepilot.db.dao;

import androidx.lifecycle.LiveData;
import androidx.room.Dao;
import androidx.room.Insert;
import androidx.room.Query;
import androidx.room.Update;

import com.storepilot.db.entities.SupportMessage;

import java.util.List;

@Dao
public interface SupportMessageDao {

// Save a new support message
@Insert
void insert(SupportMessage message);

// Mark messages as read
@Update
void update(SupportMessage message);

// Get all messages for a customer's conversation (sorted oldest first)
@Query("SELECT * FROM support_messages WHERE customerId = :customerId ORDER BY timestamp ASC")
LiveData<List<SupportMessage>> getMessagesForCustomer(int customerId);

// Get distinct customer IDs that have sent messages (for support inbox list)
@Query("SELECT DISTINCT customerId FROM support_messages ORDER BY timestamp DESC")
LiveData<List<Integer>> getConversationCustomerIds();

// Get the latest message from each conversation (for inbox preview)
@Query("SELECT * FROM support_messages WHERE customerId = :customerId ORDER BY timestamp DESC LIMIT
1")
SupportMessage getLatestMessage(int customerId);

// Count unread messages for a customer conversation
@Query("SELECT COUNT(*) FROM support_messages WHERE customerId = :customerId AND isRead = 0 AND
senderRole = 'CUSTOMER'")
LiveData<Integer> getUnreadCount(int customerId);

// Mark all messages in a conversation as read
```

```
@Query("UPDATE support_messages SET isRead = 1 WHERE customerId = :customerId")
void markAllAsRead(int customerId);
}
```

repositories/TaskRepository.java

```
package com.storepilot.repositories;

import android.app.Application;

import androidx.lifecycle.LiveData;

import com.storepilot.core.AppDatabase;
import com.storepilot.db.dao.TaskDao;
import com.storepilot.db.entities.Task;

import java.util.List;

public class TaskRepository {

private final TaskDao taskDao;

public TaskRepository(Application application) {
AppDatabase db = AppDatabase.getInstance(application);
taskDao = db.taskDao();
}

public LiveData<List<Task>> getAllTasks() {
return taskDao.getAllTasks();
}

public LiveData<Task> getById(int id) {
return taskDao.getById(id);
}

public LiveData<List<Task>> getTasksByUser(int userId) {
return taskDao.getTasksByUser(userId);
}

public LiveData<List<Task>> getTasksByStatus(String status) {
return taskDao.getTasksByStatus(status);
}

public LiveData<List<Task>> getTeamTasks() {
return taskDao.getTeamTasks();
}
```

```

public LiveData<List<Task>> getPrivateTasks(int userId) {
return taskDao.getPrivateTasks(userId);
}

public LiveData<Integer> getPendingTaskCount(int userId) {
return taskDao.getPendingTaskCount(userId);
}

public void insert(Task task) {
AppDatabase.dbExecutor.execute(() -> taskDao.insert(task));
}

public void update(Task task) {
AppDatabase.dbExecutor.execute(() -> taskDao.update(task));
}

public void delete(Task task) {
AppDatabase.dbExecutor.execute(() -> taskDao.delete(task));
}
}
```

repositories/SupportRepository.java

```
package com.storepilot.repositories;

import android.app.Application;

import androidx.lifecycle.LiveData;

import com.storepilot.core.AppDatabase;
import com.storepilot.db.dao.SupportMessageDao;
import com.storepilot.db.entities.SupportMessage;

import java.util.List;

// Handles support chat messages between customers and the store team
public class SupportRepository {

private final SupportMessageDao supportMessageDao;

public SupportRepository(Application application) {
AppDatabase db = AppDatabase.getInstance(application);
supportMessageDao = db.supportMessageDao();
}

// Send a text message in the support chat
public void sendMessage(int senderId, String senderRole, String text, int customerId) {
AppDatabase.dbExecutor.execute(() -> {
SupportMessage msg = new SupportMessage(senderId, senderRole, text, null,
System.currentTimeMillis(), customerId);
supportMessageDao.insert(msg);
});
}

// Send a message with an attached image
public void sendImageMessage(int senderId, String senderRole, String imageUrl, int customerId) {
AppDatabase.dbExecutor.execute(() -> {
SupportMessage msg = new SupportMessage(senderId, senderRole, "", imageUrl,
System.currentTimeMillis(), customerId);
supportMessageDao.insert(msg);
});
}
```

```
// Get all messages in a customer's conversation (live updates)
public LiveData<List<SupportMessage>> getMessagesForCustomer(int customerId) {
return supportMessageDao.getMessagesForCustomer(customerId);
}

// Get list of customer IDs who have open conversations (for inbox)
public LiveData<List<Integer>> getConversationCustomerIds() {
return supportMessageDao.getConversationCustomerIds();
}

// Mark all messages in a conversation as read
public void markAllAsRead(int customerId) {
AppDatabase.dbExecutor.execute(() -> supportMessageDao.markAllAsRead(customerId));
}

// Get count of unread messages
public LiveData<Integer> getUnreadCount(int customerId) {
return supportMessageDao.getUnreadCount(customerId);
}
}
```

viewmodels/TaskViewModel.java

```
package com.storepilot.viewmodels;

import android.app.Application;

import androidx.annotation.NonNull;
import androidx.lifecycle.AndroidViewModel;
import androidx.lifecycle.LiveData;

import com.storepilot.db.entities.Task;
import com.storepilot.repositories.TaskRepository;

import java.util.List;

public class TaskViewModel extends AndroidViewModel {

private final TaskRepository repository;

public TaskViewModel(@NonNull Application application) {
super(application);
repository = new TaskRepository(application);
}

public LiveData<List<Task>> getAllTasks() {
return repository.getAllTasks();
}

public LiveData<Task> getById(int id) {
return repository.getById(id);
}

public LiveData<List<Task>> getTasksByUser(int userId) {
return repository.getTasksByUser(userId);
}

public LiveData<List<Task>> getTasksByStatus(String status) {
return repository.getTasksByStatus(status);
}

public LiveData<List<Task>> getTeamTasks() {
return repository.getTeamTasks();
```

```
}

public LiveData<List<Task>> getPrivateTasks(int userId) {
return repository.getPrivateTasks(userId);
}

public LiveData<Integer> getPendingTaskCount(int userId) {
return repository.getPendingTaskCount(userId);
}

public void insert(Task task) {
repository.insert(task);
}

public void update(Task task) {
repository.update(task);
}

public void delete(Task task) {
repository.delete(task);
}
}
```

viewmodels/SupportViewModel.java

```
package com.storepilot.viewmodels;

import android.app.Application;

import androidx.annotation.NonNull;
import androidx.lifecycle.AndroidViewModel;
import androidx.lifecycle.LiveData;

import com.storepilot.db.entities.SupportMessage;
import com.storepilot.repositories.SupportRepository;

import java.util.List;

// ViewModel for the support chat screens
public class SupportViewModel extends AndroidViewModel {

private final SupportRepository supportRepository;

public SupportViewModel(@NonNull Application application) {
super(application);
supportRepository = new SupportRepository(application);
}

// Send a text message to support (or reply as manager)
public void sendMessage(int senderId, String senderRole, String text, int customerId) {
supportRepository.sendMessage(senderId, senderRole, text, customerId);
}

// Send a message with an image attachment
public void sendImageMessage(int senderId, String senderRole, String imageUrl, int customerId) {
supportRepository.sendImageMessage(senderId, senderRole, imageUrl, customerId);
}

// Observe messages in a conversation (auto-updates when new messages arrive)
public LiveData<List<SupportMessage>> getMessages(int customerId) {
return supportRepository.getMessagesForCustomer(customerId);
}

// Get all customer IDs with open conversations (for manager inbox)
public LiveData<List<Integer>> getConversationIds() {
```

```
return supportRepository.getConversationCustomerIds();
}

// Mark all messages in a conversation as read
public void markRead(int customerId) {
supportRepository.markAllAsRead(customerId);
}

// Get unread message count for a conversation
public LiveData<Integer> getUnreadCount(int customerId) {
return supportRepository.getUnreadCount(customerId);
}
}
```

tasks/TaskListFragment.java

```
package com.storepilot.tasks;

import android.content.Intent;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.lifecycle.ViewModelProvider;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.google.android.material.floatingactionbutton.FloatingActionButton;
import com.google.android.material.tabs.TabLayout;
import com.storepilot.R;
import com.storepilot.core.PermissionManager;
import com.storepilot.core.SessionManager;
import com.storepilot.viewmodels.TaskViewModel;

public class TaskListFragment extends Fragment {

private RecyclerView recyclerView;
private FloatingActionButton fabAdd;
private TabLayout tabLayout;
private TaskAdapter adapter;
private TaskViewModel taskViewModel;

// Single MutableLiveData-backed selection to avoid multiple observers
private androidx.lifecycle.MutableLiveData<Integer> selectedTab =
new androidx.lifecycle.MutableLiveData<>(0);

@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container,
@Nullable Bundle savedInstanceState) {
return inflater.inflate(R.layout.fragment_task_list, container, false);
}
```

```
@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
super.onViewCreated(view, savedInstanceState);

recyclerView = view.findViewById(R.id.recyclerTasks);
fabAdd = view.findViewById(R.id.fabAddTask);
tabLayout = view.findViewById(R.id.tabLayoutTasks);

adapter = new TaskAdapter();
recyclerView.setLayoutManager(new LinearLayoutManager(getContext()));
recyclerView.setAdapter(adapter);

taskViewModel = new ViewModelProvider(this).get(TaskViewModel.class);

int userId = SessionManager.getInstance().getLoggedInUser() != null
? SessionManager.getInstance().getLoggedInUser().getId() : 0;

// Use switchMap to avoid stacking observers on tab change
androidx.lifecycle.LiveData<java.util.List<com.storepilot.db.entities.Task>> tasksLiveData =
androidx.lifecycle.Transformations.switchMap(selectedTab, tab -> {
if (tab == null || tab == 0) return taskViewModel.getTasksByUser(userId);
if (tab == 1) return taskViewModel.getTeamTasks();
return taskViewModel.getPrivateTasks(userId);
});
tasksLiveData.observe(getViewLifecycleOwner(), tasks -> adapter.setTasks(tasks));

tabLayout.addOnTabSelectedListener(new TabLayout.OnTabSelectedListener() {
@Override
public void onTabSelected(TabLayout.Tab tab) {
selectedTab.setValue(tab.getPosition());
}
@Override public void onTabUnselected(TabLayout.Tab tab) {}
@Override public void onTabReselected(TabLayout.Tab tab) {}
});

fabAdd.setOnClickListener(v ->
startActivity(new Intent(getActivity(), AddEditTaskActivity.class)));
}
}
```

tasks/AddEditTaskActivity.java

```
package com.storepilot.tasks;

import android.os.Bundle;
import android.widget.Button;
import android.widget.CheckBox;
import android.widget.EditText;
import android.widget.Spinner;
import android.widget.ArrayAdapter;
import android.widget.Toast;

import androidx.lifecycle.ViewModelProvider;

import com.storepilot.R;
import com.storepilot.core.BaseActivity;
import com.storepilot.core.SessionManager;
import com.storepilot.db.entities.Task;
import com.storepilot.viewmodels.TaskViewModel;

import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Locale;

public class AddEditTaskActivity extends BaseActivity {

public static final String EXTRA_TASK_ID = "task_id";

private EditText etTaskTitle, etTaskDescription, etAssignedTo, etDueDate;
private Spinner spinnerStatus, spinnerPriority;
private CheckBox cbPrivate;
private Button btnSaveTask;
private TaskViewModel taskViewModel;
private Task existingTask = null;
private final SimpleDateFormat dateFormat = new SimpleDateFormat("yyyy-MM-dd", Locale.getDefault());

@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_add_edit_task);

etTaskTitle = findViewById(R.id.etTaskTitle);
```

```
etTaskDescription = findViewById(R.id.etTaskDescription);
etAssignedTo = findViewById(R.id.etAssignedTo);
etDueDate = findViewById(R.id.etTaskDueDate);
spinnerStatus = findViewById(R.id.spinnerTaskStatus);
spinnerPriority = findViewById(R.id.spinnerTaskPriority);
cbPrivate = findViewById(R.id.cbTaskPrivate);
btnSaveTask = findViewById(R.id.btnSaveTask);

ArrayAdapter<CharSequence> statusAdapter = ArrayAdapter.createFromResource(
this, R.array.task_statuses, android.R.layout.simple_spinner_item);
statusAdapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
spinnerStatus.setAdapter(statusAdapter);

ArrayAdapter<CharSequence> priorityAdapter = ArrayAdapter.createFromResource(
this, R.array.task_priorities, android.R.layout.simple_spinner_item);
priorityAdapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
spinnerPriority.setAdapter(priorityAdapter);

taskViewModel = new ViewModelProvider(this).get(TaskViewModel.class);

int taskId = getIntent().getIntExtra(EXTRA_TASK_ID, -1);
if (taskId != -1) {
taskViewModel.getById(taskId).observe(this, task -> {
if (task != null && existingTask == null) {
existingTask = task;
etTaskTitle.setText(task.getTitle());
etTaskDescription.setText(task.getDescription());
if (task.getDueDate() > 0) {
etDueDate.setText(dateFormat.format(new java.util.Date(task.getDueDate())));
}
cbPrivate.setChecked(task.isPrivate());
}
});
}

btnSaveTask.setOnClickListener(v -> saveTask());
}

private void saveTask() {
String title = etTaskTitle.getText().toString().trim();
```

```
String description = etTaskDescription.getText().toString().trim();
String dueDateStr = etDueDate.getText().toString().trim();
String status = spinnerStatus.getSelectedItem().toString();
String priority = spinnerPriority.getSelectedItem().toString();
boolean isPrivate = cbPrivate.isChecked();
int createdBy = SessionManager.getInstance().getLoggedInUser() != null
? SessionManager.getInstance().getLoggedInUser().getId() : 0;

if (title.isEmpty()) {
Toast.makeText(this, getString(R.string.error_fill_all_fields), Toast.LENGTH_SHORT).show();
return;
}

long dueDate = 0;
if (!dueDateStr.isEmpty()) {
try {
dueDate = dateFormat.parse(dueDateStr).getTime();
} catch (ParseException e) {
Toast.makeText(this, getString(R.string.error_invalid_date), Toast.LENGTH_SHORT).show();
return;
}
}

Integer assignedTo = null;
String assignedStr = etAssignedTo.getText().toString().trim();
if (!assignedStr.isEmpty()) {
try { assignedTo = Integer.parseInt(assignedStr); } catch (NumberFormatException ignored) {}
}

if (existingTask != null) {
existingTask.title = title;
existingTask.description = description;
existingTask.assignedTo = assignedTo;
existingTask.status = status;
existingTask.priority = priority;
existingTask.isPrivate = isPrivate;
existingTask.dueDate = dueDate;
taskViewModel.update(existingTask);
} else {
Task task = new Task(title, description, assignedTo, createdBy, status, priority,
```

```
isPrivate, dueDate, System.currentTimeMillis());
taskViewModel.insert(task);
}
finish();
}
}
```

customer/SupportChatFragment.java

```
package com.storepilot.customer;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.EditText;
import android.widget.ImageButton;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.lifecycle.ViewModelProvider;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.core.SessionManager;
import com.storepilot.customer.adapters.MessageAdapter;
import com.storepilot.db.entities.User;
import com.storepilot.viewmodels.SupportViewModel;

import java.util.ArrayList;

// WhatsApp-style support chat screen for customers
public class SupportChatFragment extends Fragment {

private RecyclerView rvMessages;
private EditText etMessage;
private ImageButton btnSend;
private TextView tvEmpty;
private SupportViewModel supportViewModel;
private MessageAdapter adapter;

@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater,
@Nullable ViewGroup container,
@Nullable Bundle savedInstanceState) {
```

```
return inflater.inflate(R.layout.fragment_support_chat, container, false);
}

@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
super.onViewCreated(view, savedInstanceState);

rvMessages = view.findViewById(R.id.rvMessages);
etMessage = view.findViewById(R.id.etMessage);
btnSend = view.findViewById(R.id.btnSend);
tvEmpty = view.findViewById(R.id.tvEmpty);

// Linear layout so messages appear in order, newest at bottom
LinearLayoutManager layoutManager = new LinearLayoutManager(requireContext());
layoutManager.setStackFromEnd(true);
rvMessages.setLayoutManager(layoutManager);

supportViewModel = new ViewModelProvider(this).get(SupportViewModel.class);

User currentUser = SessionManager.getInstance().getLoggedInUser();
int customerId = currentUser.getId();

// Adapter shows sent messages on the right, received on the left
adapter = new MessageAdapter(new ArrayList<>(), currentUser.getId());
rvMessages.setAdapter(adapter);

// Load messages and scroll to bottom when they update
supportViewModel.getMessages(customerId).observe(getViewLifecycleOwner(), messages -> {
adapter.setMessages(messages != null ? messages : new ArrayList<>());
boolean empty = messages == null || messages.isEmpty();
tvEmpty.setVisibility(empty ? View.VISIBLE : View.GONE);
// Scroll to latest message
if (!empty) {
rvMessages.scrollToPosition(messages.size() - 1);
}
});

// Mark all messages as read when the chat is opened
supportViewModel.markRead(customerId);
```

```
// Send button — post the message
btnSend.setOnClickListener(v -> {
String text = etMessage.getText().toString().trim();
if (text.isEmpty()) return;

supportViewModel.sendMessage(currentUser.getId(), currentUser.getRole(), text, customerId);
etMessage.setText("");
});
}
}
```

manager/SupportConversationsFragment.java

```
package com.storepilot.manager;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.lifecycle.ViewModelProvider;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.core.AppDatabase;
import com.storepilot.db.entities.User;
import com.storepilot.viewmodels.SupportViewModel;

import java.util.ArrayList;
import java.util.List;

// Manager inbox — lists all customers who have open support conversations
public class SupportConversationsFragment extends Fragment {

private RecyclerView rvConversations;
private TextView tvEmpty;
private SupportViewModel supportViewModel;

@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater,
@Nullable ViewGroup container,
@Nullable Bundle savedInstanceState) {
return inflater.inflate(R.layout.fragment_support_conversations, container, false);
}

@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
```

```
super.onViewCreated(view, savedInstanceState);

rvConversations = view.findViewById(R.id.rvConversations);
tvEmpty = view.findViewById(R.id.tvEmpty);

rvConversations.setLayoutManager(new LinearLayoutManager(requireContext()));
supportViewModel = new ViewModelProvider(this).get(SupportViewModel.class);

// Observe the list of customer IDs with conversations
supportViewModel.getConversationIds().observe(getViewLifecycleOwner(), customerIds -> {
if (customerIds == null || customerIds.isEmpty()) {
tvEmpty.setVisibility(View.VISIBLE);
rvConversations.setVisibility(View.GONE);
return;
}

tvEmpty.setVisibility(View.GONE);
rvConversations.setVisibility(View.VISIBLE);

// Load user details for each conversation
AppDatabase.dbExecutor.execute(() -> {
List<User> customers = new ArrayList<>();
for (int id : customerIds) {
User u = AppDatabase.getInstance(requireContext()).userDao().findById(id);
if (u != null) customers.add(u);
}

requireActivity().runOnUiThread(() -> {
// Build a simple list of conversation items
ConversationListAdapter adapter = new ConversationListAdapter(customers, customer -> {
// Open the chat for the selected customer
requireActivity().getSupportFragmentManager()
.beginTransaction()
.replace(R.id.fragmentContainer,
SupportInboxFragment.forCustomer(customer.getId()))
.addToBackStack(null)
.commit();
});
rvConversations.setAdapter(adapter);
});
```

```
});
});
}
}
```

manager/SupportInboxFragment.java

```
package com.storepilot.manager;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.EditText;
import android.widget.ImageButton;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.lifecycle.ViewModelProvider;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.core.AppDatabase;
import com.storepilot.core.SessionManager;
import com.storepilot.customer.adapters.MessageAdapter;
import com.storepilot.db.entities.User;
import com.storepilot.viewmodels.SupportViewModel;

import java.util.ArrayList;

// Manager's support chat — shows conversation with a specific customer and allows replies
public class SupportInboxFragment extends Fragment {

// The customer ID whose conversation is being viewed
private int targetCustomerId;
private RecyclerView rvMessages;
private EditText etReply;
private ImageButton btnSend;
private TextView tvCustomerName, tvEmpty;
private SupportViewModel supportViewModel;
private MessageAdapter adapter;

// Pass the customer ID when navigating to this fragment
public static SupportInboxFragment forCustomer(int customerId) {
```

```
SupportInboxFragment f = new SupportInboxFragment();
Bundle args = new Bundle();
args.putInt("customerId", customerId);
f.setArguments(args);
return f;
}

@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater,
@Nullable ViewGroup container,
@Nullable Bundle savedInstanceState) {
return inflater.inflate(R.layout.fragment_support_inbox, container, false);
}

@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
super.onViewCreated(view, savedInstanceState);

targetCustomerId = getArguments() != null ? getArguments().getInt("customerId", -1) : -1;

rvMessages = view.findViewById(R.id.rvMessages);
etReply = view.findViewById(R.id.etReply);
btnSend = view.findViewById(R.id.btnSendReply);
tvCustomerName = view.findViewById(R.id.tvCustomerName);
tvEmpty = view.findViewById(R.id.tvEmpty);

LinearLayoutManager lm = new LinearLayoutManager(requireContext());
lm.setStackFromEnd(true);
rvMessages.setLayoutManager(lm);

supportViewModel = new ViewModelProvider(this).get(SupportViewModel.class);

User currentUser = SessionManager.getInstance().getLoggedInUser();

// Show the customer's name in the toolbar
AppDatabase.dbExecutor.execute(() -> {
User customer = AppDatabase.getInstance(requireContext()).userDao().findById(targetCustomerId);
if (customer != null && getActivity() != null) {
requireActivity().runOnUiThread(() ->
```

```
tvCustomerName.setText(customer.getFullName()));
}
});

// Use the manager's ID as currentUserId so sent messages appear on the right
adapter = new MessageAdapter(new ArrayList<>(), currentUser.getId());
rvMessages.setAdapter(adapter);

// Load conversation messages
supportViewModel.getMessages(targetCustomerId).observe(getViewLifecycleOwner(), messages -> {
adapter.setMessages(messages != null ? messages : new ArrayList<>());
boolean empty = messages == null || messages.isEmpty();
tvEmpty.setVisibility(empty ? View.VISIBLE : View.GONE);
if (!empty) rvMessages.scrollToPosition(messages.size() - 1);
});

// Mark messages as read when manager opens the conversation
supportViewModel.markRead(targetCustomerId);

// Send a reply from the manager
btnSend.setOnClickListener(v -> {
String text = etReply.getText().toString().trim();
if (text.isEmpty()) return;
supportViewModel.sendMessage(currentUser.getId(), currentUser.getRole(), text, targetCustomerId);
etReply.setText("");
});
}
}
```

125 Utilities & System

App entry, notifications, dashboard, sales, purchases, seasons

Application entry points, system utilities (AlarmManager, BroadcastReceiver, NotificationManager), the management dashboard, and the sales/purchase/season history modules. Includes the full Gradle build configuration.

StorePilotApp.java

```
package com.storepilot;

import android.app.AlarmManager;
import android.app.Application;
import android.app.PendingIntent;
import android.content.Context;
import android.content.Intent;

import com.google.firebase.FirebaseApp;
import com.storepilot.core.LowStockReceiver;
import com.storepilot.core.NotificationHelper;

public class StorePilotApp extends Application {

@Override
public void onCreate() {
super.onCreate();
FirebaseApp.initializeApp(this);
NotificationHelper.createNotificationChannel(this);
scheduleLowStockAlarm(this);
}

public static void scheduleLowStockAlarm(Context context) {
AlarmManager alarmManager =
(AlarmManager) context.getSystemService(Context.ALARM_SERVICE);
if (alarmManager == null) return;

Intent intent = new Intent(context, LowStockReceiver.class);
PendingIntent pendingIntent = PendingIntent.getBroadcast(
context,
0,
intent,
PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE
);

// Check once per hour; setInexactRepeating needs no special permission
alarmManager.setInexactRepeating(
AlarmManager.RTC_WAKEUP,
System.currentTimeMillis() + AlarmManager.INTERVAL_HOUR,
AlarmManager.INTERVAL_HOUR,
```

```
pendingIntent
);
}
}
```

MainActivity.java

```
package com.storepilot;

import android.Manifest;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.os.Build;
import android.os.Bundle;
import android.view.Menu;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;
import androidx.fragment.app.Fragment;
import androidx.fragment.app.FragmentTransaction;

import com.google.android.material.bottomnavigation.BottomNavigationView;
import com.storepilot.auth.LoginActivity;
import com.storepilot.core.BaseActivity;
import com.storepilot.core.LowStockReceiver;
import com.storepilot.core.PermissionManager;
import com.storepilot.core.SessionManager;
import com.storepilot.dashboard.DashboardFragment;
import com.storepilot.inventory.ProductListFragment;
import com.storepilot.purchases.PurchaseHistoryFragment;
import com.storepilot.seasons.SeasonListFragment;
import com.storepilot.sales.SalesHistoryFragment;
import com.storepilot.tasks.TaskListFragment;
import com.storepilot.admin.UserManagementFragment;
import com.storepilot.manager.OrderManagementFragment;
import com.storepilot.manager.SupportConversationsFragment;

public class MainActivity extends BaseActivity {

private static final int REQUEST_NOTIFICATION_PERMISSION = 100;
private BottomNavigationView bottomNavigationView;

@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
```

```
if (!SessionManager.getInstance().isLoggedIn()) {
startActivity(new Intent(this, LoginActivity.class));
finish();
return;
}

setContentView(R.layout.activity_main);

// Request notification permission on Android 13+
requestNotificationPermissionIfNeeded();

bottomNavigationView = findViewById(R.id.bottomNavigation);

bottomNavigationView.setOnItemSelectedListener(item -> {
int id = item.getItemId();
if (id == R.id.nav_dashboard) {
loadFragment(new DashboardFragment());
return true;
} else if (id == R.id.nav_inventory) {
loadFragment(new ProductListFragment());
return true;
} else if (id == R.id.nav_sales) {
loadFragment(new SalesHistoryFragment());
return true;
} else if (id == R.id.nav_tasks) {
loadFragment(new TaskListFragment());
return true;
} else if (id == R.id.nav_more) {
showMoreMenu();
return true;
}
return false;
});

if (savedInstanceState == null) {
bottomNavigationView.setSelectedItemId(R.id.nav_dashboard);
}
}
```

```
private void requestNotificationPermissionIfNeeded() {
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
if (ContextCompat.checkSelfPermission(this, Manifest.permission.POST_NOTIFICATIONS)
!= PackageManager.PERMISSION_GRANTED) {
ActivityCompat.requestPermissions(
this,
new String[]{Manifest.permission.POST_NOTIFICATIONS},
REQUEST_NOTIFICATION_PERMISSION
);
}
}
}

@Override
public void onRequestPermissionsResult(int requestCode, @NonNull String[] permissions,
@NonNull int[] grantResults) {
super.onRequestPermissionsResult(requestCode, permissions, grantResults);
if (requestCode == REQUEST_NOTIFICATION_PERMISSION) {
if (grantResults.length > 0 && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
Toast.makeText(this, "Notifications enabled", Toast.LENGTH_SHORT).show();
} else {
Toast.makeText(this, "Notifications blocked — enable in Settings", Toast.LENGTH_LONG).show();
}
}
}

@Override
public boolean onCreateOptionsMenu(Menu menu) {
return super.onCreateOptionsMenu(menu);
}

private void loadFragment(Fragment fragment) {
FragmentTransaction transaction = getSupportFragmentManager().beginTransaction();
transaction.replace(R.id.fragmentContainer, fragment);
transaction.commit();
}

private void showMoreMenu() {
androidx.appcompat.widget.PopupMenu popupMenu =
new androidx.appcompat.widget.PopupMenu(this, findViewById(R.id.nav_more));
```

```
popupMenu.getMenuInflater().inflate(R.menu.more_menu, popupMenu.getMenu());

Menu menu = popupMenu.getMenu();
menu.findItem(R.id.menu_purchases).setVisible(
checkPermission(PermissionManager.MANAGE_PURCHASES));
menu.findItem(R.id.menu_seasons).setVisible(
checkPermission(PermissionManager.MANAGE_SEASONS));
menu.findItem(R.id.menu_admin).setVisible(
checkPermission(PermissionManager.VIEW_ADMIN));
// Test notification visible to owner and store manager only
menu.findItem(R.id.menu_test_notification).setVisible(
checkPermission(PermissionManager.VIEW_REPORTS));

popupMenu.setOnMenuItemClickListener(item -> {
int id = item.getItemId();
if (id == R.id.menu_purchases) {
loadFragment(new PurchaseHistoryFragment());
} else if (id == R.id.menu_seasons) {
loadFragment(new SeasonListFragment());
} else if (id == R.id.menu_admin) {
loadFragment(new UserManagementFragment());
} else if (id == R.id.menu_orders) {
loadFragment(new OrderManagementFragment());
} else if (id == R.id.menu_support) {
loadFragment(new SupportConversationsFragment());
} else if (id == R.id.menu_logout) {
SessionManager.getInstance().logout();
startActivity(new Intent(this, LoginActivity.class));
finish();
} else if (id == R.id.menu_test_notification) {
Toast.makeText(this, "Checking low stock...", Toast.LENGTH_SHORT).show();
LowStockReceiver.checkNow(this, true);
}
return true;
});
popupMenu.show();
}
}
```

customer/CustomerMainActivity.java

```
package com.storepilot.customer;

import android.content.Intent;
import android.os.Bundle;
import android.view.MenuItem;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.Fragment;

import com.google.android.material.bottomnavigation.BottomNavigationView;
import com.storepilot.R;
import com.storepilot.auth.LoginActivity;
import com.storepilot.core.SessionManager;

// Main activity for customers — hosts the bottom navigation and fragment container
public class CustomerMainActivity extends AppCompatActivity {

private BottomNavigationView bottomNav;

@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_customer_main);

bottomNav = findViewById(R.id.customerBottomNav);

// Load the home fragment by default
if (savedInstanceState == null) {
loadFragment(new CustomerHomeFragment());
}

// Handle bottom navigation tab switching
bottomNav.setOnItemSelectedListener(item -> {
int id = item.getItemId();
if (id == R.id.nav_customer_home) {
loadFragment(new CustomerHomeFragment());
return true;
} else if (id == R.id.nav_customer_cart) {
loadFragment(new CartFragment());
```

```
return true;
} else if (id == R.id.nav_customer_orders) {
loadFragment(new OrderHistoryFragment());
return true;
} else if (id == R.id.nav_customer_favorites) {
loadFragment(new FavoritesFragment());
return true;
} else if (id == R.id.nav_customer_support) {
loadFragment(new SupportChatFragment());
return true;
}
return false;
});
}

// Replace the main container with a new fragment
private void loadFragment(Fragment fragment) {
getSupportFragmentManager()
.beginTransaction()
.replace(R.id.customerFragmentContainer, fragment)
.commit();
}

// Log out and return to login screen
public void logout() {
SessionManager.getInstance().logout();
startActivity(new Intent(this, LoginActivity.class));
finish();
}
}
```

core/NotificationHelper.java

```
package com.storepilot.core;

import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.content.Context;
import android.os.Build;

import androidx.core.app.NotificationCompat;

import com.storepilot.R;

public class NotificationHelper {

public static final String CHANNEL_ID = "storepilot_low_stock";
private static final String CHANNEL_NAME = "Low Stock Alerts";
private static final int NOTIFICATION_ID = 1001;

public static void createNotificationChannel(Context context) {
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
NotificationChannel channel = new NotificationChannel(
CHANNEL_ID,
CHANNEL_NAME,
NotificationManager.IMPORTANCE_HIGH
);
channel.setDescription("Alerts when products are running low on stock");
channel.enableVibration(true);
NotificationManager manager = context.getSystemService(NotificationManager.class);
if (manager != null) {
manager.createNotificationChannel(channel);
}
}
}

public static void sendLowStockNotification(Context context, int count) {
String text = count + " product(s) are running low. Tap to review.";

NotificationCompat.Builder builder = new NotificationCompat.Builder(context, CHANNEL_ID)
.setSmallIcon(R.drawable.ic_notification)
.setContentTitle("Low Stock Alert")
.setContentText(text)
```

```
.setStyle(new NotificationCompat.BigTextStyle().bigText(text))
.setPriority(NotificationCompat.PRIORITY_HIGH)
.setAutoCancel(true);

NotificationManager manager =
(NotificationManager) context.getSystemService(Context.NOTIFICATION_SERVICE);
if (manager != null) {
manager.notify(NOTIFICATION_ID, builder.build());
}
}
}
```

core/LowStockReceiver.java

```
package com.storepilot.core;

import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.widget.Toast;

import com.google.firebase.firestore.FirebaseFirestore;

public class LowStockReceiver extends BroadcastReceiver {

public static final int LOW_STOCK_THRESHOLD = 5;

@Override
public void onReceive(Context context, Intent intent) {
checkNow(context, false);
}

/**
* Queries Firestore for products at or below LOW_STOCK_THRESHOLD and fires a notification.
*
* @param testMode if true, always fires a notification even when nothing is low
* so you can verify the channel works during development
*/
public static void checkNow(Context context, boolean testMode) {
FirebaseFirestore.getInstance()
.collection("products")
.whereLessThanOrEqualTo("quantity", LOW_STOCK_THRESHOLD)
.get()
.addOnSuccessListener(snapshots -> {
int count = snapshots != null ? snapshots.size() : 0;
if (count > 0) {
NotificationHelper.sendLowStockNotification(context, count);
} else if (testMode) {
// No real low-stock items — send a demo notification so you can
// confirm the channel and permission are working
NotificationHelper.sendLowStockNotification(context, 1);
// Toast runs only in test mode (main thread not guaranteed here,
// but checkNow is called from the UI in test mode)
Toast.makeText(context,
```

```
"No real low-stock items — demo notification sent",
Toast.LENGTH_LONG).show();
}
})
.addOnFailureListener(e -> {
if (testMode) {
Toast.makeText(context,
"Firestore error: " + e.getMessage(),
Toast.LENGTH_LONG).show();
}
});
}
}
```

dashboard/DashboardFragment.java

```
package com.storepilot.dashboard;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.lifecycle.ViewModelProvider;

import com.storepilot.R;
import com.storepilot.core.SessionManager;
import com.storepilot.viewmodels.OrderViewModel;
import com.storepilot.viewmodels.ProductViewModel;
import com.storepilot.viewmodels.SaleViewModel;
import com.storepilot.viewmodels.SeasonViewModel;
import com.storepilot.viewmodels.TaskViewModel;

import java.text.NumberFormat;
import java.util.Calendar;
import java.util.Locale;

public class DashboardFragment extends Fragment {

private TextView tvSeasonAlert, tvTodaySales, tvLowStockCount, tvPendingTasks;
private TextView tvTodayOrders;
private View cardSeasonAlert;

@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container,
@Nullable Bundle savedInstanceState) {
return inflater.inflate(R.layout.fragment_dashboard, container, false);
}

@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
```

```
super.onViewCreated(view, savedInstanceState);

cardSeasonAlert = view.findViewById(R.id.cardSeasonAlert);
tvSeasonAlert = view.findViewById(R.id.tvSeasonAlert);
tvTodaySales = view.findViewById(R.id.tvTodaySales);
tvLowStockCount = view.findViewById(R.id.tvLowStockCount);
tvPendingTasks = view.findViewById(R.id.tvPendingTasks);
tvTodayOrders = view.findViewById(R.id.tvTodayOrders);

SeasonViewModel seasonVM = new ViewModelProvider(this).get(SeasonViewModel.class);
SaleViewModel saleVM = new ViewModelProvider(this).get(SaleViewModel.class);
ProductViewModel productVM = new ViewModelProvider(this).get(ProductViewModel.class);
TaskViewModel taskVM = new ViewModelProvider(this).get(TaskViewModel.class);

long now = System.currentTimeMillis();

// Season alert
seasonVM.getSeasonsEndingSoon(now).observe(getViewLifecycleOwner(), seasons -> {
if (seasons != null && !seasons.isEmpty()) {
cardSeasonAlert.setVisibility(View.VISIBLE);
tvSeasonAlert.setText(getString(R.string.season_ending_soon, seasons.get(0).getName()));
} else {
cardSeasonAlert.setVisibility(View.GONE);
}
});

// Today's sales
Calendar cal = Calendar.getInstance();
cal.set(Calendar.HOUR_OF_DAY, 0);
cal.set(Calendar.MINUTE, 0);
cal.set(Calendar.SECOND, 0);
cal.set(Calendar.MILLISECOND, 0);
long startOfDay = cal.getTimeInMillis();
long endOfDay = startOfDay + 86400000L;

saleVM.getTodaySalesTotal(startOfDay, endOfDay).observe(getViewLifecycleOwner(), total -> {
double value = (total != null) ? total : 0.0;
tvTodaySales.setText(NumberFormat.getCurrencyInstance(Locale.US).format(value));
});
```

```
// Low stock (threshold = 10)
productVM.getLowStockProducts(10).observe(getViewLifecycleOwner(), products -> {
int count = (products != null) ? products.size() : 0;
tvLowStockCount.setText(String.valueOf(count));
});

// Pending tasks
int userId = SessionManager.getInstance().getLoggedInUser() != null
? SessionManager.getInstance().getLoggedInUser().getId() : 0;
taskVM.getPendingTaskCount(userId).observe(getViewLifecycleOwner(), count -> {
tvPendingTasks.setText(String.valueOf(count != null ? count : 0));
});

// Today's order count and revenue from customer orders
OrderViewModel orderVM = new ViewModelProvider(this).get(OrderViewModel.class);
orderVM.getTodayOrderCount(startOfDay).observe(getViewLifecycleOwner(), count -> {
if (tvTodayOrders != null)
tvTodayOrders.setText(String.valueOf(count != null ? count : 0));
});
}
}
```

admin/UserManagementFragment.java

```
package com.storepilot.admin;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.lifecycle.ViewModelProvider;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.viewmodels.AuthViewModel;

import java.util.List;

public class UserManagementFragment extends Fragment {

private RecyclerView recyclerView;
private UserAdapter adapter;

@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container,
@Nullable Bundle savedInstanceState) {
return inflater.inflate(R.layout.fragment_user_management, container, false);
}

@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
super.onViewCreated(view, savedInstanceState);
recyclerView = view.findViewById(R.id.recyclerUsers);
adapter = new UserAdapter();
recyclerView.setLayoutManager(new LinearLayoutManager(getContext()));
recyclerView.setAdapter(adapter);

// Use AuthViewModel's repository indirectly via a shared ViewModel
```

```
// For simplicity, load users via a ProductViewModel equivalent
com.storepilot.repositories.UserRepository repo =
new com.storepilot.repositories.UserRepository(requireActivity().getApplication());
repo.getAllUsers().observe(getViewLifecycleOwner(), users -> adapter.setUsers(users));
}
}
```

db/dao/SaleDao.java

```
package com.storepilot.db.dao;

import androidx.lifecycle.LiveData;
import androidx.room.Dao;
import androidx.room.Delete;
import androidx.room.Insert;
import androidx.room.OnConflictStrategy;
import androidx.room.Query;
import androidx.room.Update;

import com.storepilot.db.entities.Sale;

import java.util.List;

@Dao
public interface SaleDao {

@Insert(onConflict = OnConflictStrategy.REPLACE)
long insert(Sale sale);

@Update
void update(Sale sale);

@Delete
void delete(Sale sale);

@Query("SELECT * FROM sales ORDER BY saleDate DESC")
LiveData<List<Sale>> getAllSales();

@Query("SELECT * FROM sales WHERE id = :id LIMIT 1")
LiveData<Sale> getById(int id);

@Query("SELECT * FROM sales WHERE saleDate BETWEEN :startDate AND :endDate ORDER BY saleDate DESC")
LiveData<List<Sale>> getSalesByDateRange(long startDate, long endDate);

@Query("SELECT SUM(totalPrice) FROM sales WHERE saleDate BETWEEN :startDate AND :endDate")
LiveData<Double> getSalesTotalByRange(long startDate, long endDate);

@Query("SELECT SUM(totalPrice) FROM sales WHERE strftime('%Y-%W', saleDate/1000, 'unixepoch') =
strftime('%Y-%W', :refDate/1000, 'unixepoch')")
LiveData<Double> getSalesTotalByWeek(long refDate);
```

```

@Query("SELECT SUM(totalPrice) FROM sales WHERE strftime('%Y-%m', saleDate/1000, 'unixepoch') =
strftime('%Y-%m', :refDate/1000, 'unixepoch')")
LiveData<Double> getSalesTotalByMonth(long refDate);

@Query("SELECT SUM(totalPrice) FROM sales WHERE strftime('%Y', saleDate/1000, 'unixepoch') =
strftime('%Y', :refDate/1000, 'unixepoch')")
LiveData<Double> getSalesTotalByYear(long refDate);

@Query("SELECT SUM(totalPrice) FROM sales WHERE saleDate >= :startOfDay AND saleDate < :endOfDay")
LiveData<Double> getTodaySalesTotal(long startOfDay, long endOfDay);
}
```

db/dao/PurchaseDao.java

```
package com.storepilot.db.dao;

import androidx.lifecycle.LiveData;
import androidx.room.Dao;
import androidx.room.Delete;
import androidx.room.Insert;
import androidx.room.OnConflictStrategy;
import androidx.room.Query;
import androidx.room.Update;

import com.storepilot.db.entities.Purchase;

import java.util.List;

@Dao
public interface PurchaseDao {

@Insert(onConflict = OnConflictStrategy.REPLACE)
long insert(Purchase purchase);

@Update
void update(Purchase purchase);

@Delete
void delete(Purchase purchase);

@Query("SELECT * FROM purchases ORDER BY purchaseDate DESC")
LiveData<List<Purchase>> getAllPurchases();

@Query("SELECT * FROM purchases WHERE id = :id LIMIT 1")
LiveData<Purchase> getById(int id);

@Query("SELECT * FROM purchases WHERE purchaseDate BETWEEN :startDate AND :endDate ORDER BY
purchaseDate DESC")
LiveData<List<Purchase>> getPurchasesByDateRange(long startDate, long endDate);
}
```

db/dao/SeasonDao.java

```
package com.storepilot.db.dao;

import androidx.lifecycle.LiveData;
import androidx.room.Dao;
import androidx.room.Delete;
import androidx.room.Insert;
import androidx.room.OnConflictStrategy;
import androidx.room.Query;
import androidx.room.Update;

import com.storepilot.db.entities.Season;

import java.util.List;

@Dao
public interface SeasonDao {

@Insert(onConflict = OnConflictStrategy.REPLACE)
long insert(Season season);

@Update
void update(Season season);

@Delete
void delete(Season season);

@Query("SELECT * FROM seasons ORDER BY startDate DESC")
LiveData<List<Season>> getAllSeasons();

@Query("SELECT * FROM seasons WHERE id = :id LIMIT 1")
LiveData<Season> getById(int id);

@Query("SELECT * FROM seasons WHERE endDate <= (:currentDate + (alertDaysBeforeEnd * 86400000)) AND
isActive = 1 ORDER BY endDate ASC")
LiveData<List<Season>> getSeasonsEndingSoon(long currentDate);
}
```

db/dao/VideoMetricDao.java

```
package com.storepilot.db.dao;

import androidx.lifecycle.LiveData;
import androidx.room.Dao;
import androidx.room.Delete;
import androidx.room.Insert;
import androidx.room.OnConflictStrategy;
import androidx.room.Query;
import androidx.room.Update;

import com.storepilot.db.entities.VideoMetric;

import java.util.List;

@Dao
public interface VideoMetricDao {

@Insert(onConflict = OnConflictStrategy.REPLACE)
long insert(VideoMetric metric);

@Update
void update(VideoMetric metric);

@Delete
void delete(VideoMetric metric);

@Query("SELECT * FROM video_metrics ORDER BY videoDate DESC")
LiveData<List<VideoMetric>> getAllMetrics();

@Query("SELECT * FROM video_metrics WHERE id = :id LIMIT 1")
LiveData<VideoMetric> getById(int id);
}
```

repositories/SaleRepository.java

```
package com.storepilot.repositories;

import android.app.Application;

import androidx.lifecycle.LiveData;

import com.storepilot.core.AppDatabase;
import com.storepilot.db.dao.SaleDao;
import com.storepilot.db.entities.Sale;

import java.util.List;

public class SaleRepository {

private final SaleDao saleDao;

public SaleRepository(Application application) {
AppDatabase db = AppDatabase.getInstance(application);
saleDao = db.saleDao();
}

public LiveData<List<Sale>> getAllSales() {
return saleDao.getAllSales();
}

public LiveData<Sale> getById(int id) {
return saleDao.getById(id);
}

public LiveData<List<Sale>> getSalesByDateRange(long startDate, long endDate) {
return saleDao.getSalesByDateRange(startDate, endDate);
}

public LiveData<Double> getSalesTotalByRange(long startDate, long endDate) {
return saleDao.getSalesTotalByRange(startDate, endDate);
}

public LiveData<Double> getSalesTotalByWeek(long refDate) {
return saleDao.getSalesTotalByWeek(refDate);
}
```

```
public LiveData<Double> getSalesTotalByMonth(long refDate) {
return saleDao.getSalesTotalByMonth(refDate);
}

public LiveData<Double> getSalesTotalByYear(long refDate) {
return saleDao.getSalesTotalByYear(refDate);
}

public LiveData<Double> getTodaySalesTotal(long startOfDay, long endOfDay) {
return saleDao.getTodaySalesTotal(startOfDay, endOfDay);
}

public void insert(Sale sale) {
AppDatabase.dbExecutor.execute(() -> saleDao.insert(sale));
}

public void update(Sale sale) {
AppDatabase.dbExecutor.execute(() -> saleDao.update(sale));
}

public void delete(Sale sale) {
AppDatabase.dbExecutor.execute(() -> saleDao.delete(sale));
}
}
```

repositories/PurchaseRepository.java

```
package com.storepilot.repositories;

import android.app.Application;

import androidx.lifecycle.LiveData;

import com.storepilot.core.AppDatabase;
import com.storepilot.db.dao.PurchaseDao;
import com.storepilot.db.entities.Purchase;

import java.util.List;

public class PurchaseRepository {

private final PurchaseDao purchaseDao;

public PurchaseRepository(Application application) {
AppDatabase db = AppDatabase.getInstance(application);
purchaseDao = db.purchaseDao();
}

public LiveData<List<Purchase>> getAllPurchases() {
return purchaseDao.getAllPurchases();
}

public LiveData<Purchase> getById(int id) {
return purchaseDao.getById(id);
}

public LiveData<List<Purchase>> getPurchasesByDateRange(long startDate, long endDate) {
return purchaseDao.getPurchasesByDateRange(startDate, endDate);
}

public void insert(Purchase purchase) {
AppDatabase.dbExecutor.execute(() -> purchaseDao.insert(purchase));
}

public void update(Purchase purchase) {
AppDatabase.dbExecutor.execute(() -> purchaseDao.update(purchase));
}
```

```

public void delete(Purchase purchase) {
AppDatabase.dbExecutor.execute(() -> purchaseDao.delete(purchase));
}
}
```

repositories/SeasonRepository.java

```
package com.storepilot.repositories;

import android.app.Application;

import androidx.lifecycle.LiveData;

import com.storepilot.core.AppDatabase;
import com.storepilot.db.dao.SeasonDao;
import com.storepilot.db.entities.Season;

import java.util.List;

public class SeasonRepository {

private final SeasonDao seasonDao;

public SeasonRepository(Application application) {
AppDatabase db = AppDatabase.getInstance(application);
seasonDao = db.seasonDao();
}

public LiveData<List<Season>> getAllSeasons() {
return seasonDao.getAllSeasons();
}

public LiveData<Season> getById(int id) {
return seasonDao.getById(id);
}

public LiveData<List<Season>> getSeasonsEndingSoon(long currentDate) {
return seasonDao.getSeasonsEndingSoon(currentDate);
}

public void insert(Season season) {
AppDatabase.dbExecutor.execute(() -> seasonDao.insert(season));
}

public void update(Season season) {
AppDatabase.dbExecutor.execute(() -> seasonDao.update(season));
}
```

```

public void delete(Season season) {
AppDatabase.dbExecutor.execute(() -> seasonDao.delete(season));
}
}
```

viewmodels/SaleViewModel.java

```
package com.storepilot.viewmodels;

import android.app.Application;

import androidx.annotation.NonNull;
import androidx.lifecycle.AndroidViewModel;
import androidx.lifecycle.LiveData;

import com.storepilot.db.entities.Sale;
import com.storepilot.repositories.SaleRepository;

import java.util.List;

public class SaleViewModel extends AndroidViewModel {

private final SaleRepository repository;

public SaleViewModel(@NonNull Application application) {
super(application);
repository = new SaleRepository(application);
}

public LiveData<List<Sale>> getAllSales() {
return repository.getAllSales();
}

public LiveData<Sale> getById(int id) {
return repository.getById(id);
}

public LiveData<List<Sale>> getSalesByDateRange(long startDate, long endDate) {
return repository.getSalesByDateRange(startDate, endDate);
}

public LiveData<Double> getSalesTotalByRange(long startDate, long endDate) {
return repository.getSalesTotalByRange(startDate, endDate);
}

public LiveData<Double> getSalesTotalByWeek(long refDate) {
return repository.getSalesTotalByWeek(refDate);
```

```
}

public LiveData<Double> getSalesTotalByMonth(long refDate) {
return repository.getSalesTotalByMonth(refDate);
}

public LiveData<Double> getSalesTotalByYear(long refDate) {
return repository.getSalesTotalByYear(refDate);
}

public LiveData<Double> getTodaySalesTotal(long startOfDay, long endOfDay) {
return repository.getTodaySalesTotal(startOfDay, endOfDay);
}

public void insert(Sale sale) {
repository.insert(sale);
}

public void update(Sale sale) {
repository.update(sale);
}

public void delete(Sale sale) {
repository.delete(sale);
}
}
```

viewmodels/PurchaseViewModel.java

```
package com.storepilot.viewmodels;

import android.app.Application;

import androidx.annotation.NonNull;
import androidx.lifecycle.AndroidViewModel;
import androidx.lifecycle.LiveData;

import com.storepilot.db.entities.Purchase;
import com.storepilot.repositories.PurchaseRepository;

import java.util.List;

public class PurchaseViewModel extends AndroidViewModel {

private final PurchaseRepository repository;

public PurchaseViewModel(@NonNull Application application) {
super(application);
repository = new PurchaseRepository(application);
}

public LiveData<List<Purchase>> getAllPurchases() {
return repository.getAllPurchases();
}

public LiveData<Purchase> getById(int id) {
return repository.getById(id);
}

public LiveData<List<Purchase>> getPurchasesByDateRange(long startDate, long endDate) {
return repository.getPurchasesByDateRange(startDate, endDate);
}

public void insert(Purchase purchase) {
repository.insert(purchase);
}

public void update(Purchase purchase) {
repository.update(purchase);
```

```
}

public void delete(Purchase purchase) {
repository.delete(purchase);
}
}
```

viewmodels/SeasonViewModel.java

```
package com.storepilot.viewmodels;

import android.app.Application;

import androidx.annotation.NonNull;
import androidx.lifecycle.AndroidViewModel;
import androidx.lifecycle.LiveData;

import com.storepilot.db.entities.Season;
import com.storepilot.repositories.SeasonRepository;

import java.util.List;

public class SeasonViewModel extends AndroidViewModel {

private final SeasonRepository repository;

public SeasonViewModel(@NonNull Application application) {
super(application);
repository = new SeasonRepository(application);
}

public LiveData<List<Season>> getAllSeasons() {
return repository.getAllSeasons();
}

public LiveData<Season> getById(int id) {
return repository.getById(id);
}

public LiveData<List<Season>> getSeasonsEndingSoon(long currentDate) {
return repository.getSeasonsEndingSoon(currentDate);
}

public void insert(Season season) {
repository.insert(season);
}

public void update(Season season) {
repository.update(season);
```

```
}

public void delete(Season season) {
repository.delete(season);
}
}
```

sales/SalesHistoryFragment.java

```
package com.storepilot.sales;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.lifecycle.ViewModelProvider;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.viewmodels.SaleViewModel;

public class SalesHistoryFragment extends Fragment {

private RecyclerView recyclerView;
private SaleAdapter adapter;

@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container,
@Nullable Bundle savedInstanceState) {
return inflater.inflate(R.layout.fragment_sales_history, container, false);
}

@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
super.onViewCreated(view, savedInstanceState);
recyclerView = view.findViewById(R.id.recyclerSales);
adapter = new SaleAdapter();
recyclerView.setLayoutManager(new LinearLayoutManager(getContext()));
recyclerView.setAdapter(adapter);

SaleViewModel vm = new ViewModelProvider(this).get(SaleViewModel.class);
vm.getAllSales().observe(getViewLifecycleOwner(), sales -> adapter.setSales(sales));
}
```

```
}
```

sales/AddSaleActivity.java

```
package com.storepilot.sales;

import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.lifecycle.ViewModelProvider;

import com.storepilot.R;
import com.storepilot.core.BaseActivity;
import com.storepilot.core.SessionManager;
import com.storepilot.db.entities.Sale;
import com.storepilot.viewmodels.SaleViewModel;

public class AddSaleActivity extends BaseActivity {

private EditText etProductId, etQuantity, etTotalPrice, etNotes;
private Button btnSaveSale;
private SaleViewModel saleViewModel;

@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_add_sale);

etProductId = findViewById(R.id.etSaleProductId);
etQuantity = findViewById(R.id.etSaleQuantity);
etTotalPrice = findViewById(R.id.etSaleTotalPrice);
etNotes = findViewById(R.id.etSaleNotes);
btnSaveSale = findViewById(R.id.btnSaveSale);

saleViewModel = new ViewModelProvider(this).get(SaleViewModel.class);

btnSaveSale.setOnClickListener(v -> {
String productIdStr = etProductId.getText().toString().trim();
String qtyStr = etQuantity.getText().toString().trim();
String totalStr = etTotalPrice.getText().toString().trim();
String notes = etNotes.getText().toString().trim();
```

```
if (productIdStr.isEmpty() || qtyStr.isEmpty() || totalStr.isEmpty()) {
Toast.makeText(this, getString(R.string.error_fill_all_fields), Toast.LENGTH_SHORT).show();
return;
}

int productId = Integer.parseInt(productIdStr);
int quantity = Integer.parseInt(qtyStr);
double total = Double.parseDouble(totalStr);
int userId = SessionManager.getInstance().getLoggedInUser() != null
? SessionManager.getInstance().getLoggedInUser().getId() : 0;

Sale sale = new Sale(productId, quantity, total, System.currentTimeMillis(), userId, notes);
saleViewModel.insert(sale);
finish();
});
}
}
```

purchases/PurchaseHistoryFragment.java

```
package com.storepilot.purchases;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.lifecycle.ViewModelProvider;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.viewmodels.PurchaseViewModel;

public class PurchaseHistoryFragment extends Fragment {

private RecyclerView recyclerView;
private PurchaseAdapter adapter;

@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container,
@Nullable Bundle savedInstanceState) {
return inflater.inflate(R.layout.fragment_purchase_history, container, false);
}

@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
super.onViewCreated(view, savedInstanceState);
recyclerView = view.findViewById(R.id.recyclerPurchases);
adapter = new PurchaseAdapter();
recyclerView.setLayoutManager(new LinearLayoutManager(getContext()));
recyclerView.setAdapter(adapter);

PurchaseViewModel vm = new ViewModelProvider(this).get(PurchaseViewModel.class);
vm.getAllPurchases().observe(getViewLifecycleOwner(), purchases -> adapter.setPurchases(purchases));
}
```

```
}
```

purchases/AddPurchaseActivity.java

```
package com.storepilot.purchases;

import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.lifecycle.ViewModelProvider;

import com.storepilot.R;
import com.storepilot.core.BaseActivity;
import com.storepilot.core.SessionManager;
import com.storepilot.db.entities.Purchase;
import com.storepilot.viewmodels.PurchaseViewModel;

public class AddPurchaseActivity extends BaseActivity {

private EditText etProductId, etQuantity, etTotalCost, etSupplier, etNotes;
private Button btnSavePurchase;
private PurchaseViewModel purchaseViewModel;

@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_add_purchase);

etProductId = findViewById(R.id.etPurchaseProductId);
etQuantity = findViewById(R.id.etPurchaseQuantity);
etTotalCost = findViewById(R.id.etPurchaseTotalCost);
etSupplier = findViewById(R.id.etPurchaseSupplier);
etNotes = findViewById(R.id.etPurchaseNotes);
btnSavePurchase = findViewById(R.id.btnSavePurchase);

purchaseViewModel = new ViewModelProvider(this).get(PurchaseViewModel.class);

btnSavePurchase.setOnClickListener(v -> {
String productIdStr = etProductId.getText().toString().trim();
String qtyStr = etQuantity.getText().toString().trim();
String costStr = etTotalCost.getText().toString().trim();
String supplier = etSupplier.getText().toString().trim();
```

```
String notes = etNotes.getText().toString().trim();

if (productIdStr.isEmpty() || qtyStr.isEmpty() || costStr.isEmpty()) {
Toast.makeText(this, getString(R.string.error_fill_all_fields), Toast.LENGTH_SHORT).show();
return;
}

int productId = Integer.parseInt(productIdStr);
int quantity = Integer.parseInt(qtyStr);
double cost = Double.parseDouble(costStr);
int userId = SessionManager.getInstance().getLoggedInUser() != null
? SessionManager.getInstance().getLoggedInUser().getId() : 0;

Purchase purchase = new Purchase(productId, quantity, cost,
System.currentTimeMillis(), userId, supplier, notes);
purchaseViewModel.insert(purchase);
finish();
});
}
}
```

seasons/SeasonListFragment.java

```
package com.storepilot.seasons;

import android.content.Intent;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.lifecycle.ViewModelProvider;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.google.android.material.floatingactionbutton.FloatingActionButton;
import com.storepilot.R;
import com.storepilot.viewmodels.SeasonViewModel;

public class SeasonListFragment extends Fragment {

private RecyclerView recyclerView;
private FloatingActionButton fabAdd;
private SeasonAdapter adapter;

@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container,
@Nullable Bundle savedInstanceState) {
return inflater.inflate(R.layout.fragment_season_list, container, false);
}

@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
super.onViewCreated(view, savedInstanceState);
recyclerView = view.findViewById(R.id.recyclerSeasons);
fabAdd = view.findViewById(R.id.fabAddSeason);
adapter = new SeasonAdapter();
recyclerView.setLayoutManager(new LinearLayoutManager(getContext()));
recyclerView.setAdapter(adapter);
```

```

SeasonViewModel vm = new ViewModelProvider(this).get(SeasonViewModel.class);
vm.getAllSeasons().observe(getViewLifecycleOwner(), seasons -> adapter.setSeasons(seasons));

fabAdd.setOnClickListener(v ->
startActivity(new Intent(getActivity(), AddEditSeasonActivity.class)));
}
}
```

seasons/AddEditSeasonActivity.java

```
package com.storepilot.seasons;

import android.os.Bundle;
import android.widget.Button;
import android.widget.CheckBox;
import android.widget.EditText;
import android.widget.Toast;

import androidx.lifecycle.ViewModelProvider;

import com.storepilot.R;
import com.storepilot.core.BaseActivity;
import com.storepilot.db.entities.Season;
import com.storepilot.viewmodels.SeasonViewModel;

import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.Locale;

public class AddEditSeasonActivity extends BaseActivity {

public static final String EXTRA_SEASON_ID = "season_id";

private EditText etSeasonName, etStartDate, etEndDate, etAlertDays, etSeasonNotes;
private CheckBox cbIsActive;
private Button btnSaveSeason;
private SeasonViewModel seasonViewModel;
private Season existingSeason = null;
private final SimpleDateFormat dateFormat = new SimpleDateFormat("yyyy-MM-dd", Locale.getDefault());

@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_add_edit_season);

etSeasonName = findViewById(R.id.etSeasonName);
etStartDate = findViewById(R.id.etSeasonStartDate);
etEndDate = findViewById(R.id.etSeasonEndDate);
etAlertDays = findViewById(R.id.etAlertDays);
```

```
etSeasonNotes = findViewById(R.id.etSeasonNotes);
cbIsActive = findViewById(R.id.cbSeasonActive);
btnSaveSeason = findViewById(R.id.btnSaveSeason);

seasonViewModel = new ViewModelProvider(this).get(SeasonViewModel.class);

int seasonId = getIntent().getIntExtra(EXTRA_SEASON_ID, -1);
if (seasonId != -1) {
seasonViewModel.getById(seasonId).observe(this, season -> {
if (season != null && existingSeason == null) {
existingSeason = season;
etSeasonName.setText(season.getName());
etStartDate.setText(dateFormat.format(new Date(season.getStartDate())));
etEndDate.setText(dateFormat.format(new Date(season.getEndDate())));
etAlertDays.setText(String.valueOf(season.getAlertDaysBeforeEnd()));
etSeasonNotes.setText(season.getNotes());
cbIsActive.setChecked(season.isActive());
}
});
}

btnSaveSeason.setOnClickListener(v -> saveSeason());
}

private void saveSeason() {
String name = etSeasonName.getText().toString().trim();
String startStr = etStartDate.getText().toString().trim();
String endStr = etEndDate.getText().toString().trim();
String alertStr = etAlertDays.getText().toString().trim();
String notes = etSeasonNotes.getText().toString().trim();
boolean isActive = cbIsActive.isChecked();

if (name.isEmpty() || startStr.isEmpty() || endStr.isEmpty()) {
Toast.makeText(this, getString(R.string.error_fill_all_fields), Toast.LENGTH_SHORT).show();
return;
}

try {
long startDate = dateFormat.parse(startStr).getTime();
long endDate = dateFormat.parse(endStr).getTime();
```

```
int alertDays = alertStr.isEmpty() ? 30 : Integer.parseInt(alertStr);

if (existingSeason != null) {
existingSeason.name = name;
existingSeason.startDate = startDate;
existingSeason.endDate = endDate;
existingSeason.alertDaysBeforeEnd = alertDays;
existingSeason.notes = notes;
existingSeason.isActive = isActive;
seasonViewModel.update(existingSeason);
} else {
Season season = new Season(name, startDate, endDate, alertDays, isActive, notes);
seasonViewModel.insert(season);
}
finish();
} catch (ParseException e) {
Toast.makeText(this, getString(R.string.error_invalid_date), Toast.LENGTH_SHORT).show();
}
}
}
```

manager/OrderManagementFragment.java

```
package com.storepilot.manager;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.lifecycle.ViewModelProvider;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.storepilot.R;
import com.storepilot.manager.adapters.ManagerOrderAdapter;
import com.storepilot.viewmodels.OrderViewModel;

import java.util.ArrayList;

// Manager screen for viewing and updating all customer orders
public class OrderManagementFragment extends Fragment {

private RecyclerView rvOrders;
private TextView tvEmpty;
private OrderViewModel orderViewModel;
private ManagerOrderAdapter adapter;

@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater,
@Nullable ViewGroup container,
@Nullable Bundle savedInstanceState) {
return inflater.inflate(R.layout.fragment_order_management, container, false);
}

@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
super.onViewCreated(view, savedInstanceState);
```

```

rvOrders = view.findViewById(R.id.rvManagerOrders);
tvEmpty = view.findViewById(R.id.tvEmpty);

rvOrders.setLayoutManager(new LinearLayoutManager(requireContext()));
orderViewModel = new ViewModelProvider(this).get(OrderViewModel.class);

// Adapter with status change callback
adapter = new ManagerOrderAdapter(new ArrayList<>(), (order, newStatus) ->
orderViewModel.updateOrderStatus(order, newStatus));
rvOrders.setAdapter(adapter);

// Load all orders from all customers
orderViewModel.getAllOrders().observe(getViewLifecycleOwner(), orders -> {
adapter.setOrders(orders != null ? orders : new ArrayList<>());
boolean empty = orders == null || orders.isEmpty();
tvEmpty.setVisibility(empty ? View.VISIBLE : View.GONE);
rvOrders.setVisibility(empty ? View.GONE : View.VISIBLE);
});
}
}
```

app/build.gradle

```
plugins {
id 'com.android.application'
}

android {
namespace 'com.storepilot'
compileSdk 34

defaultConfig {
applicationId "com.storepilot"
minSdk 24
targetSdk 34
versionCode 1
versionName "1.0"
javaCompileOptions {
annotationProcessorOptions {
arguments += ["room.schemaLocation": "$projectDir/schemas".toString()]
}
}
}

buildTypes {
release {
minifyEnabled false
proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-rules.pro'
}
}

compileOptions {
// Changed from 17 to 11 to fix jlink.exe crash on Windows (AGP 8.x known issue)
sourceCompatibility JavaVersion.VERSION_11
targetCompatibility JavaVersion.VERSION_11
}

buildFeatures {
viewBinding true
}
}

dependencies {
```

```
// AndroidX
implementation 'androidx.appcompat:appcompat:1.6.1'
implementation 'com.google.android.material:material:1.11.0'
implementation 'androidx.constraintlayout:constraintlayout:2.1.4'
implementation 'androidx.recyclerview:recyclerview:1.3.2'
implementation 'androidx.cardview:cardview:1.0.0'

// Lifecycle (ViewModel + LiveData)
implementation 'androidx.lifecycle:lifecycle-viewmodel:2.7.0'
implementation 'androidx.lifecycle:lifecycle-livedata:2.7.0'
implementation 'androidx.lifecycle:lifecycle-runtime:2.7.0'
annotationProcessor 'androidx.lifecycle:lifecycle-compiler:2.7.0'

// Room
implementation 'androidx.room:room-runtime:2.6.1'
annotationProcessor 'androidx.room:room-compiler:2.6.1'

// Navigation Component
implementation 'androidx.navigation:navigation-fragment:2.7.6'
implementation 'androidx.navigation:navigation-ui:2.7.6'

// Fragment
implementation 'androidx.fragment:fragment:1.6.2'

// Firebase
implementation 'com.google.firebase:firebase-auth:22.3.1'
implementation 'com.google.firebase:firebase-firestore:24.11.0'
}
```

Conclusion & Appendices

148 Self-Reflection

Building StorePilot was a comprehensive exercise in designing a production-grade Android application from scratch using MVVM architecture. The project challenged careful thinking about separation of concerns, thread safety, and user experience across two very different user types within a single codebase.

What Went Well

- The Repository pattern proved extremely valuable — swapping Room for Firestore would only require changing the repository layer.
- PBKDF2 password hashing was straightforward to implement with Java's javax.crypto and provides genuinely secure local credential storage.
- LiveData made the dashboard self-updating without any polling loops.
- Dual-role architecture (one APK, two separate UIs) was clean using a role check at login time.
- Demo seed data made testing the full order and chat flows much faster.
- Using switchMap in TaskListFragment elegantly solved the "stacking observers on tab change" bug.

Challenges Encountered

- Room's fallbackToDestructiveMigration wiped data during early schema iterations — a reminder that production apps need proper migrations.
- The ManagerOrderAdapter's spinner fired its listener on initial bind — solved by the firstTime flag guard.
- Firebase Firestore's async callback model clashed with Room's synchronous background-thread pattern.
- Supporting Android 13's POST_NOTIFICATIONS runtime permission required a VERSION_CODES.TIRAMISU check.

Future Improvements

- Replace fallbackToDestructiveMigration with versioned Room migrations.
- Integrate a real payment gateway (Stripe Android SDK).

- Add Glide or Coil for product image loading.
- Implement full Firestore sync for multi-device inventory.
- Add DiffUtil to adapters for smooth animated list updates.
- Add JUnit + Mockito tests for ViewModels and Room in-memory DAO tests.
- Implement Paging 3 for large product and order lists.

151 References (APA)

Android Developers. (2024). *Room persistence library*. Google. https://developer.android.com/training/data-storage/room

Android Developers. (2024). *LiveData overview*. Google. https://developer.android.com/topic/libraries/architecture/livedata

Android Developers. (2024). *ViewModel overview*. Google. https://developer.android.com/topic/libraries/architecture/viewmodel

Android Developers. (2024). *Navigation component*. Google. https://developer.android.com/guide/navigation

Android Developers. (2024). *Create and manage notification channels*. Google. https://developer.android.com/develop/ui/views/notifications/channels

Android Developers. (2024). *Request app permissions*. Google. https://developer.android.com/training/permissions/requesting

Firebase. (2024). *Firebase Authentication documentation*. Google. https://firebase.google.com/docs/auth/android/start

Firebase. (2024). *Cloud Firestore documentation*. Google. https://firebase.google.com/docs/firestore

NIST. (2017). *SP 800-63B — Digital Identity Guidelines*. https://doi.org/10.6028/NIST.SP.800-63b

OWASP. (2023). *Password storage cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Google. (2024). *Guide to app architecture*. https://developer.android.com/topic/architecture

Google. (2024). *Material Design 3 — Android components*. https://m3.material.io/develop/android/mdc-android

153 Appendix A — Permissions Map

PermissionManager — role to permission mapping

The table below maps each permission constant to the roles that hold it. PermissionManager is the single source of truth for this mapping.

Permission	OWNER	STORE_MGR	SHIFT_MGR	EMPLOYEE
MANAGE_USERS	v			
MANAGE_PRODUCTS	v	v		
VIEW_PRODUCTS	v	v	v	v
CREATE_SALE	v	v	v	v
VIEW_SALES_HISTORY	v	v	v	
MANAGE_PURCHASES	v	v		
VIEW_REPORTS	v	v		
MANAGE_SEASONS	v	v		
VIEW_MARKETING	v	v		
CREATE_PRIVATE_TASK	v	v	v	v
VIEW_TEAM_TASKS	v	v	v	
MANAGE_TASKS	v	v		
VIEW_ADMIN	v			

Note: CUSTOMER role uses a completely separate UI (CustomerMainActivity) and does not interact with PermissionManager.

154 Appendix B — Notification Channels

Channel ID	storepilot_low_stock
Channel Name	Low Stock Alerts
Importance	IMPORTANCE_HIGH (heads-up notification)
Vibration	Enabled
Description	Alerts when products are running low on stock
Notification ID	1001 (constant — replaces previous alert)
Trigger	LowStockReceiver.onReceive() or LowStockReceiver.checkNow()
Schedule	AlarmManager.setInexactRepeating — every 1 hour
Data source	Firestore collection "products" — quantity <= LOW_STOCK_THRESHOLD (5)
Android 13+	POST_NOTIFICATIONS runtime permission requested at MainActivity.onCreate
Test mode	Sends 1 demo notification if no real low-stock items found

155 Appendix C — Room Database Operations

Key SQL queries used across DAOs

Low stock products

```
SELECT * FROM products WHERE quantity <= :threshold ORDER BY quantity ASC
```

Top selling products (date range)

```
SELECT p.* FROM products p
INNER JOIN (
SELECT productId, SUM(quantity) as totalSold FROM sales
WHERE saleDate BETWEEN :startDate AND :endDate
GROUP BY productId ORDER BY totalSold DESC LIMIT 10
) s ON p.id = s.productId ORDER BY s.totalSold DESC
```

Today's sales total

```
SELECT SUM(totalPrice) FROM sales
WHERE saleDate >= :startOfDay AND saleDate < :endOfDay
```

Today's order revenue (delivered only)

```
SELECT COALESCE(SUM(totalPrice), 0) FROM orders
WHERE status = 'DELIVERED' AND createdAt >= :startOfDay
```

Today's order count

```
SELECT COUNT(*) FROM orders WHERE createdAt >= :startOfDay
```

Pending task count for user

```
SELECT COUNT(*) FROM tasks
WHERE assignedTo = :userId AND status != 'DONE'
```

Cart item count (badge)

```
SELECT COUNT(*) FROM cart_items WHERE customerId = :customerId
```

Unread support messages

```
SELECT COUNT(*) FROM support_messages
WHERE customerId = :customerId AND isRead = 0 AND senderRole = 'CUSTOMER'
```

Sales total by week

```
SELECT SUM(totalPrice) FROM sales
WHERE strftime('%Y-%W', saleDate/1000, 'unixepoch')
= strftime('%Y-%W', :refDate/1000, 'unixepoch')
```

Sales total by month

```
SELECT SUM(totalPrice) FROM sales
WHERE strftime('%Y-%m', saleDate/1000, 'unixepoch')
= strftime('%Y-%m', :refDate/1000, 'unixepoch')
```

156 Appendix D — Build & Deploy Note

Building a Debug APK

```
git clone https://github.com/ahmadhijazys2/storepilot.git
cd StorePilot
# Optional: add app/google-services.json for Firebase
./gradlew assembleDebug
# APK: app/build/outputs/apk/debug/app-debug.apk
adb install app/build/outputs/apk/debug/app-debug.apk
```

Building a Release APK

Create a keystore and configure signingConfigs.release in app/build.gradle, then run:

```
keytool -genkey -v -keystore storepilot.jks \
-alias storepilot -keyalg RSA -keysize 2048 -validity 10000

./gradlew assembleRelease
# AAB for Play Store:
./gradlew bundleRelease
```

Google Play Store Checklist

- minifyEnabled true + configure ProGuard rules
- Remove AppDatabase.seedDemoData() calls
- Replace fallbackToDestructiveMigration with versioned Room migrations
- Set up Firebase with production google-services.json
- Configure Firestore security rules for the products collection
- Review AndroidManifest for any unnecessary permissions
- Test on minimum SDK device (API 24)
- Run ./gradlew bundleRelease and upload .aab to Play Console

StorePilot · Ahmad Hijazy · 2025